

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Кафедра «Програмна інженерія та
інтелектуальні технології управління»

ЗВІТ
з лабораторної роботи № 3
з навчальної дисципліни
«WEB-ДОДАТКИ НА ОСНОВІ JAVA»

ВИКОНАВ

Студентка групи КН-222а

Інесса РЕПЕШКО

ПЕРЕВІРИВ

Доцент кафедри ПІІТУ

кандидат технічних наук

Владислав КОВТУН

Харків 2024

ДОСЛІДЖЕННЯ ПРИНЦИПІВ СТВОРЕННЯ СКЛАДНИХ ПРОЄКТІВ ІЗ ЗАСТОСУВАННЯМ SPRING BOOT, SPRING DATA, THYMELAF

Мета

- 1 Набути навичок створення Spring-проєктів та реалізації CRUD із застосуванням можливостей даного фреймворку.
- 2 Набути навичок створення репозитаріїв та сервісів, а також створення методів для витягу даних.
- 3 Набути навичок тестування методів доступу до баз даних при застосуванні фреймворку Spring.
- 4 Набути навичок створення Thymeleaf-сторінок для відображення даних з таблиць, а також для додавання та редагування даних.
- 5 Набути навичок створення методів контролерів для обробки запитів до серверу.
- 6 Удосконалити навички розробки user-friendly застосунків.

Завдання роботи

Розробити Spring-застосунок, що відображає на вебсторінках інформацію, яка міститься у двох пов'язаних таблицях бази даних, а також забезпечує виконання операцій CRUD із ними.

Застосунок має забезпечити:

- відображення вмісту головної таблиці на вебсторінці та управління відображенням вмісту сторінки (пошук, фільтрація, сортування);
- додавання записів до головної таблиці та їх редагування, а також вилучення із врахуванням встановленого зовнішнього ключа між таблицями та специфіки предметної області;
- відображення вмісту зв'язаної таблиці та управління відображенням вмісту сторінки (пошук, фільтрація, сортування);

– додавання, редагування та видалення записів із зв'язаної таблиці із урахуванням встановленого зовнішнього ключа між таблицями та специфіки даних, що в ній зберігаються.

Нефункціональною вимогою до програмного забезпечення є обов'язкова реалізація контролю повноти та коректності даних, що вводиться, враховуючи специфіку предметної області та типів полів, які створені для зберігання даних.

Реалізацію даної лабораторної роботи можна поділити на декілька завдань описаних нижче.

1 Створити та налаштувати Spring Boot проєкт. Реалізувати шар бізнес-логіки (класи репозиторію, сервісу та імплементації сервісу) та тестування методів CRUD для першої сутності індивідуального завдання.

2 Реалізувати шар вебінтерфейсу (HTML сторінки та контролери до них) для першої сутності та основних сторінок користувацького інтерфейсу. У даному пункті також необхідно реалізувати наступний функціонал для першої сутності: сортування, фільтрації та пошуку даних на таблиці, а також додавання, редагування та видалення даних з таблиці.

3 Створити entity-клас другої сутності індивідуального завдання. Реалізувати шар бізнес-логіки (класи репозиторію, сервісу та імплементації сервісу) та тестування методів CRUD для другої сутності індивідуального завдання.

4 Реалізувати шар вебінтерфейсу (HTML сторінки та контролери до них) для другої сутності. У даному пункті також необхідно реалізувати наступний функціонал для другої сутності: сортування, фільтрації та пошуку даних на таблиці, а також додавання, редагування та видалення даних з таблиці з урахуванням особливостей специфіки відношення між двома сутностями.

Умови варіанту № 1 індивідуального завдання для реалізації у даній лабораторній роботі представлені на рис. 1.

Варіант №1 (ЗВО №1)			
Опис структури полів класу «Abiturient» (суттєвість 1)			
Ім'я поля	Тип даних	Опис	Властивості поля
id	Long	Сурогатний ключ	PK
p_code	String	Номер особової справи	Маска номеру "KH23-NNNN", унікальне для кожного студента
surname	String	Прізвище	Not null
name	String	Ім'я	Not null
pname	String	По-батькові	Not null
birthday	LocalDate	Дата народження	Not null
gender	boolean	Стать	Not null; true – чоловіча, false - жіноча
rating	double	Рейтинговий бал	Not null від 120.01 до 200.00

Опис структури полів класу «StudentDopInfo» (суттєвість 2)			
Ім'я поля	Тип даних	Опис	Властивості поля
id	Long	Сурогатний ключ	PK
student	Abiturient	Студент-власник особистої інформації	@OneToOne (FK до Student)
contract	boolean (можна String)	Контракт чи бюджет?	Not null. Перелік значень: «контракт»; «бюджет»
scholarship	boolean (можна String)	Є стипендія чи немає?	Not null. Перелік значень: «стипендія»; «ні». Якщо студент на контракті, то встановлюється значення «ні».
email	String	Адреса у корпоративній пошті НТУ «ХПІ»	Not null. Unique. Складається з транскрипції імені та прізвища студента, домен - @cs.khpi.edu.ua

Рисунок 1 – Умова варіанту № 1 індивідуального завдання

Хід роботи

1 Створення Spring-проєкту, реалізація шару бізнес-логіки та тестування методів CRUD для першої сутності

Створення та налаштування проєкту. Створення нового Spring-проєкту в IntelliJ IDEA та його налаштування може виконуватись наступними кроками.

- 1 Запустити IntelliJ IDEA і оберіть “New Project”.
- 2 Налаштувати основні параметри проєкту (рис. 2).
- 3 Натиснути “Next” і вибрати версію платформи та залежності для проєкту (рис. 3-4).
- 4 Після натискання “Create” проєкт буде створено і відкрито в IntelliJ IDEA зі стандартною структурою.

Примітка. Налаштування на рисунках 2-4 є рекомендованими, але можна використовувати інші комбінації, переконавшись у сумісності компонентів.

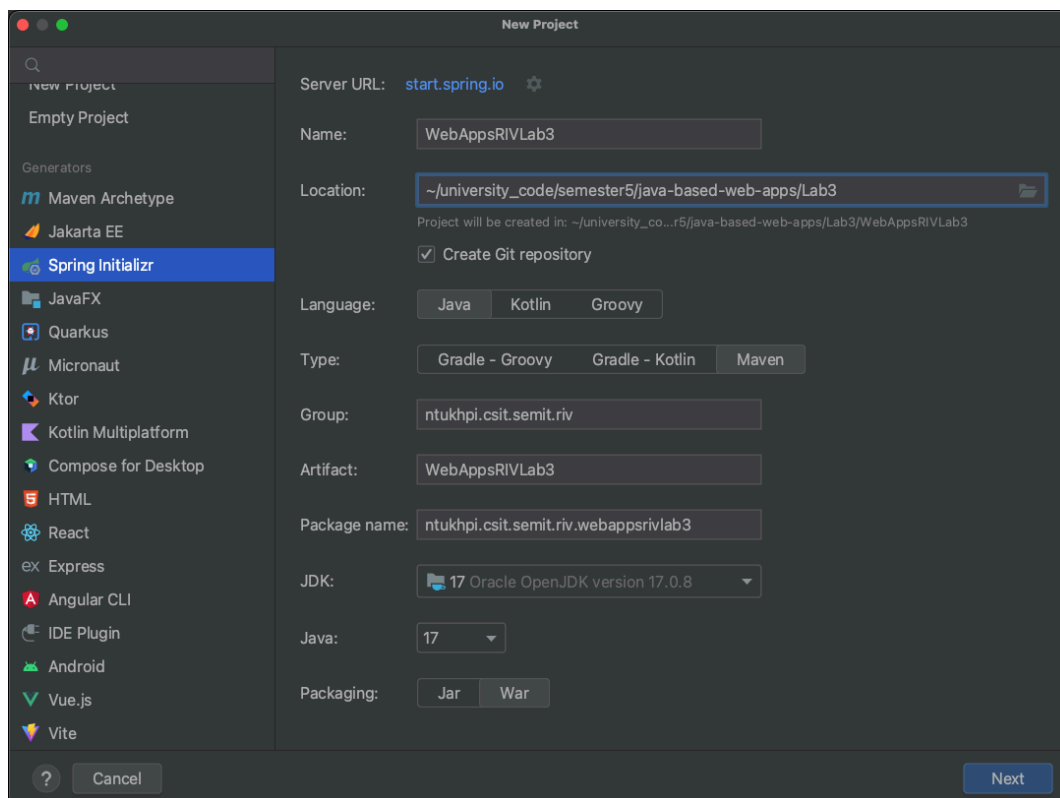


Рисунок 2 – Налаштування основних параметрів Spring-проєкту в IntelliJ IDEA

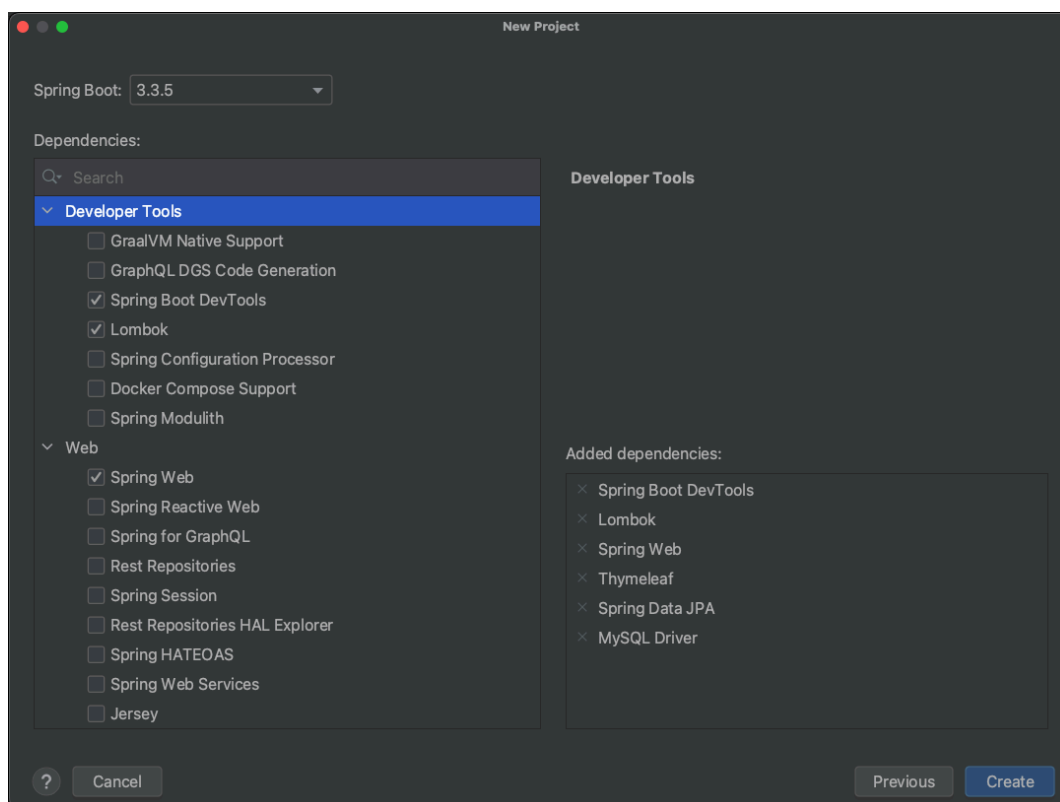


Рисунок 3 – Сторінка вибору версії платформи та залежностей проєкту в IntelliJ IDEA частина 1

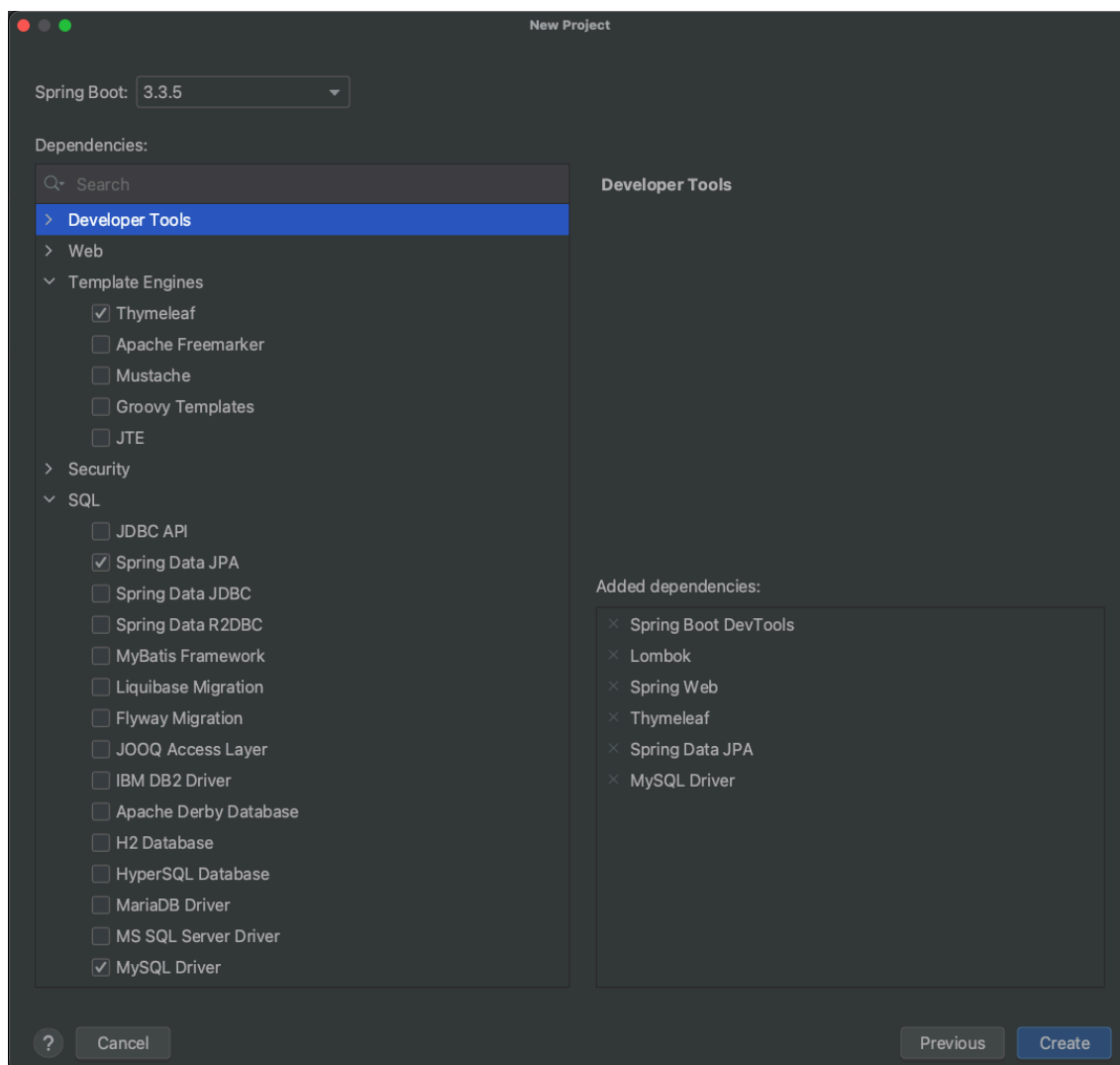


Рисунок 4 – Сторінка вибору версії платформи та залежностей проєкту в IntelliJ IDEA частина 2

Також в процесі реалізації завдань лабораторної роботи були додані наступні залежності до `pom.xml` файлу, що відображено на рис. 5 далі в тексті роботи. У додатку A.1 представлений файл `pom.xml` конфігурації проєкту, що містить всі залежності реалізованого проєкту.

Архітектура проєкту. У даному Spring Boot проєкті реалізовано багатoshарову архітектуру (Multi-layered Architecture) що представлена на рис. 6-7 далі в тексті роботи. Структура включає різні пакети для контролерів, моделей, репозиторіїв, сервісів та шаблонів.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.15.0</version>
</dependency>

<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.4.12</version>
</dependency>

```

Рисунок 5 – Додані в процесі розробки залежності у pom.xml файл

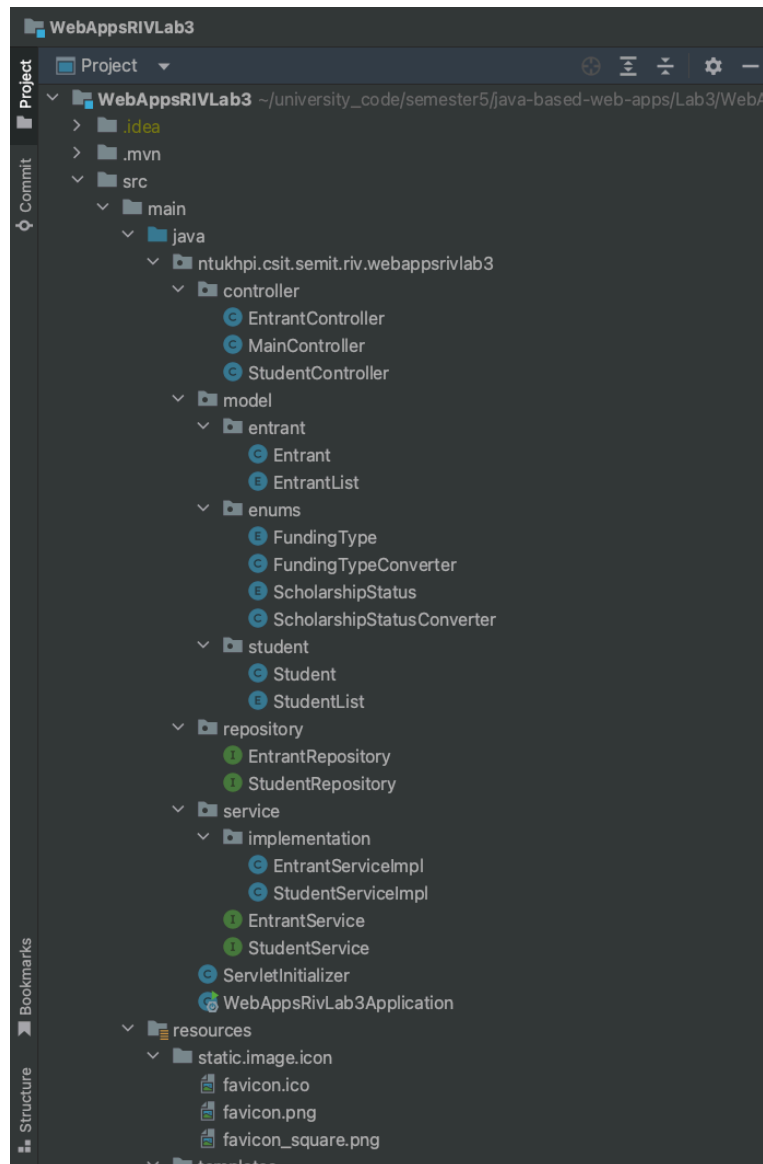


Рисунок 6 – Структура проекту WebAppsRIVLab3 частина 1

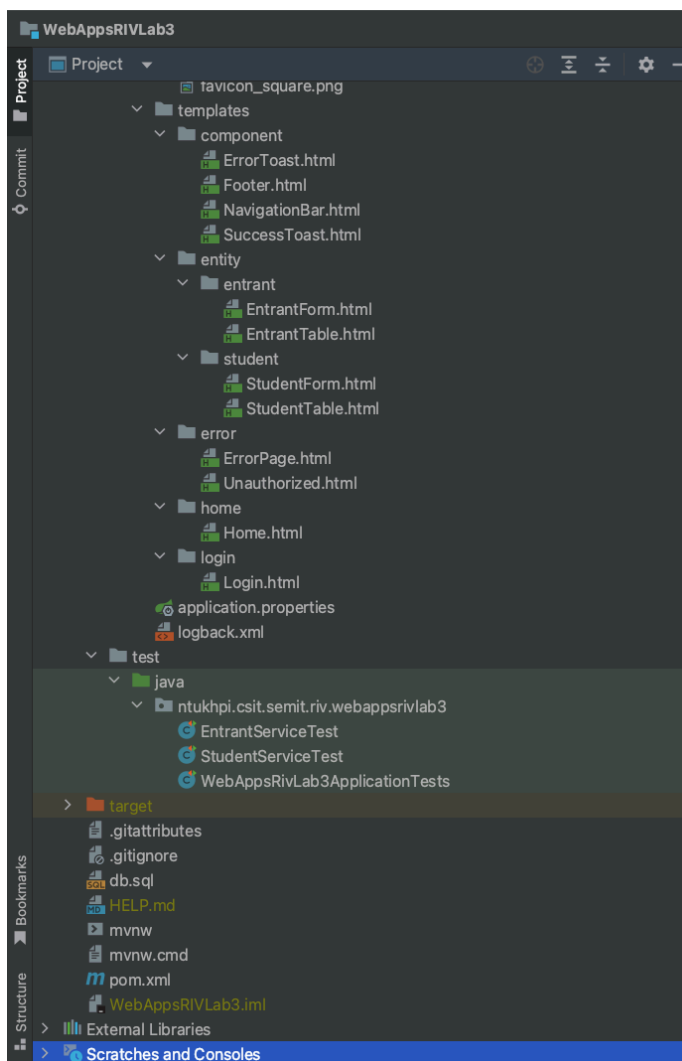


Рисунок 7 – Структура проєкту WebAppsRIVLab3 частина 2

Основні шари проєкту можна визначити наступним чином описаним нижче.

1 Шар контролерів (Controller Layer). Пакет “controller” містить класи контролерів, які обробляють HTTP-запити та керують взаємодією з користувачем:

- 1) клас EntrantController для обробки запитів, пов’язаних із сутністю Entrant;
- 2) клас MainController для основних запитів або початкової сторінки;
- 3) клас StudentController для обробки запитів, пов’язаних із сутністю Student.

2 Шар моделей (Model Layer). Пакет “model” містить класи моделей даних та переліки (enum) для визначення можливих типів значень:

1) пакет “entrant” містить:

- а) клас Entrant для представлення сутності абітурієнта;
- б) клас EntrantList для представлення списку об’єктів Entrant;

2) пакет “enums” включає переліки та конвертери для специфічних типів даних:

- а) класи FundingType та ScholarshipStatus, які описують можливі значення типу фінансування та статусу стипендії;
- б) класи конвертери FundingTypeConverter та ScholarshipStatusConverter для роботи з переліками в базі даних;

3) пакет “student” містить:

- а) клас Student клас для представлення сутності студента;
- б) клас StudentList представлення списку об’єктів Student.

3 Шар репозиторіїв (Repository Layer). Пакет “repository” містить інтерфейси репозиторіїв, які відповідають за доступ до бази даних:

1) клас EntrantRepository для роботи з таблицею, що зберігає дані про абітурієнтів;

2) клас StudentRepository для роботи з таблицею, що зберігає дані про студентів.

4 Шар сервісів (Service Layer). Пакет “service” містить інтерфейси сервісів та їхні імплементації для бізнес-логіки:

1) інтерфейси EntrantService та StudentService визначають основні методи для обробки логіки, пов’язаної із сутностями абітурієнта (Entrant) та студента (Student);

2) пакет “implementation” містить конкретні реалізації сервісів, а саме класи EntrantServiceImpl та StudentServiceImpl реалізують сервіси, що містять бізнес-логіку для обох сутностей (Entrant та Student).

5 Презентаційний шар (Presentation Layer). Папка “resources” містить наступні ресурси проєкту:

- 1) папка “static.image.icon” містить файли іконок та фівіконів;
- 2) папка “templates” містить HTML-шаблони для різних компонентів:

а) папка “component” містить шаблони компонентів сторінок: повідомлення про помилку (ErrorToast.html), повідомлення про успіх (SuccessToast.html), футер (Footer.html), хедер (NavigationBar.html);

б) папка “entity” містить шаблони для сутностей:

– папка “entrant” містить шаблони для сторінок із формою створення, редагування та видалення абітурієнтів (EntrantForm.html) та сторінки з таблицею даних абітурієнтів (EntrantTable.html);

– папка “student” містить шаблони для сторінок із формою створення, редагування та видалення студентів (StudentForm.html) та сторінки з таблицею даних студентів (StudentTable.html);

в) папка “error” містить універсальний шаблон для сторінок з помилками (ErrorPage.html) та сторінку помилки авторизації (Unauthorized.html);

г) папка “home” містить шаблон Home.html для головної сторінки вебзастосунку;

д) папка “login” містить шаблон Login.html для сторінки авторизації.

Реалізований проєкт також містить супровідні компоненти описані нижче.

1 Запуск Spring Boot вебзастосунку. Наступні класи важливі для конфігурації та запуску Spring Boot застосунку:

1) клас WebAppsRivLab3Application є основним класом, який запускає застосунок, виконує початкову логіку для збереження Entrant та Student в базу даних і логування даних через CommandLineRunner (код файлу представлений у додатку А.4);

2) клас `ServletInitializer` забезпечує можливість розгортання вашого Spring Boot застосунку як традиційного вебзастосунку (WAR-файл) у контейнері сервлетів (код файлу представлений у додатку A.5).

2 Тести. Папка “test” містить JUnit тести для перевірки функціональності сервісів та основного застосунку:

1) класи `EntrantServiceTest` та `StudentServiceTest` реалізують тестування CRUD операцій у сервісах обох сутностей `EntrantService` та `StudentService`;

2) клас `WebAppsRivLab3ApplicationTests` реалізує тестування загальної функціональності застосунку.

3 Конфігурація та допоміжні файли:

1) конфігураційний файл `application.properties` визначає налаштування для Spring Boot проєкту (код файлу представлений у додатку A.2);

2) конфігураційний файл `logback.xml` визначає для налаштування логування (код файлу представлений у додатку A.3);

3) конфігураційний файл `Maven pom.xml` для налаштування залежностей проєкту (код файлу представлений у додатку A.1).

Схема бази даних. Схема створеної бази даних на основі DBMS MySQL відображена на рис. 8 далі в тексті роботи. Структура створених таблиць `entrants` та `students` наведена на рис. 9 та рис. 10 далі в тексті роботи.

З минулої лабораторної роботи у даний проєкт було перенесено наступні файли для реалізації даного завдання.

Клас `Entrant.java`. Клас `Entrant` є сутністю, що представляє запис у таблиці “entrants” бази даних. Він зберігає таку інформацію про вступника:

- унікальний ідентифікатор запису в базі даних (Long);
- номер особової справи вступника (String);
- прізвище вступника (String);
- ім’я вступника (String);
- по-батькові вступника (String);

columns (entrants)											
	Name	Comment	Data Type	Not Null	Auto Increment	Column Kind	Default Expression	Hidden	On Update	Position	Collation
1	birthday		date	true	36	NORMAL	'2000-01-01'	false		1	null
2	gender		bit(1)	true		NORMAL	b'1'	false		2	null
3	rating_score		double	true		NORMAL	120.001	false		3	null
4	id		bigint	true		NORMAL		false		4	null
5	student_id		bigint	false		NORMAL		false		5	null
6	case_number		varchar(20)	true		NORMAL		false		6	null
7	name		varchar(50)	true		NORMAL		false		7	null
8	patronymic		varchar(50)	true		NORMAL		false		8	null
9	surname		varchar(50)	true		NORMAL		false		9	null

keys (entrants)			
	Name	Primary	
1	PRIMARY	true	
2	UKj36ja5h4xym91tmpdw81xno84	false	

indexes (entrants)				
	Name	Comment	Unique	Type
1	PRIMARY		true	btree
2	UKj36ja5h4xym91tmpdw81xno84		true	btree

checks (entrants)	
Name	Predicate
1	entrants_chk_2 ('rating_score' > 120.000) and ('rating_score' <= 200.000)
2	entrants_chk_1 'birthday' between _utf8mb4\1914-01-01\ and _utf8mb4\2008-01-01\
3	regexp_like('case_number', '_utf8mb4\'^[0DCS-00000`0DCS0IND0SSA0ESA]{1,10}[0-9]{2}-[0-9]{1,4}\$\'', _utf8mb4\`c\`) = 1
4	entrants_chk_4 regexp_like('name', '_utf8mb4\`^?(=. {1,50}\$)[0DCS-00000`0DCS0IND0SSA0ESA](?:[0DCS-0`0DCS0IND0SSA0ESA0`-NSS30PUINCCHNSPAN0PA0X])*[0DCS-0`0DCS0I
5	entrants_chk_5 regexp_like('patronymic', '_utf8mb4\`^?(=. {1,50}\$)[0DCS-00000`0DCS0IND0SSA0ESA](?:[0DCS-0`0DCS0IND0SSA0ESA0`-NSS30PUINCCHNSPAN0PA0X])*[0DCS-0`0DCS-0`0DCS0I
6	entrants_chk_6 regexp_like('surname', '_utf8mb4\`^?(=. {1,50}\$)[0DCS-00000`0DCS0IND0SSA0ESA](?:[0DCS-0`0DCS0IND0SSA0ESA0`-NSS30PUINCCHNSPAN0PA0X])*[0DCS-0`0DCS-0`0DCS0I

Рисунок 9 – Структура таблиці entrants

columns (students)											
Name	Comment	Data Type	Not Null	Auto Increment	Column Kind	Default Expression	Hidden	On Update	Position	Collation	
1 entrant_id		bigint	false		NORMAL		false		1	null	
2 id		bigint	true	31	NORMAL		false		2	null	
3 corporate_email		varchar(123)	true		NORMAL		false		3	null	
4 funding_type		enum('BUDGET', 'CONTRACT')	true		NORMAL	'CONTRACT'	false		4	null	
5 scholarship_status		enum('ENHANCED', 'NONE', 'ORDINARY')	true		NORMAL	'NONE'	false		5	null	

keys (students)		
Name	Primary	
1 PRIMARY	true	
2 UKebgu37Lcg62sdwu3n7d7gre0v	false	
3 UKng7nfmdg9Laddmqsl1be38mkx	false	

foreign keys (students)			
Name	Target Table	On Delete	On Update
1 fk_student_entrant	entrants	cascade	cascade

indexes (students)				
Name	Comment	Unique	Type	
1 UKebgu37Lcg62sdwu3n7d7gre0v		true	btree	
2 PRIMARY		true	btree	
3 UKng7nfmdg9Laddmqsl1be38mkx		true	btree	

checks (students)	
Name	Predicate
1 students_chk_1	regexp_like('corporate_email', '_utf8mb4'[a-z](.?[a-z]*){1,99}@[a-z]{1,10}.khi.edu.ua\$', '_utf8mb4'c')

Рисунок 10 – Структура таблиці students

- дата народження вступника (LocalDate);
- стать вступника (Boolean);
- рейтинговий бал вступника (Double);
- унікальний ідентифікатор запису сутності студента пов’язаного з даним абітурієнтом з таблиці “students” (Long).

Клас використовує Hibernate для ORM (Object-Relational Mapping) і включає валідаційні обмеження для різних полів, таких як номер справи, складові імені та оцінка рейтингу.

Основні особливості включають валідацію полів для дотримання правильного формату, такі як для номера справи та складових імені. Код класу Entrant представлений у додатку А.6.

Клас EntrantList.java. Клас EntrantList реалізує шаблон Singleton для зберігання списку об’єктів типу Entrant (абітурієнтів). Він успадковує функціональність від класу ArrayList та забезпечує доступ до єдиного екземпляра цього списку впродовж життєвого циклу застосунку.

Клас ініціалізує список абітурієнтів із заздалегідь визначеними даними, що використовуються для тестування та демонстрації функціональності. EntrantList служить для моделювання бази даних, що полегшує роботу з колекцією даних у проєкті. Код класу EntrantList представлений у додатку А.7.

Результатом виконання даного завдання є наступні створені класи для реалізації шару бізнес-логіки та тестування методів CRUD для першої сутності.

Інтерфейс EntrantRepository.java. EntrantRepository є інтерфейсом для доступу до даних про об'єкти Entrant у базі даних і реалізує JpaRepository, що забезпечує базові CRUD-операції. Він містить спеціалізовані методи:

- отримання списку абітурієнтів за повним ім'ям, фільтрації абітурієнтів за різними параметрами (номер справи, дати народження, рейтинг) з підтримкою сортування;
- вибірка абітурієнтів, які не мають відповідного запису в таблиці студентів;
- пошук ідентифікатора студента за прив'язаним до цього ідентифікатором абітурієнта.

Клас анотовано як @Repository, що позначає його як компонент для зберігання даних у Spring. Код реалізованого інтерфейсу EntrantRepository.java представлений у додатку А.8.

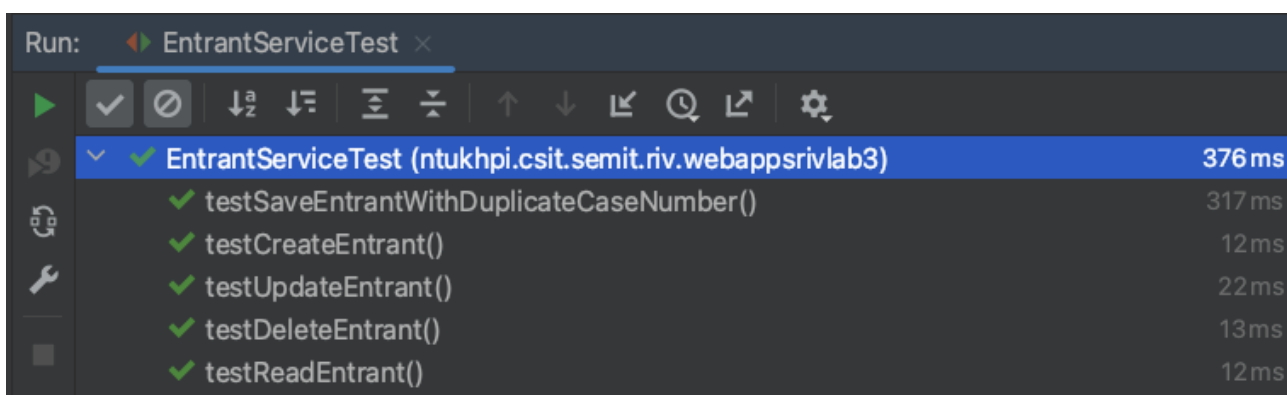
Інтерфейс EntrantService.java. Інтерфейс EntrantService визначає набір бізнес-логіки для роботи з об'єктами Entrant, включаючи методи для отримання всіх абітурієнтів, фільтрації та сортування за різними критеріями, пошуку абітурієнтів без відповідних записів у таблиці студентів, збереження списку абітурієнтів, пошуку абітурієнта за id або за прикладом, отримання id студента за id абітурієнта, а також для оновлення та видалення абітурієнтів. Цей інтерфейс служить контрактом для сервісного шару і забезпечує чітке розмежування бізнес-логіки та доступу до даних. Код реалізованого інтерфейсу EntrantService.java представлений у додатку А.9.

Клас EntrantServiceImpl.java. Клас EntrantServiceImpl є реалізацією сервісу для роботи з об'єктами Entrant у Spring-застосунку. Він позначений як @Service, що робить його компонентом Spring для бізнес-логіки. Він реалізує інтерфейс EntrantService та забезпечує методи для отримання всіх абітурієнтів, фільтрації та сортування за різними критеріями, пошуку абітурієнтів без пов'язаних записів у таблиці студентів, збереження, оновлення та видалення

записів. Клас використовує `EntrantRepository` для виконання операцій із базою даних і включає детальне логування для відстеження процесу виконання операцій. Обробка помилок включає перевірку цілісності даних, порушення унікальності, версійну конфліктність, що забезпечує надійність сервісного шару. Методи `saveEntrant`, `updateEntrant`, і `deleteEntrant` мають анотацію `@Transactional`, що забезпечує відкат змін у разі виникнення винятку. Код реалізованого класу `EntrantServiceImpl.java` представлений у додатку А.10.

Тест `EntrantServiceTest.java`. Клас `EntrantServiceTest` є тестовим класом для перевірки методів сервісу `EntrantService` у Spring-застосунку. Він анотований як `@SpringBootTest`, що забезпечує підняття контексту Spring для інтеграційного тестування. Клас містить кілька тестових методів, кожен з яких перевіряє окрему функціональність сервісу: створення абітурієнта (`testCreateEntrant`), пошук за ідентифікатором (`testReadEntrant`), оновлення даних (`testUpdateEntrant`), видалення абітурієнта (`testDeleteEntrant`) та обробку дублювання номера справи (`testSaveEntrantWithDuplicateCaseNumber`). Метод `setUp` ініціалізує тестові дані, а `reset` видаляє їх після виконання кожного тесту, щоб забезпечити ізолюваність тестових випадків. Код реалізованого тесту `EntrantServiceTest.java` представлений у додатку А.11.

Результати виконання тестових випадків набору тестів `EntrantServiceTest` представлені на рис. 11.



Run: EntrantServiceTest	
✓	EntrantServiceTest (ntukhpi.csit.semit.riv.webappsrvlab3) 376 ms
✓	testSaveEntrantWithDuplicateCaseNumber() 317 ms
✓	testCreateEntrant() 12 ms
✓	testUpdateEntrant() 22 ms
✓	testDeleteEntrant() 13 ms
✓	testReadEntrant() 12 ms

Рисунок 11 – Результати виконання тестового набору `EntrantServiceTest.java`

2 Реалізація шару вебінтерфейсу для першої сутності та основних сторінок користувацького інтерфейсу

Результатом виконання даного завдання є реалізовані сторінки авторизації, помилки авторизації, домашня сторінка вебзастосунку, таблиця з даними першої сутності та шаблону з формою додавання, редагування та видалення запису з таблиці, а також контролери до них.

Сторінка Login.html. Ця сторінка входу реалізована з використанням Thymeleaf та Bootstrap 5, що забезпечує адаптивний та сучасний інтерфейс. Верхня частина сторінки включає заголовок "Student Database Viewer" з піктограмою (favicon), а також елемент ErrorToast, що виводить повідомлення про помилки, завантажене через Thymeleaf (th:replace). Основна форма для входу містить поля для введення логіна та пароля, обгорнуті в стилізовані елементи input-group з іконками. Кнопка входу відображає іконку входу з використанням bootstrap-icons. У футері використовується компонент Footer, також завантажений через Thymeleaf. Для забезпечення кросбраузерної сумісності та інтерактивності додано зовнішні бібліотеки Bootstrap CSS і JS. Код даної сторінки представлений у додатку Б.1.

Сторінка Unauthorized.html. Сторінка "Unauthorized Error" використовується для інформування користувача про невдалу авторизацію у випадку введення некоректних облікових даних. Вона побудована за допомогою Thymeleaf та Bootstrap 5, забезпечуючи адаптивний і естетичний інтерфейс. У центрі сторінки відображається картка з повідомленням про помилку 401 та текстом, що інформує користувача про причину відмови у доступі. Додано кнопку для повернення на попередню сторінку (window.history.back()), стилізовану з іконкою з бібліотеки bootstrap-icons. У футері використовується компонент Footer, підключений через Thymeleaf. Для зовнішнього вигляду та інтерактивності сторінки підключені стилі та скрипти Bootstrap. Код даної сторінки представлений у додатку Б.2.

Сторінка Home.html. Сторінка "Home" побудована з використанням Thymeleaf та Bootstrap 5, надаючи зручний інтерфейс користувача для роботи з

базою даних студентів. У верхній частині сторінки є привітання, яке персоналізується залежно від стану авторизації користувача. Основний вміст складається з кількох карток, які представляють інформацію про розробника, а також розділи "Entrant Table" та "Student Table" з описом доступних даних і кнопками для переходу до відповідних таблиць. Стилізовані плейсхолдери використовуються для додаткових карток, які можуть слугувати місцем для майбутнього контенту. Компоненти `NavigationBar` і `Footer` включені через `Thymeleaf` для створення загального вигляду і структури сторінки. Код даної сторінки представлений у додатку Б.3.

Компонент `NavigationBar.html`. Цей компонент містить навігаційний бар, який дозволяє користувачеві переходити між основними розділами додатку, такими як "Home", "Tables" та "Forms". Меню реалізоване з використанням `Bootstrap` і включає випадаючі списки для швидкого доступу до таблиць та форм для вступників і студентів. Бар також містить заголовок "Student Database Viewer" з піктограмою. Код даного компонента представлений у додатку Б.4.

Компонент `Footer.html`. Цей компонент містить інформацію про автора, установу, адресу та контактні дані. Футер має секції "Powered by", "Address" та "Contact Us", що відображають ім'я автора, навчальну групу, установу (НТУ "ХПІ"), а також контактний номер і електронну пошту. Дизайн футера адаптивний, створений з використанням `Bootstrap`, і відповідає стилю решти застосунку. Код даного компонента представлений у додатку Б.5.

Компонент `ErrorToast.html`. Цей компонент відображає сповіщення про помилку. Він використовує `Thymeleaf`, щоб умовно показати попередження (якщо змінна `errorMessage` не порожня) з іконкою та повідомленням про невдалу дію. Сповіщення реалізовано з використанням `Bootstrap` і має кнопку для закриття, що робить його зручним для користувача. Код даного компонента представлений у додатку Б.6.

Компонент `SuccessToast.html`. Цей компонент відповідає за відображення повідомлення про успішну дію, якщо `successMessage` не порожня. Сповіщення містить іконку успішного виконання, кнопку закриття та текст, що інформує

користувача про успіх операції (створення, оновлення чи видалення). Використання Bootstrap забезпечує його інтерактивність та привабливий дизайн. Код даного компонента представлений у додатку Б.7.

Сторінка `ErrorPage.html`. Дана сторінка використовується для відображення повідомлень про помилки в застосунку. Вона налаштована за допомогою Thymeleaf, який динамічно замінює значення полів `errorTitle`, `errorCode`, `errorMessage`, і `errorException`. Якщо код помилки відрізняється від 401 (помилки авторизації), на сторінці відображається компонент навігації `NavigationBar`. У центрі сторінки розміщена картка з кодом помилки та описом проблеми, стилізована з використанням Bootstrap. Додатково, в залежності від коду помилки, надаються кнопки для повернення на попередню сторінку (для 401) або перехід на головну сторінку. Футер додається з використанням компонента `Footer` для створення загального вигляду. Код даної сторінки представлений у додатку Б.8.

Сторінка `EntrantTable.html`. Сторінка `EntrantTable.html` відображає таблицю вступників з можливістю пошуку, фільтрації та сортування даних. Використовуючи Thymeleaf і Bootstrap, вона забезпечує зручний інтерфейс для взаємодії з даними про вступників. Ліва панель містить елементи для пошуку за номером справи або повним ім'ям, а також фільтри за датою народження та рейтингом. Центральна таблиця динамічно заповнюється даними про вступників і підтримує сортування по різних полях. Таблиця містить інтерактивні кнопки для перегляду пов'язаного запису студента, редагування та видалення записів абітураєнта. Компоненти `ErrorToast` і `SuccessToast` відображають повідомлення про статус операцій. Додатково, JavaScript забезпечує валідацію полів вводу та обробку параметрів URL для реалізації пошуку, фільтрації та сортування на серверній стороні застосунку. Код даної сторінки представлений у додатку Б.9.

Шаблон `EntrantForm.html`. Шаблон `EntrantForm` реалізований з використанням Thymeleaf та HTML для створення форми введення даних абітурієнта. Заголовок сторінки, вміст полів та доступ до полів форми

змінюються залежно від режиму: створення, перегляд, оновлення чи видалення запису. Форма забезпечує зручний інтерфейс для заповнення форми з полями, такими як номер справи, прізвище, ім'я, по батькові, дата народження, стать та рейтинг. Для кожного поля передбачена валідація, включаючи перевірку формату даних і діапазонів значень. Кнопки управління дозволяють зберегти, скинути або видалити запис. JavaScript забезпечує додаткову валідацію та підтвердження перед видаленням запису. Повідомлення про успіх або помилку відображаються через компоненти `ErrorToast` і `SuccessToast`. Код даної шаблону представлений у додатку Б.10.

Контролер `MainController.java`. Клас `MainController` є контролером у Spring-застосунку, який керує маршрутизацією сторінок для авторизації та доступу до основних розділів. Він визначає статичні значення для логіна та пароля для перевірки входу користувача. Метод `showStartPage` (GET запит на `/`) відображає сторінку входу. Метод `processLogin` (POST запит на `/process-login`) перевіряє введені логін та пароль. Якщо дані правильні, користувач перенаправляється на головну сторінку (`/home`), а його ім'я користувача додається в сесію. У разі помилкових даних користувача перенаправляють на сторінку помилки (`/unauthorized-error`) з повідомленням про невдалу авторизацію. Метод `showHomePage` показує головну сторінку, а `showUnauthorizedErrorPage` відображає сторінку помилки з відповідним кодом і повідомленням про помилку. Код даного класу представлений у додатку Б.11.

Клас `EntrantController.java`. Даний клас відповідає за обробку HTTP-запитів для керування об'єктами `Entrant` у вебзастосунку на основі Spring. Він забезпечує маршрутизацію для створення, оновлення, видалення та відображення таблиці вступників. Методи включають `showEntrantTable` для відображення таблиці з можливістю фільтрації та сортування, `showCreateEntrantForm`, `showUpdateEntrantForm`, і `showDeleteEntrantForm` для відображення відповідних форм на основі шаблону `StudentForm.html`. Метод `saveEntrant` обробляє POST-запити для створення запису, `updateEntrant` – для оновлення, а `deleteEntrant` – для видалення. Контролер використовує валідацію

введених даних та обробку можливих помилок, що відображаються через сторінку помилок або компоненти повідомлень. Вся інформація динамічно додається до моделі, що забезпечує інтерактивність і зручність роботи користувача з інтерфейсом. Код даного класу представлений у додатку Б.12.

Результати роботи функціоналу вебзастосунку реалізованого до завдання 1 та 2 даної лабораторної роботи наведені на рис. Б.13.1-Б.13.13 у додатку Б.13.

3 Реалізація шару бізнес-логіки та тестування методів CRUD для другої сутності

З минулої лабораторної роботи у даний проєкт було перенесено наступні файли для реалізації даного завдання.

Клас Student.java. Клас Student представляє сутність студента, який пройшов процес вступу і зберігає таку інформацію про студента:

- унікальний ідентифікатор запису в базі даних (Long);
- пов'язані дані абітурієнта з особистою інформацією студента (Entrant);
- тип фінансування навчання студента (FundingType);
- статус нарахування стипендії студента (ScholarshipStatus);
- корпоративна пошта студента (String).

Клас пов'язаний з таблицею students у базі даних і використовує анотації JPA та Hibernate для визначення зв'язків і обмежень, а також включає валідаційні обмеження для різних полів, таких як переліки та корпоративна пошта.

Основні атрибути включають entrant – зв'язок "один до одного" з об'єктом Entrant, fundingType – тип фінансування (контракт чи бюджет), та scholarshipStatus – статус стипендії (звичайна, підвищена або відсутня). Поле corporateEmail має специфічний формат і унікальність, що перевіряється регулярним виразом. Клас підтримує автоматичне оновлення або видалення пов'язаного запису в таблиці entrants при відповідних змінах. Код класу Student представлений у додатку В.1.

Клас StudentList.java. Клас StudentList реалізує шаблон Singleton для зберігання списку об'єктів типу Student (студентів). Він успадковує функціональність від класу ArrayList та забезпечує доступ до єдиного екземпляра цього списку впродовж життєвого циклу застосунку.

Клас ініціалізує список студентів із заздалегідь визначеними даними, що використовуються для тестування та демонстрації функціональності. StudentList служить для моделювання бази даних, що полегшує роботу з колекцією даних у проєкті. Код класу StudentList представлений у додатку В.2.

Результатом виконання даного завдання є наступні створені класи для реалізації шару бізнес-логіки та тестування методів CRUD для другої сутності.

Інтерфейс StudentRepository.java. Інтерфейс StudentRepository є репозиторієм Spring Data JPA, який надає методи для доступу до даних про студентів у базі даних. Він розширює JpaRepository<Student, Long>, забезпечуючи базові CRUD-операції для сутності Student. Крім того, StudentRepository містить спеціалізований метод findFilteredAndSortedStudents, що дозволяє отримувати список студентів з фільтрацією та сортуванням. Метод підтримує фільтрацію за прізвищем, іменем та по батькові студента, типом фінансування (fundingType), а також списком статусів стипендії (scholarshipStatuses). Для зручності сортування використовується параметр Sort. Це дозволяє гнучко вибирати студентів відповідно до зазначених критеріїв. Код реалізованого інтерфейсу StudentRepository.java представлений у додатку В.3.

Інтерфейс StudentService.java. Інтерфейс StudentService визначає контракт для сервісного шару, який забезпечує бізнес-логіку для роботи з об'єктами Student. Він містить методи для отримання всіх студентів, фільтрації та сортування студентів за різними критеріями (пошук за іменем, типом фінансування та статусом стипендії), збереження списку студентів, пошуку студента за id або за прикладом, а також для створення, оновлення та видалення студентів. Цей інтерфейс забезпечує чітке розмежування бізнес-логіки від рівня доступу до даних, дозволяючи реалізувати його через відповідний сервіс, який

взаємодіє з репозиторієм. Код реалізованого інтерфейсу `StudentService.java` представлений у додатку В.4.

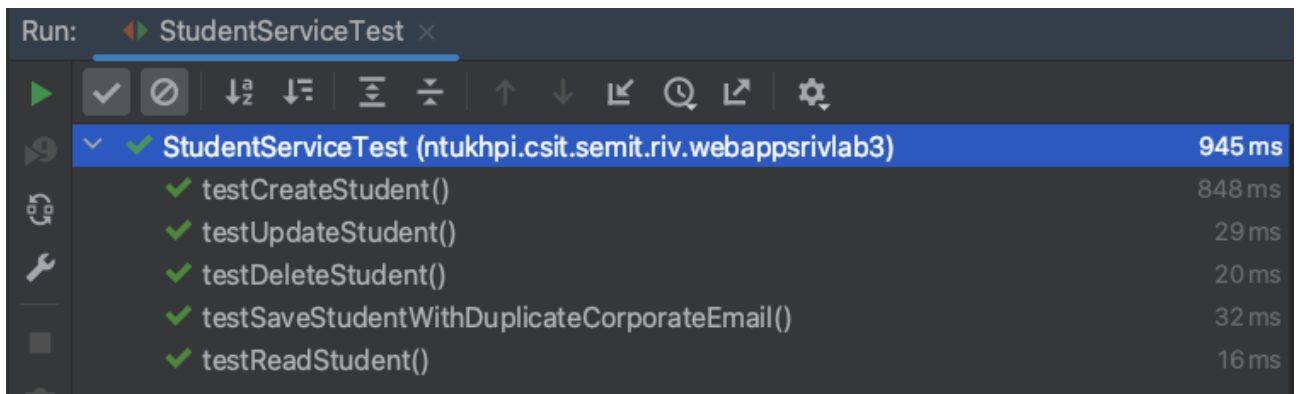
Клас `StudentServiceImpl.java`. Клас `StudentServiceImpl` є реалізацією інтерфейсу `StudentService` і забезпечує бізнес-логіку для роботи з об'єктами `Student` у Spring-застосунку. Він використовує `StudentRepository` для доступу до бази даних і методи з `EntrantService` для пошуку зв'язаних записів вступників. Клас підтримує операції отримання всіх студентів, фільтрацію та сортування, а також CRUD-операції – створення, оновлення і видалення студентів. Для обробки помилок використовуються специфічні виключення, включаючи порушення цілісності даних, обмеження валідності, конфлікти версій та відсутність записів. Методи `saveStudent`, `updateStudent`, і `deleteStudent` мають анотацію `@Transactional`, що забезпечує відкат змін у разі виникнення винятку. Код реалізованого класу `StudentServiceImpl.java` представлений у додатку В.5.

Тест `StudentServiceTest.java`. Клас `StudentServiceTest` є тестовим класом для перевірки функціональності сервісу `StudentService`, який керує CRUD-операціями для студентів. За допомогою JUnit і Spring Boot, цей клас тестує методи створення, читання, оновлення та видалення студентів у базі даних. Тестові методи включають:

- перевірку створення запису студента (`testCreateStudent`);
- пошук за ідентифікатором (`testReadStudent`);
- оновлення даних (`testUpdateStudent`);
- видалення студента (`testDeleteStudent`);
- обробку дублювання корпоративної пошти (`testSaveStudentWithDuplicateCorporateEmail`).

Тестовий клас використовує залежність від `EntrantService` для збереження об'єкта `Entrant`, необхідного для тестування `Student`. Метод `setUp` ініціалізує тестові дані, а `reset` видаляє їх після виконання кожного тесту, щоб забезпечити ізольованість тестових випадків. Код реалізованого тесту `StudentServiceTest.java` представлений у додатку В.6.

Результати виконання тестових випадків набору тестів StudentServiceTest представлені на рис. 12.



Test Method	Execution Time
StudentServiceTest (ntukhpi.csit.semit.riv.webappsrivlab3)	945 ms
testCreateStudent()	848 ms
testUpdateStudent()	29 ms
testDeleteStudent()	20 ms
testSaveStudentWithDuplicateCorporateEmail()	32 ms
testReadStudent()	16 ms

Рисунок 12 – Результати виконання тестового набору StudentServiceTest.java

4 Реалізація шару вебінтерфейсу для другої сутності

Результатом виконання даного завдання є реалізовані сторінки таблиці з даними другої сутності та шаблону з формою додавання, редагування та видалення запису з таблиці, а також контролери до них.

Сторінка StudentTable.htm. Сторінка StudentTable.html відображає список студентів з можливістю пошуку, фільтрації та сортування, що реалізовано за допомогою Thymeleaf та HTML. Користувачі можуть шукати студентів за повним ім'ям, фільтрувати за типом фінансування (бюджет або контракт) та статусом стипендії (відсутня, звичайна, підвищена). Таблиця містить інформацію про повне ім'я пов'язаного абітурієнта, тип фінансування, статус стипендії та корпоративний email кожного студента, з інтерактивними кнопками для перегляду пов'язаного запису абітурієнта, редагування та видалення записів студента. Компоненти ErrorToast і SuccessToast відображають повідомлення про статус операцій. Додатково, JavaScript забезпечує валідацію полів вводу та обробку параметрів URL для реалізації пошуку, фільтрації та сортування на серверній стороні застосунку. Код даної сторінки представлений у додатку Г.1.

Шаблон StudentForm.html. Шаблон StudentForm реалізований з використанням Thymeleaf та HTML для створення форми введення даних студента. Заголовок сторінки, вміст полів та доступ до полів форми змінюються залежно від режиму: створення, перегляд, оновлення чи видалення запису. Форма містить поля для вибору вступника, типу фінансування (бюджет/контракт), статусу стипендії (відсутня, звичайна, підвищена) та корпоративної електронної пошти. У шаблоні також реалізована валідація форми за допомогою JavaScript, яка перевіряє відповідність електронної пошти заданому шаблону та контролює доступність полів стипендії залежно від типу фінансування. Інтерфейс містить кнопки для збереження, скидання та видалення запису з підтвердженням, а також кнопки для повернення на попередню сторінку та перехід до таблиці студентів. Повідомлення про успіх або помилку відображаються через компоненти ErrorToast і SuccessToast. Код даної шаблону представлений у додатку Г.2.

Клас StudentController.java. Даний клас відповідає за обробку HTTP-запитів для керування об'єктами Student у вебзастосунку на основі Spring. Він забезпечує маршрутизацію для створення, оновлення, видалення та відображення таблиці студентів. Методи включають:

- showStudentTable для відображення таблиці з можливістю фільтрації та сортування;
- showCreateStudentForm, showUpdateStudentForm, і showDeleteStudentForm для відображення відповідних форм на основі шаблону StudentForm.html;
- метод saveStudent обробляє POST-запити для створення запису, updateStudent – для оновлення, а deleteStudent — для видалення.

Контролер використовує валідацію введених даних та обробку можливих помилок, що відображаються через сторінку помилок або компоненти повідомлень. Вся інформація динамічно додається до моделі, що забезпечує інтерактивність і зручність роботи користувача з інтерфейсом. Код даного класу представлений у додатку Г.3.

Результати роботи функціоналу вебзастосунку реалізованого до завдання 3 та 4 даної лабораторної роботи наведені на рис. Г.4.1-Г.4.11 у додатку Г.4.

Висновки

Отримані результати під час виконання даної лабораторної роботи відображають переваги застосування фреймворку Spring для Java-розробників, які працюють з реляційними базами даних. Фреймворк Spring надає потужні інструменти для створення CRUD-застосунків, що значно спрощує реалізацію бізнес-логіки та вебінтерфейсу. Завдяки архітектурі Spring, було побудовано багатoshарову структуру програми з чітким розділенням обов'язків між контролерами, сервісами та репозиторіями, що сприяє кращій підтримці коду та його розширюваності.

Spring забезпечує наступні переваги при роботі з базами даних.

1 Автоматизація взаємодії з базою даних. Spring Data JPA надає інтерфейси для автоматичного створення репозиторіїв та полегшує реалізацію CRUD-операцій без написання SQL-запитів. Це значно скорочує обсяг коду та зменшує ймовірність помилок, пов'язаних з написанням запитів.

2 Забезпечення цілісності даних. За рахунок використання анотацій та валідацій у Spring легко налаштовувати перевірки коректності та повноти даних на рівні сутностей. Це дозволяє контролювати специфіку предметної області та забезпечити відповідність типів даних, що зберігаються.

3 Обробка зв'язків між таблицями. Spring підтримує управління зв'язками між сутностями, що полегшує реалізацію операцій з пов'язаними таблицями. Це дозволяє ефективно враховувати обмеження зовнішнього ключа та відображати взаємозв'язки між даними.

3 Просте налаштування вебінтерфейсу. Spring MVC надає гнучкі можливості для створення контролерів та вебсторінок, що підтримують функції сортування, фільтрації та пошуку. Це забезпечує інтуїтивний інтерфейс для користувача, де він може легко керувати вмістом таблиць.

4 Модульність та тестування: Архітектура Spring дозволяє модульно тестувати кожен рівень програми (сервіси, репозиторії) окремо, що підвищує надійність застосунку та спрощує процес його підтримки.

Загалом, фреймворк Spring дозволив ефективно побудувати вебзастосунок, який інтегрується з реляційною базою даних та забезпечує повноцінний функціонал для керування даними в кількох пов'язаних таблицях. Це досягається завдяки високому рівню абстракції, вбудованій підтримці стандартів, таких як JPA, та розвинутій інфраструктурі для роботи з вебзастосунками, що робить Spring зручним і потужним інструментом для Java-розробників.

ДОДАТОК А

Створення Spring-проєкту, реалізація шару бізнес-логіки та тестування методів CRUD для першої сутності

А.1 Код файлу pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>
  <groupId>ntukhpi.csit.semit.riv</groupId>
  <artifactId>WebAppsRIVLab3</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <name>WebAppsRIVLab3</name>
  <description>WebAppsRIVLab3</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
```

```

        <developerConnection/>
    </tag>
    <url/>
</scm>
<properties>
    <java.version>17</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

```

```

    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.apache.commons</groupId>
      <artifactId>commons-lang3</artifactId>
      <version>3.15.0</version>
    </dependency>

    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.4.12</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>

```

```

        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
            <excludes>
                <exclude>
                    <groupId>org.projectlombok</groupId>
                    <artifactId>lombok</artifactId>
                </exclude>
            </excludes>
        </configuration>
    </plugin>
</plugins>
</build>

</project>

```

A.2 Код файла application.properties

```

spring.application.name=WebAppsRIVLab3
spring.datasource.url=jdbc:mysql://localhost:3307/webappsrivlab3?s
erverTimezone=Europe/Kiev&useSSL=false
spring.datasource.username=root
spring.datasource.password=rootpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=create
logging.level.org.hibernate.SQL=info
spring.jpa.open-in-view=false

```

A.3 Код файла logback.xml

```

<configuration>
    <appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS}    [%-5level]
%-55logger{36} - %msg%n</pattern>

```

```

        </encoder>

    </appender>

    <logger name="ntukhpi.csit.semit.riv.webappsrivlab3"
level="TRACE" />

    <root level="INFO">
        <appender-ref ref="CONSOLE" />
    </root>
</configuration>

```

A.4 Код класу WebAppsRivLab3Application.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3;

import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.EntrantList;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.StudentList;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.StudentService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

```

```

import java.util.List;

@SpringBootApplication
public class WebAppsRivLab3Application implements
CommandLineRunner {
    private static final Logger logger =
LoggerFactory.getLogger(WebAppsRivLab3Application.class);
    private final EntrantService entrantService;
    private final StudentService studentService;

    @Autowired
    public WebAppsRivLab3Application(EntrantService entrantService,
StudentService studentService) {
        this.entrantService = entrantService;
        this.studentService = studentService;
    }

    @Override
    public void run(String... args) {
        List<Entrant> entrantList =
EntrantList.INSTANCE.getEntrants();
        entrantService.saveAllEntrants(entrantList);

        List<Student> studentList =
StudentList.INSTANCE.getStudents();
        studentService.saveAllStudents(studentList);

        entrantList.forEach(entrant -> {
            entrant.setStudentId(
entrantService.findStudentIdForEntrant(entrant.getId()));
            entrantService.updateEntrant(entrant);
        });
    }

    public static void main(String[] args) {

```



```

        SpringApplication.run(WebAppsRivLab3Application.class,
args);
    }
}

```

A.5 Код класы ServletInitializer.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3;

import org.springframework.boot.builder.SpringApplicationBuilder;
import
org.springframework.boot.web.servlet.support.SpringBootServletInit
ializer;

public class ServletInitializer extends
SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder
configure(SpringApplicationBuilder application) {
        return
application.sources(WebAppsRivLab3Application.class);
    }
}

```

A.6 Код класы Entrant.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant;

import jakarta.persistence.Table;
import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.*;

```

```

import org.hibernate.validator.constraints.Length;
import org.springframework.format.annotation.*;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.*;

/**
 * Entrant is an entity class representing a record in the
 * "entrants" table.
 * It stores detailed information about an entrant, including
 * personal data and academic rating.
 * The class also includes validation constraints and custom
 * methods for data formatting.
 * <p>
 * Key features:
 * - Validation for fields such as case number, name, surname,
 * patronymic, birthday, gender, and rating score.
 * <p>
 * The class includes constructors for creating Entrant objects and
 * methods to manipulate or validate data fields.
 * It also contains utility methods for converting dates and
 * generating formatted output.
 * <p>
 * Validation rules:
 * - The case number must follow a specific format.
 * - The name, surname, and patronymic must only contain letters
 * from the Ukrainian alphabet.
 * - The birthday must be in the past and the entrant must be
 * between 16 and 110 years old.
 * - The rating score must be between 120.01 and 200.00.
 * <p>
 * This class uses Hibernate annotations for ORM mapping and
 * Jakarta validation for input validation.
 *

```

```
* @author Inessa Repeshko CS-222a
```

```
* @see Table
```

```
* @see Filter
```

```
* @see NotBlank
```

```
* @see Size
```

```
* @see Pattern
```

```
* @see Past
```

```
* @see DecimalMin
```

```
* @see DecimalMax
```

```
* @see FilterDef
```

```
* @see Data
```

```
* @see AllArgsConstructor
```

```
* @see NoArgsConstructor
```

```
*/
```

```
@Entity
```

```
@Table(name = "entrants")
```

```
@Data
```

```
@AllArgsConstructor
```

```
@NoArgsConstructor
```

```
public class Entrant {
```

```
    private static final String REGEX_CASE_NUMBER =
    "^(?=.{5,20}$) [A-ЩЮЯҐЄІЇ]{1,10} \\d{2} [\\-] \\d{1,4}$";
```

```
    private static final String REGEX_NAMES =
    "^(?=.{1,50}$) [A-ЩЮЯҐЄІЇ] (?:[A-ЯҐЄІїа-яґєіі']*[A-ЯҐЄІїа-яґєіі])?(?:[
    \\-] [A-ЩЮЯҐЄІЇ] (?:[A-ЯҐЄІїа-яґєіі']*[A-ЯҐЄІїа-яґєіі])?)*$";
```

```
    private static final DateTimeFormatter dateFormatterUA =
    DateTimeFormatter.ofPattern("dd.MM.yyyy");
```

```
    private static final DateTimeFormatter dateFormatterEN =
    DateTimeFormatter.ofPattern("yyyy-MM-dd");
```

```
    @Column(name = "id", insertable = false, updatable = false)
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

    @Positive(message = "The id must be a positive number.")

    private Long id;

    @Column(name = "case_number", unique = true, nullable = false,
length = 20)
    @Check(constraints = "REGEXP_LIKE(case_number,
'^[A-ЩЮЯҐЄІї]{1,10}[0-9]{2}-[0-9]{1,4}$', 'c') = 1")
    @Length(min = 5, max = 20, message = "The case number length
should be between 5 to 20 characters.")

    @NotBlank(message = "The case number must not be blank.")
    @Pattern(regexp = REGEX_CASE_NUMBER, message = "The case number
should be in the format 'LLLLLLLLLLNN-NNNN'.")

    private String caseNumber;

    @Column(name = "surname", nullable = false, length = 50)
    @Check(constraints = "REGEXP_LIKE(surname, '" + REGEX_NAMES +
"', 'c') = 1")
    @Length(min = 1, max = 50, message = "The surname length should
be between 1 to 50 characters.")

    @NotBlank(message = "The surname must not be blank.")
    @Size(min = 1, max = 50, message = "The surname length should
be between 1 to 50 characters.")
    @Pattern(regexp = REGEX_NAMES, message = "The surname can
contain only letters of the Ukrainian alphabet and can be complex
using a space or dash.")

    private String surname;

```

```

    @Column(name = "name", nullable = false, length = 50)
    @Check(constraints = "REGEXP_LIKE(name, '" + REGEX_NAMES + "',
'c') = 1")
    @Length(min = 1, max = 50, message = "The name length should be
between 1 to 50 characters.")

    @NotBlank(message = "The name must not be blank.")
    @Size(min = 1, max = 50, message = "The name length should be
between 1 to 50 characters.")
    @Pattern(regexp = REGEX_NAMES, message = "The name can contain
only letters of the Ukrainian alphabet and can be complex using a
space or dash.")

    private String name;

    @Column(name = "patronymic", nullable = false, length = 50)
    @Check(constraints = "REGEXP_LIKE(patronymic, '" + REGEX_NAMES
+ "', 'c') = 1")
    @Length(min = 1, max = 50, message = "The patronymic length
should be between 1 to 50 characters.")

    @NotBlank(message = "The patronymic must not be blank.")
    @Size(min = 1, max = 50, message = "The patronymic length
should be between 1 to 50 characters.")
    @Pattern(regexp = REGEX_NAMES, message = "The patronymic can
contain only letters of the Ukrainian alphabet and can be complex
using a space or dash.")

    private String patronymic;

    @Column(name = "birthday", nullable = false)
    @ColumnDefault(value = "'2000-01-01'")
    @Check(constraints = "birthday BETWEEN '1914-01-01' AND
'2008-01-01'")

```

```

@NotNull(message = "The birthday must not be null.")
@Past(message = "The birthday must be in the past.")
@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)

private LocalDate birthday;

@Column(name = "gender", nullable = false)
@ColumnDefault(value = "1")

@NotNull(message = "The gender must not be null.")

private Boolean gender;

@Column(name = "rating_score", nullable = false)
@ColumnDefault(value = "120.001")
@Check(constraints = "rating_score > 120.000 AND rating_score
<= 200.000")

@NotNull(message = "The rating score must not be null.")
@DecimalMin(value = "120.000", inclusive = false, message =
"The rating score must be greater than 120,000.")
@DecimalMax(value = "200.000", inclusive = true, message = "The
rating score must be less than or equal to 200,000.")
@Positive(message = "The rating score must be a positive
number.")

@NumberFormat(pattern = "###.###")

private Double ratingScore;

@Column(name = "student_id", nullable = true)

```

```

    @Positive(message = "The student id must be a positive
number.")

    private Long studentId;

    public Entrant(String caseNumber, String surname, String name,
String patronymic, String birthday, String gender, String
ratingScore) {
        setStringFields(caseNumber, surname, name, patronymic,
birthday, gender, ratingScore);
    }

    public Entrant(String id, String caseNumber, String surname,
String name, String patronymic, String birthday, String gender,
String ratingScore) {
        try {
            setId(Long.parseLong(id));
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("The id should not
be empty and should be long.");
        }

        setStringFields(caseNumber, surname, name, patronymic,
birthday, gender, ratingScore);
    }

    private void setStringFields(String caseNumber, String surname,
String name, String patronymic, String birthday, String gender,
String ratingScore) {
        setCaseNumber(caseNumber);
        setSurname(surname);
        setName(name);
        setPatronymic(patronymic);
        setBirthday(birthday);
    }

```

```

        if (gender != null && !gender.isBlank()) {
            setGender(Boolean.parseBoolean(gender));
        } else {
            throw new IllegalArgumentException("The gender should
not be empty and should be boolean.");
        }

        try {
            setRatingScore(Double.parseDouble(ratingScore));
        } catch (NumberFormatException | NullPointerException e) {
            throw new IllegalArgumentException("The rating score
should not be empty and should be double.");
        }
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setCaseNumber(String caseNumber) {
        this.caseNumber = validateCaseNumber(caseNumber);
    }

    public void setSurname(String surname) {
        this.surname = validateName(surname);
    }

    public void setName(String name) {
        this.name = validateName(name);
    }

    public void setPatronymic(String patronymic) {
        this.patronymic = patronymic.trim().equals("-") ?
patronymic.trim() : validateName(patronymic);
    }

```



```

public void setBirthday(String birthday) {
    this.birthday = validateBirthday(birthday);
}

public void setGender(boolean gender) {
    this.gender = Boolean.valueOf(gender);
}

public void setRatingScore(double ratingScore) {
    this.ratingScore = validateRatingScore(ratingScore);
}

private String validateCaseNumber(String caseNumber) {
    if (caseNumber == null) {
        throw new IllegalArgumentException("The case number
cannot be empty.");
    }

    caseNumber = caseNumber.trim();

    if (caseNumber.isEmpty() ||
!caseNumber.matches(Regex.CASE_NUMBER)) {
        throw new IllegalArgumentException("The case number
should be in the format 'LLLLLLLLLLNN-NNNN'.");
    }

    return caseNumber;
}

private String validateName(String name) {
    if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException("Parts of the name
cannot be empty.");
    }

    name = name.trim();

```

```

String[] words = name.split("\\s+");
StringBuilder formattedName = new StringBuilder();

for (String word : words) {
    String[] subWords = word.split("-");

    for (String subWord : subWords) {
        if
(!subWord.matches("^[A-Щa-щЮюЯяҐґЄєІіЇї](?:[A-Яa-яҐґЄєІіЇї']+)?$"))
{
            throw new IllegalArgumentException("Parts of
the name can contain only letters of the Ukrainian alphabet and
can be complex using a space or dash.");
        }

        if (!subWord.isEmpty()) {
formattedName.append(Character.toUpperCase(subWord.charAt(0)))
.append(subWord.substring(1).toLowerCase())
.append("-");
        }
    }

    formattedName.setLength(formattedName.length() - 1);
    formattedName.append(" ");
}

return formattedName.toString().trim();
}

private LocalDate validateBirthday(String birthday) {
    if (birthday == null) {
        throw new IllegalArgumentException("The birthday cannot
be empty.");
    }
}

```

```

    }

    LocalDate birthDate;

    try {
        birthDate = LocalDate.parse(birthday, dateFormatterUA);
    } catch (DateTimeParseException e) {
        try {
            birthDate = LocalDate.parse(birthday,
dateFormatterEN);
        } catch (DateTimeParseException e1) {
            throw new IllegalArgumentException("The birthday
should be in the format 'dd.MM.yyyy' or 'yyyy-MM-dd'.");
        }
    }

    LocalDate now = LocalDate.now();

    if (birthDate.isBefore(now.minusYears(110))
        || birthDate.isAfter(now.minusYears(16))) {
        throw new IllegalArgumentException("Date of birth must
be between 16 and 110 years ago.");
    }

    return birthDate;
}

private Double validateRatingScore(double ratingScore) {
    if (ratingScore < 120.001 || ratingScore > 200.000) {
        throw new IllegalArgumentException("The rating score
should be between 120.01 and 200.00.");
    }

    return ratingScore;
}

```

```

    public static String[][] getFieldNamesAsFormattedStrings() {
        List<String[]> fieldNames = Arrays.asList(
            new String[]{"caseNumber", "Case Number"},
            new String[]{"fullName", "Full Name"},
            new String[]{"birthday", "Birthday"},
            new String[]{"gender", "Gender"},
            new String[]{"ratingScore", "Rating Score"}
        );

        return fieldNames.toArray(String[][]::new);
    }
}

```

A.7 Код класу EntrantList.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * EntrantList is a singleton class implemented using an Enum to
 * store a predefined list of {@link Entrant} objects.
 * It provides a method for initializing the list with sample data
 * and retrieving the list of entrants.
 * <p>
 * Key features:
 * - Enum-based singleton to ensure only one instance of
 * EntrantList exists.
 * - Predefined list of entrants that can be used for testing or
 * demo purposes.
 * - The list includes both valid and logically deleted entrants,
 * as well as examples with complex name formats.
 * <p>

```

```

* This class is primarily used to simulate a database or
repository of entrants.
*
* @author Inessa Repeshko CS-222a
* @see Entrant
* @see ArrayList
*/

public enum EntrantList {
    INSTANCE;

    private final List<Entrant> entrants;

    EntrantList() {
        this.entrants = new ArrayList<>();

        List<String[]> entrantData = Arrays.asList(
            new String[]{"KH22-4814", "Бабічев", "Олексій",
"Вікторович", "2005-05-22", "TRUE", "168.111"},
            new String[]{"KH22-5835", "Волков", "Дмитро",
"Олександрович", "2005-07-16", "TRUE", "151.202"},
            new String[]{"KH22-5511", "Геєнко", "Анна",
"Олексіївна", "2005-12-03", "FALSE", "150.013"},
            new String[]{"KH22-9299", "Голоп'яров", "Микита",
"Сергійович", "2005-11-07", "TRUE", "161.484"},
            new String[]{"KH22-9012", "Гриценко", "Максим",
"Георгійович", "2005-12-04", "TRUE", "191.945"},
            new String[]{"KH22-6636", "Жарий", "Вадим",
"Віталійович", "2005-10-25", "TRUE", "128.346"},
            new String[]{"KH22-5140", "Жупанов", "Андрій",
"Володимирович", "2005-06-22", "TRUE", "183.627"},
            new String[]{"KH22-4896", "Кльоз", "Михайло",
"Романович", "2005-01-15", "TRUE", "127.288"},
            new String[]{"KH22-4694", "Коваль", "Микита",
"Артемович", "2005-10-11", "TRUE", "139.739"},

```

```

        new String[]{"KH22-5529", "Ковтун", "Єгор",
"Вячеславович", "2005-04-10", "TRUE", "187.790"},
        new String[]{"KH22-6874", "Колесниченко", "Денис",
"Євгенович", "2005-03-26", "TRUE", "165.641"},
        new String[]{"KH22-2947", "Ласкевич", "Андрій",
"Сергійович", "2005-09-05", "TRUE", "175.392"},
        new String[]{"KH22-3856", "Лукаш", "Антон",
"Олександрович", "2005-11-24", "TRUE", "184.473"},
        new String[]{"KH22-7534", "Палій", "Нікіта",
"Андрійович", "2005-05-05", "TRUE", "168.004"},
        new String[]{"KH22-3843", "Прокопов", "Владислав",
"Сергійович", "2004-12-12", "TRUE", "164.665"},
        new String[]{"KH22-7819", "Репешко", "Інесса",
"Віталіївна", "2005-05-06", "FALSE", "185.306"},
        new String[]{"KH22-9115", "Скиба", "Андрій",
"Григорович", "2004-02-26", "TRUE", "125.907"},
        new String[]{"KH22-3812", "Цибань", "Ольга",
"Юріївна", "2004-05-19", "FALSE", "190.298"},
        new String[]{"KH22-1828", "Шевченко", "Данило",
"Валерійович", "2004-10-25", "TRUE", "166.849"},
        new String[]{"KH22-9514", "Антипенко", "Катерина",
"Віталіївна", "2005-01-06", "FALSE", "168.460"},
        new String[]{"KH22-2929", "Багрянцева", "Марина",
"Русланівна", "2005-10-01", "FALSE", "157.631"},
        new String[]{"KH22-1637", "Бондаренко", "Ярослав",
"Юрійович", "2005-04-24", "TRUE", "199.532"},
        new String[]{"KH22-6681", "Булгаков", "Віктор",
"Олександрович", "2005-05-17", "TRUE", "153.683"},
        new String[]{"KH22-7557", "Васильєв", "Михайло",
"Олександрович", "2005-01-25", "TRUE", "170.864"},
        new String[]{"KH22-8759", "Деркач", "Дарина",
"Євгенівна", "2005-03-03", "FALSE", "171.715"},
        new String[]{"KH22-7309", "Ілюхін", "Тимур",
"Ігорович", "2005-01-24", "TRUE", "125.776"},
        new String[]{"KH22-8254", "Криженко", "Аліна",
"Степанівна", "2005-08-29", "FALSE", "138.747"},

```

```

        new String[]{"KH22-3825", "Посмашний", "Ярослав",
"Дмитрович", "2000-02-25", "TRUE", "160.718"},
        new String[]{"KH22-8153", "Рожкова", "Вікторія",
"Андріївна", "1993-10-04", "FALSE", "188.309"},
        new String[]{"KH22-4788", "Сизоненко", "Єлизавета",
"Олександрівна", "1997-02-17", "FALSE", "124.240"},
        new String[]{"KH22-8255", "Криженко", "Аліна",
"Степанівна", "2005-08-29", "FALSE", "138.747"},
        new String[]{"KH22-3826", "Посмашний", "Ярослав",
"Дмитрович", "2000-02-25", "TRUE", "160.718"},
        new String[]{"KH22-8154", "Рожкова", "Вікторія",
"Андріївна", "1993-10-04", "FALSE", "188.309"},
        new String[]{"KH22-6682", "Булгаков", "Віктор",
"Олександрович", "2005-05-17", "TRUE", "153.683"}
    );

    entrantData.forEach(
        data -> addEntrant(new Entrant(
            data[0],
            data[1],
            data[2],
            data[3],
            data[4],
            data[5],
            data[6])));

    addEntrant(new Entrant(
        "KH22-9998",
        "Марчук",
        "Регіна",
        "Віталіївна",
        "06.06.1996",
        "FALSE",
        "200"));
}

```

```

    public List<Entrant> getEntrants() {
        return new ArrayList<>(entrants);
    }

    public void addEntrant(Entrant entrant) {
        entrants.add(entrant);
    }

    public void removeEntrant(Entrant entrant) {
        entrants.remove(entrant);
    }

    public void clearEntrants() {
        entrants.clear();
    }
}

```

A.8 Код інтерфейсу EntrantRepository.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.repository;

import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

@Repository

```



```

public interface EntrantRepository extends JpaRepository<Entrant,
Long> {
    List<Entrant> getEntrantsBySurnameAndNameAndPatronymic(String
surname, String name, String patronymic);

    @Query("SELECT e FROM Entrant e "
        + "WHERE (:search IS NULL OR UPPER(e.caseNumber) LIKE
UPPER(CONCAT(:search, '%'))) "
        + "OR UPPER(CONCAT(e.surname, ' ', e.name, ' ',
e.patronymic)) LIKE UPPER(CONCAT(:search, '%')) "
        + "AND (:birthdayStart IS NULL OR e.birthday >=
:birthdayStart) "
        + "AND (:birthdayEnd IS NULL OR e.birthday <=
:birthdayEnd) "
        + "AND (:ratingScoreMin IS NULL OR e.ratingScore >=
:ratingScoreMin) "
        + "AND (:ratingScoreMax IS NULL OR e.ratingScore <=
:ratingScoreMax) ")
    List<Entrant> findFilteredAndSortedEntrants(@Param("search")
String search,

@Param("birthdayStart") LocalDate birthdayStart,

@Param("birthdayEnd") LocalDate birthdayEnd,

@Param("ratingScoreMin") Double ratingScoreMin,

@Param("ratingScoreMax") Double ratingScoreMax,

Sort sortBy);

    @Query("SELECT e FROM Entrant e WHERE e.id NOT IN (SELECT
s.entrant.id FROM Student s WHERE s.entrant IS NOT NULL)")
    List<Entrant> findAllWithoutStudent(Sort sort);

    @Query("SELECT s.id FROM Student s WHERE s.entrant.id =
:entrantId")

```

```
Optional<Long> findStudentIdByEntrantId(@Param("entrantId")
Long entrantId);
}
```

A.9 Код інтерфейсу EntrantService.java

```
package ntukhpi.csit.semit.riv.webappsrivlab3.service;

import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;

import java.time.LocalDate;
import java.util.List;

public interface EntrantService {
    List<Entrant> getAllEntrants();

    List<Entrant> getFilteredAndSortedEntrants(String search,
                                                LocalDate
birthdayStart,
                                                LocalDate
birthdayEnd,
                                                Double
ratingScoreMin,
                                                Double
ratingScoreMax,
                                                String sort);

    List<Entrant> findEntrantsWithoutStudents();

    void saveAllEntrants(List<Entrant> entrants);

    Entrant findEntrantById(Long id);

    Entrant findEntrantByExample(Entrant entrant);
}
```

```

    Long findStudentIdForEntrant(Long entrantId);

    Entrant saveEntrant(Entrant entrant);

    Entrant updateEntrant(Entrant entrant);

    void deleteEntrant(Entrant entrant);
}

```

A.10 Код класы EntrantServiceImpl.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab3.service.implementation;

import jakarta.persistence.EntityNotFoundException;
import jakarta.validation.ConstraintViolationException;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab3.repository.EntrantRepository
;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.data.domain.Example;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.transaction.annotation.Transactional;

```



```

        String sortPattern =
            "(id|caseNumber|fullName|surname|name|patronymic|birthday|gender|ratingScore)-(asc|desc)$";

        if (!sort.trim().matches(sortPattern)) {
            logger.warn("Invalid sort field provided: " + sort + ". It should be in a format: 'fieldName-direction'.");
            throw new IllegalArgumentException("Invalid sort field provided: " + sort + ". It should be in a format: 'fieldName-direction'.");
        }

        String[] sortParams = sort.split("-");
        String sortField = sortParams[0];
        Sort.Direction sortDirection;

        try {
            sortDirection = Sort.Direction.fromString(sortParams[1]);
        } catch (IllegalArgumentException e) {
            logger.error(e.getMessage());
            throw new IllegalArgumentException(e.getMessage());
        }

        List<Sort.Order> sortOrders = new ArrayList<>();

        switch (sortField) {
            case "caseNumber":
                sortOrders.add(new Sort.Order(sortDirection, "caseNumber"));
                sortOrders.add(new Sort.Order(sortDirection, "surname"));
                sortOrders.add(new Sort.Order(sortDirection, "name"));
                sortOrders.add(new Sort.Order(sortDirection, "patronymic"));

```

```

        break;

        case "surname":
        case "fullName":
            sortOrders.add(new Sort.Order(sortDirection,
"surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"name"));
            sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));
            sortOrders.add(new Sort.Order(sortDirection,
"caseNumber"));
            break;

        default:
            sortOrders.add(new Sort.Order(sortDirection,
sortField));
            sortOrders.add(new Sort.Order(sortDirection,
"surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"name"));
            sortOrders.add(new Sort.Order(sortDirection,
"patronymic"));
            sortOrders.add(new Sort.Order(sortDirection,
"caseNumber"));
            break;
    }

    Sort sortBy = Sort.by(sortOrders);

    try {
        return entrantRepository.findFilteredAndSortedEntrants(
            search,
            birthdayStart, birthdayEnd,
            ratingScoreMin, ratingScoreMax,
            sortBy);
    }

```

```

        } catch (Exception e) {
            logger.error("An error occurred while getting the
filtered and sorted entrants: " + e.getMessage());
            throw new RuntimeException("An error occurred while
getting the filtered and sorted entrants: " + e.getMessage());
        }
    }

    @Override
    public List<Entrant> findEntrantsWithoutStudents() {
        List<Sort.Order> sortOrders = new ArrayList<>();

        sortOrders.add(new Sort.Order(Sort.Direction.ASC,
"surname"));
        sortOrders.add(new Sort.Order(Sort.Direction.ASC, "name"));
        sortOrders.add(new Sort.Order(Sort.Direction.ASC,
"patronymic"));
        sortOrders.add(new Sort.Order(Sort.Direction.ASC,
"caseNumber"));

        Sort sort = Sort.by(sortOrders);

        return entrantRepository.findAllWithoutStudent(sort);
    }

    @Override
    @Transactional(rollbackFor =
{DataIntegrityViolationException.class,
ConstraintViolationException.class,
OptimisticLockingFailureException.class})
    public void saveAllEntrants(List<Entrant> entrants) {
        try {
            entrantRepository.saveAll(entrants);
        } catch (DataIntegrityViolationException e) {
            /* Handling a duplicate or uniqueness exception */

```

```

        logger.error("Duplicate or invalid data detected.\nData
integrity violation: " + e.getMessage());

        throw new IllegalArgumentException("Duplicate or
invalid data detected.");

    } catch (ConstraintViolationException e) {
        /* Handling violations of validation constraints */
        logger.error("Validation failed for the
entity.\nConstraint violation: " + e.getMessage());
        throw new IllegalArgumentException("Validation failed
for the entity. Check the input data.");
    } catch (OptimisticLockingFailureException e) {
        /* Handling version conflicts (if optimistic locking is
used) */
        logger.error("Optimistic locking failure: " +
e.getMessage());
        throw new RuntimeException("The entity could not be
saved due to a version conflict.");
    } catch (Exception e) {
        /* General handling of other exceptions */
        logger.error("An error occurred while saving the
entity.\n" + e.getMessage());
        throw new RuntimeException("An error occurred while
saving the entity.");
    }
}

@Override
public Entrant findEntrantById(Long id) {
    return entrantRepository.findById(id)
        .orElseThrow(() -> {
            String errorMessage = "No record exists with
id: " + id + ".";
            logger.error(errorMessage);

```



```

        return new
EntityNotFoundException(errorMessage);
    });
}

@Override
public Entrant findEntrantByExample(Entrant entrant) {
    Example<Entrant> example = Example.of(entrant);
    return entrantRepository.findBy(example, query ->
query.first().orElse(null));
}

@Override
public Long findStudentIdForEntrant(Long entrantId) {
    return
entrantRepository.findStudentIdByEntrantId(entrantId).orElse(null)
;
}

@Override
@Transactional(rollbackFor =
{DataIntegrityViolationException.class,
ConstraintViolationException.class,
OptimisticLockingFailureException.class})
public Entrant saveEntrant(Entrant entrant) {
    try {
        return entrantRepository.save(entrant);
    } catch (DataIntegrityViolationException e) {
        /* Handling a duplicate or uniqueness exception,
foreign key, NOT NULL, CHECK, or length violation */
        logger.error("Duplicate or invalid data detected.\nData
integrity violation: " + e.getMessage());
        throw new IllegalArgumentException(
            "The record with the case number '" +
entrant.getCaseNumber() + "' already exists.");
    }
}

```

```

        } catch (ConstraintViolationException e) {
            /* Handling violations of validation constraints */
            logger.error("Validation failed for the
entity.\nConstraint violation: " + e.getMessage());
            throw new IllegalArgumentException("Validation failed
for the entity. Check the input data.");

        } catch (OptimisticLockingFailureException e) {
            /* Handling version conflicts (if optimistic locking is
used) */
            logger.error("Optimistic locking failure: " +
e.getMessage());
            throw new RuntimeException("The entity could not be
saved due to a version conflict.");

        } catch (Exception e) {
            /* General handling of other exceptions */
            logger.error("An error occurred while saving the
entity.\n" + e.getMessage());
            throw new RuntimeException("An error occurred while
saving the entity.");
        }
    }

    @Override
    @Transactional(rollbackFor =
{DataIntegrityViolationException.class,
ConstraintViolationException.class,
OptimisticLockingFailureException.class})
    public Entrant updateEntrant(Entrant entrant) {
        if (entrant.getId() == null) {
            logger.error("Cannot update a record without an id.");
            throw new IllegalArgumentException("Cannot update a
record without an id.");
        }
    }

```

```

        if (!entrantRepository.existsById(entrant.getId())) {
            logger.error("No record exists to update.");
            throw new IllegalArgumentException("No record exists to
update.");
        }

        return saveEntrant(entrant);
    }

    @Override
    @Transactional(rollbackFor =
{DataIntegrityViolationException.class,
EmptyResultDataAccessException.class,
OptimisticLockingFailureException.class})
    public void deleteEntrant(Entrant entrant) {
        try {
            entrantRepository.delete(entrant);

        } catch (DataIntegrityViolationException e) {
            /* Handling data integrity violations, for example,
when foreign keys are violated */
            logger.error("Data integrity violation occurred: " +
e.getMessage());
            throw new IllegalArgumentException("Unable to delete
the entity due to data integrity constraints.");

        } catch (EmptyResultDataAccessException e) {
            /* Handling the case when an attempt to delete a
non-existent record is made */
            logger.error("No entity found for deletion: " +
e.getMessage());
            throw new IllegalArgumentException("The entity does not
exist and cannot be deleted.");

        } catch (OptimisticLockingFailureException e) {

```

```

        /* Handling an error when trying to delete a record
        with a version conflict */
        logger.error("Optimistic locking failure: " +
e.getMessage());
        throw new IllegalArgumentException("The entity could
not be deleted due to a version conflict.");

    } catch (Exception e) {
        /* General handling of other exceptions */
        logger.error("An unexpected error occurred while
deleting the entity: " + e.getMessage());
        throw new RuntimeException("An unexpected error
occurred while deleting the entity.");
    }
}
}
}

```

A.11 Код тесту EntrantServiceTest.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3;

import static org.junit.jupiter.api.Assertions.*;
import static org.junit.jupiter.api.Assertions.assertThrows;

import jakarta.persistence.EntityNotFoundException;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;

```

```

@SpringBootTest
class EntrantServiceTest {

    @Autowired
    private EntrantService entrantService;

    private Entrant testEntrant;

    @BeforeEach
    void setUp() {
        testEntrant = new Entrant("KH22-9876", "Франко", "Іван",
"Якович", "2002-01-15", "TRUE", "150.0");
    }

    @AfterEach
    void reset() {
        entrantService.deleteEntrant(testEntrant);
    }

    @Test
    void testCreateEntrant() {
        Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);
        assertNotNull(savedEntrant.getId(), "Entrant should have an
ID after saving.");
    }

    @Test
    void testReadEntrant() {
        Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);
        Entrant foundEntrant =
entrantService.findEntrantById(savedEntrant.getId());
        assertNotNull(foundEntrant, "Entrant should be found by
ID.");
    }
}

```

```

        assertEquals(savedEntrant.getCaseNumber(),
foundEntrant.getCaseNumber(), "Case number should match.");
    }

    @Test
    void testUpdateEntrant() {
        Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);
        savedEntrant.setName("Василь");
        Entrant updatedEntrant =
entrantService.saveEntrant(savedEntrant);

        Entrant foundEntrant =
entrantService.findEntrantById(savedEntrant.getId());
        assertNotNull(foundEntrant, "Updated Entrant should be
found by ID.");
        assertEquals("Василь", foundEntrant.getName(), "Name should
be updated.");
    }

    @Test
    void testDeleteEntrant() {
        Entrant savedEntrant =
entrantService.saveEntrant(testEntrant);
        Long id = savedEntrant.getId();
        entrantService.deleteEntrant(savedEntrant);

        assertThrows(EntityNotFoundException.class, () -> {
            entrantService.findEntrantById(id);
        }, "Entrant should not be found after deletion.");
    }

    @Test
    void testSaveEntrantWithDuplicateCaseNumber() {
        entrantService.saveEntrant(testEntrant);
    }

```

```
Entrant duplicateEntrant = new Entrant("KH22-9876",  
"Шевченко", "Тарас", "Григорович", "2002-01-16", "FALSE",  
"180.0");  
  
assertThrows(IllegalArgumentException.class, () -> {  
    entrantService.saveEntrant(duplicateEntrant);  
}, "Duplicate case number should throw  
IllegalArgumentException.");  
}  
}
```

ДОДАТОК Б

Реалізація шару вебінтерфейсу для першої сутності та основних сторінок користувачького інтерфейсу

Б.1 Код сторінки Login.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title>Login</title>
  <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
crossorigin="anonymous">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>
<div th:replace="~{component/ErrorToast1 :: errorToast}"></div>

<div class="container-fluid vh-100 p-0 m-0 flex-column d-flex
align-content-center justify-items-center">
```



```

<div class="row align-content-center justify-content-center
mt-2">

    <h1 class="w-50 text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-1 fw-semibold font-monospace text-center
        shadow-lg p-2 mb-5 mt-2 bg-body-tertiary rounded">

        <span class="p-0 m-0 ms-2">Student Database
Viewer</span>
    </h1>
</div>

<div class="row text-center justify-content-center
align-content-center">

    <h2 class="w-25 text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-3 fw-semibold font-monospace text-center
        shadow p-2 mb-3 bg-body-tertiary rounded">

        Login
    </h2>
</div>

<div class="row text-center justify-content-center
align-content-center">

    <div class="w-50 card border border-2 border-info-subtle
p-5
        shadow mb-5 bg-body-tertiary rounded">

        <div class="card-body bg-light font-monospace">

            <form id="login-form" th:action="@{/process-login}"
method="POST">

                <div class="input-group mb-5 border border-1
border-info-subtle rounded">

                    <span class="col-4 input-group-text
text-info">Login</span>

```

```

        <input type="text" class="col-auto
form-control text-dark" name="login" required
        th:value="{userLogin}">
    </div>

    <div class="input-group mb-5 border border-1
border-info-subtle rounded">
        <span class="col-4 input-group-text
text-info">Password</span>
        <input type="password" class="col-auto
form-control text-dark" name="password" required
        th:value="{userPassword}">
    </div>

    <div class="d-flex justify-content-center">
        <button type="submit" class="btn
btn-outline-info" name="submit">
            <i class="bi
bi-box-arrow-in-right"></i>
        </button>
    </div>
</form>
</div>
</div>
</div>

<div class="mt-auto p-0">
    <footer th:replace="{component/Footer ::
footer}"></footer>
</div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

```

```

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"
    crossorigin="anonymous"></script>
</body>
</html>

```

Б.2 Код сторінки Unauthorized.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Unauthorized Error</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>
<div class="container-fluid p-0 m-0 vh-100 d-flex
justify-items-center flex-column align-content-center">

```

```

<div class="row align-content-center justify-content-center
mt-5">
    <div class="card border border-2 border-danger-subtle
        shadow bg-danger-subtle rounded mt-5 w-50 p-0">
        <div class="card-header bg-danger-subtle border-bottom
border-2 border-danger-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder">
            <span class="badge text-bg-danger">401</span>
            Unauthorized Error
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light text-center">
                <p>Authorisation <code
class="text-danger">failed</code> due to entering invalid
credentials.</p>
                <p>Please, try again!</p>
            </div>
        </div>
    </div>

    <div class="d-flex justify-content-center mt-5">
        <button class="btn btn-outline-info" type="button"
onclick="window.history.back()">
            <i class="bi bi-box-arrow-in-left"></i>
        </button>
    </div>
</div>

<div class="mt-auto m-0 p-0">
    <footer th:replace="~{component/Footer ::
footer}"></footer>
</div>
</div>

```

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"

        crossorigin="anonymous"></script>
</body>
</html>

```

Б.3 Код сторінки Home.html

```

<!DOCTYPE html>

<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1">

    <title>Home</title>

    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"

        rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"

        crossorigin="anonymous">

```

```

<link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div class="container" style="flex: 90%; max-width: 90%">
    <div class="bg-info-subtle border border-3 rounded-pill
border-info-subtle
        fs-1 fw-semibold font-monospace text-info
text-center shadow-lg p-3 my-5 bg-body-tertiary rounded mb-5">
        <span th:text="{userLogin != null} ? ('Welcome, ' +
${userLogin} + '!') : _" >Welcome to Student Database
Viewer!</span>
    </div>

    <div class="col m-0 p-0 d-flex flex-wrap gap-4
justify-content-between">

        <div class="row gap-4 align-self-start" style="flex: 33%;
max-width: 33%">

            <div class=" card p-0 border border-2
border-info-subtle shadow bg-body-tertiary rounded">

                <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle

```

```

        font-monospace text-center
text-info-emphasis fw-bolder">
        Software developer info
    </div>
    <div class="card-body bg-light font-monospace">
        <p class="card-text text-secondary-emphasis
fw-light">
            Name:
            <span class="text-primary-emphasis
fw-medium fst-italic ms-3">Repeshko Inessa</span>
        </p>
        <p class="card-text text-secondary-emphasis
fw-light">
            Group:
            <span class="text-primary-emphasis
fw-medium fst-italic ms-3">CS-222a</span>
        </p>
        <p class="card-text text-secondary-emphasis
fw-light">
            Department:
            <span class="text-primary-emphasis
fw-medium fst-italic ms-3">SEMIT</span>
        </p>
        <p class="card-text text-secondary-emphasis
fw-light">
            Year:
            <span class="text-primary-emphasis
fw-medium fst-italic ms-3">2024</span>
        </p>

```

```

        <p class="card-text text-secondary-emphasis
fw-light">

            Page access time:

            <span class="text-primary-emphasis
fw-medium fst-italic ms-3"

th:text="${#dates.format(#dates.createNow(), 'dd.MM.yyyy
HH:mm:ss')}">

                </span>

            </p>

        </div>

    </div>

    <div class=" card p-0 border border-2
border-info-subtle shadow bg-body-tertiary rounded
placeholder-wave">

        <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle

            font-monospace text-center
text-info-emphasis fw-bolder">

            <span class="placeholder w-75 bg-info
placeholder-lg"></span>

        </div>

        <div class="card-body bg-light font-monospace">

            <div class="card-text text-secondary-emphasis
fw-light">

                <p class="placeholder w-100
bg-secondary"></p>

```



```

        <p class="placeholder w-100
bg-secondary"></p>

    </div>

    <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">

        <p class="placeholder w-100
bg-secondary"></p>

        <ul>

            <li>

                <span class="placeholder w-25
bg-info me-3"></span>

                <span class="placeholder w-50
bg-secondary "></span>

            </li>

            <li>

                <span class="placeholder w-25
bg-info me-3"></span>

                <span class="placeholder w-50
bg-secondary "></span>

            </li>

            <li>

                <span class="placeholder w-25
bg-info me-3"></span>

                <span class="placeholder w-50
bg-secondary "></span>

            </li>

            <li>

                <span class="placeholder w-25
bg-info me-3"></span>

```

```

                <span class="placeholder w-50
bg-secondary "></span>

                </li>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                </li>

            </ul>

        </div>

        <div class="d-flex justify-content-center">

            <button class="btn btn-info disabled
placeholder col-2" aria-disabled="true"></button>

        </div>

    </div>

</div>

<div class=" card p-0 border border-2
border-info-subtle shadow bg-body-tertiary rounded
placeholder-wave">

    <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle

        font-monospace text-center

text-info-emphasis fw-bolder">

        <span class="placeholder w-75 bg-info
placeholder-lg"></span>

    </div>

```

```

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p class="placeholder w-100
bg-secondary"></p>
                <p class="placeholder w-100
bg-secondary"></p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                <p class="placeholder w-100
bg-secondary"></p>
                <ul>
                    <li>
                        <span class="placeholder w-25
bg-info me-3"></span>
                        <span class="placeholder w-50
bg-secondary "></span>
                    </li>
                    <li>
                        <span class="placeholder w-25
bg-info me-3"></span>
                        <span class="placeholder w-50
bg-secondary "></span>
                    </li>
                    <li>
                        <span class="placeholder w-25
bg-info me-3"></span>

```

```

                <span class="placeholder w-50
bg-secondary "></span>

                </li>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                    </li>

                    <li>

                        <span class="placeholder w-25
bg-info me-3"></span>

                        <span class="placeholder w-50
bg-secondary "></span>

                        </li>
                    </ul>
                </div>

                <div class="d-flex justify-content-center">
                    <button class="btn btn-info disabled
placeholder col-2" aria-disabled="true"></button>
                </div>
            </div>
        </div>
    </div>

    <div class="row gap-4 align-self-start" style="flex: 33%;
max-width: 33%">

        <div class=" card p-0 border border-2
border-info-subtle shadow bg-body-tertiary rounded">

```

```

        <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
        font-monospace text-center
text-info-emphasis fw-bolder">
            Entrant Table
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p>Table with data on entrants of a certain
higher education institution in Ukraine.</p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                <p class="p-0 m-0">The table contains the
following information about entrants:</p>
                <ul>
                    <li><code class="text-info">Long</code>
unique identifier;</li>
                    <li><code
class="text-info">String</code> case number of the personal
file;</li>
                    <li><code
class="text-info">String</code> surname;</li>
                    <li><code
class="text-info">String</code> name;</li>
                    <li><code
class="text-info">String</code> patronymic;</li>

```

```

        <li><code
class="text-info">LocalDate</code> date of birth;</li>

        <li><code
class="text-info">Boolean</code> gender;</li>

        <li><code
class="text-info">Double</code> rating score.</li>

    </ul>

</div>

    <div class="d-flex justify-content-center">

        <a th:href="@{/entrants}" class="btn
btn-outline-info" role="button">

            <i class="bi bi-table"></i>

        </a>

    </div>

</div>

</div>

    <div class=" card p-0 border border-2
border-info-subtle shadow bg-body-tertiary rounded
placeholder-wave">

        <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
font-monospace text-center
text-info-emphasis fw-bolder">

            <span class="placeholder w-75 bg-info
placeholder-lg"></span>

        </div>

        <div class="card-body bg-light font-monospace">

```

```

        <div class="card-text text-secondary-emphasis
fw-light">

            <p class="placeholder w-100
bg-secondary"></p>

            <p class="placeholder w-100
bg-secondary"></p>

        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">

            <p class="placeholder w-100
bg-secondary"></p>

            <ul>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                </li>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                </li>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

```

```

        </li>

        <li>

            <span class="placeholder w-25
bg-info me-3"></span>

            <span class="placeholder w-50
bg-secondary "></span>

        </li>

        <li>

            <span class="placeholder w-25
bg-info me-3"></span>

            <span class="placeholder w-50
bg-secondary "></span>

        </li>
    </ul>
</div>

    <div class="d-flex justify-content-center">
        <button class="btn btn-info disabled
placeholder col-2" aria-disabled="true"></button>
    </div>
</div>
</div>

    <div class=" card p-0 border border-2
border-info-subtle shadow bg-body-tertiary rounded
placeholder-wave">

        <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
font-monospace text-center
text-info-emphasis fw-bolder">

```



```

        <span class="placeholder w-75 bg-info
placeholder-lg"></span>

    </div>

    <div class="card-body bg-light font-monospace">
        <div class="card-text text-secondary-emphasis
fw-light">

            <p class="placeholder w-100
bg-secondary"></p>

            <p class="placeholder w-100
bg-secondary"></p>

        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">

            <p class="placeholder w-100
bg-secondary"></p>

            <ul>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                </li>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                </li>

```

```

                <li>
                    <span class="placeholder w-25
bg-info me-3"></span>
                    <span class="placeholder w-50
bg-secondary "></span>
                </li>
                <li>
                    <span class="placeholder w-25
bg-info me-3"></span>
                    <span class="placeholder w-50
bg-secondary "></span>
                </li>
                <li>
                    <span class="placeholder w-25
bg-info me-3"></span>
                    <span class="placeholder w-50
bg-secondary "></span>
                </li>
            </ul>
        </div>

        <div class="d-flex justify-content-center">
            <button class="btn btn-info disabled
placeholder col-2" aria-disabled="true"></button>
        </div>
    </div>
</div>
</div>

```

```

        <div class="row gap-4 align-self-start" style="flex: 33%;
max-width: 33%">

            <div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded">

                <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle
font-monospace text-center
text-info-emphasis fw-bolder">

                    Student Table

                </div>

                <div class="card-body bg-light font-monospace">

                    <div class="card-text text-secondary-emphasis
fw-light">

                        <p>Table with data on students of a certain
higher education institution in Ukraine.</p>

                    </div>

                    <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">

                        <p class="p-0 m-0">The table contains the
following information about students:</p>

                        <ul>

                            <li><code class="text-info">Long</code>
unique identifier;</li>

                            <li><code
class="text-info">Entrant</code> owner of personal
information;</li>

```

```

        </li><code
class="text-info">FundingType</code> funding type under the study
agreement

        (contract or budget);

    </li>

    <li><code
class="text-info">ScholarshipStatus</code> scholarship accrual
status (none,

        ordinary or enhanced);

    </li>

    <li><code
class="text-info">String</code> address of corporate mail in NTU
"KhPI".</li>

</ul>

</div>

<div class="d-flex justify-content-center">
    <a th:href="@{/students}" class="btn
btn-outline-info" role="button">
        <i class="bi bi-table"></i>
    </a>
</div>
</div>
</div>

<div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded placeholder-wave">
    <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle

```

```

        font-monospace text-center
text-info-emphasis fw-bolder">
        <span class="placeholder w-75 bg-info
placeholder-lg"></span>
    </div>

    <div class="card-body bg-light font-monospace">
        <div class="card-text text-secondary-emphasis
fw-light">
            <p class="placeholder w-100
bg-secondary"></p>
            <p class="placeholder w-100
bg-secondary"></p>
        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
            <p class="placeholder w-100
bg-secondary"></p>
            <ul>
                <li>
                    <span class="placeholder w-25
bg-info me-3"></span>
                    <span class="placeholder w-50
bg-secondary "></span>
                </li>
                <li>
                    <span class="placeholder w-25
bg-info me-3"></span>

```

```

        <span class="placeholder w-50
bg-secondary "></span>

        </li>

        <li>

            <span class="placeholder w-25
bg-info me-3"></span>

            <span class="placeholder w-50
bg-secondary "></span>

            </li>

            <li>

                <span class="placeholder w-25
bg-info me-3"></span>

                <span class="placeholder w-50
bg-secondary "></span>

                </li>

                <li>

                    <span class="placeholder w-25
bg-info me-3"></span>

                    <span class="placeholder w-50
bg-secondary "></span>

                    </li>

                </ul>

            </div>

            <div class="d-flex justify-content-center">

                <button class="btn btn-info disabled
placeholder col-2" aria-disabled="true"></button>

            </div>

        </div>

    </div>

```

```

        <div class="card p-0 border border-2 border-info-subtle
shadow bg-body-tertiary rounded placeholder-wave">

            <div class="card-header bg-info-subtle
border-bottom border-2 border-info-subtle font-monospace
text-center text-info-emphasis fw-bolder">

                <span class="placeholder w-75 bg-info
placeholder-lg"></span>

            </div>

            <div class="card-body bg-light font-monospace">

                <div class="card-text text-secondary-emphasis
fw-light">

                    <p class="placeholder w-100
bg-secondary"></p>

                    <p class="placeholder w-100
bg-secondary"></p>

                </div>

                <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">

                    <p class="placeholder w-100
bg-secondary"></p>

                    <ul>

                        <li>

                            <span class="placeholder w-25
bg-info me-3"></span>

                            <span class="placeholder w-50
bg-secondary "></span>

                        </li>

```

```

        <li>
            <span class="placeholder w-25
bg-info me-3"></span>
            <span class="placeholder w-50
bg-secondary "></span>
        </li>
        <li>
            <span class="placeholder w-25
bg-info me-3"></span>
            <span class="placeholder w-50
bg-secondary "></span>
        </li>
        <li>
            <span class="placeholder w-25
bg-info me-3"></span>
            <span class="placeholder w-50
bg-secondary "></span>
        </li>
        <li>
            <span class="placeholder w-25
bg-info me-3"></span>
            <span class="placeholder w-50
bg-secondary "></span>
        </li>
    </ul>
</div>

<div class="d-flex justify-content-center">
    <button class="btn btn-info disabled
placeholder col-2" aria-disabled="true"></button>

```



```

        </div>

    </div>

</div>

</div>

</div>

</div>

</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

    crossorigin="anonymous"></script>
</body>
</html>

```

Б.4 Код компонента NavBar.html

```

<header th:fragment="header">
    <nav class="container-fluid navbar navbar-expand-lg
bg-info-subtle text-info-emphasis font-monospace">
        <div class="mx-3">
            <div class="navbar-brand text-info">
                
                <span class="p-0 m-0 ms-2">Student Database
Viewer</span>

```

```

        </div>
    </div>

    <div class="mx-3">
        <button class="navbar-toggler" type="button"
data-bs-toggle="collapse"
            data-bs-target="#navbarNavAltMarkup"
            aria-controls="navbarNavAltMarkup"
aria-expanded="false" aria-label="Toggle navigation">
            <span class="navbar-toggler-icon"></span>
        </button>
    </div>

    <div class="collapse navbar-collapse mx-3"
id="navbarNavAltMarkup">
        <div class="navbar-nav font-monospace text-info">
            <div class="nav-item me-5 text-info">
                <a class="nav-link text-info"
aria-current="page" th:href="@{/home}">Home</a>
            </div>

            <div class="nav-item dropdown me-5
align-content-center text-info">
                <div class="nav-item dropdown-toggler"
data-bs-toggle="dropdown" aria-expanded="false">
                    Tables
                </div>

                <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
                    <li>
                        <a class="dropdown-item"
th:href="@{/entrants}">Entrant Table</a>
                    </li>
                    <li>

```

```

                <a class="dropdown-item"
th:href="@{/students}">Student Table</a>
            </li>
        </ul>
    </div>

    <div class="nav-item dropdown me-5
align-content-center text-info">
        <div class="nav-item dropdown-toggle"
data-bs-toggle="dropdown" aria-expanded="false">
            Add Record
        </div>

        <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
            <li>
                <a class="dropdown-item"
th:href="@{/entrants/add}">Add Entrant</a>
            </li>
            <li>
                <a class="dropdown-item "
th:href="@{/students/add}">Add Student</a>
            </li>
        </ul>
    </div>

    <div class="nav-item">
        <a class="nav-link disabled text-body-tertiary"
aria-disabled="true">Further more</a>
    </div>
</div>
</div>
</nav>
</header>

```

Б.5 Код компонента Footer.html

```

<footer th:fragment="footer"
    class="container-fluid mt-5 p-3 text-center fs-6
bg-info-subtle text-info-emphasis font-monospace">
    <div class="row w-100 align-items-start justify-content-center
p-0 m-0">
        <div class="col justify-content-around" id="footer-author">
            <div class="col mb-2 justify-content-around fw-bold
text-info">Powered by</div>
            <div id="author-name">Inessa Repeshko</div>
            <div id="author-group">CS-222a</div>
            <div id="author-company">
                <span id="company-facility">CSIT </span>
                <span id="company-department">SEMIT</span><br>
                <span id="company-name">NTU "KhPI"</span>
            </div>
        </div>

        <address class="col justify-content-around"
id="footer-address">
            <div class="col mb-2 justify-content-around fw-bold
text-info">Address</div>
            <div id="address-name">National Technical University
"Kharkiv Polytechnic Institute"</div>
            <div id="address-street">2 Kyrpychova</div>
            <div>
                <span id="address-postal-code">61002</span>
                <span id="address-locality">Kharkiv</span>,<br>
                <span id="address-region">Ukraine</span>
            </div>
        </address>

        <address class="col justify-content-around"
id="footer-contact">

```

```

        <div class="col mb-2 justify-content-around fw-bold
text-info">Contact Us</div>

        <div id="contact-telephone">
            <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
            id="contact-telephone-link"
href="tel:+38(057)7076634">+38- (057) -707-66-34</a>
        </div>

        <div id="contact-email">
            <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
            id="contact-email-link"
href="mailto:omsroot@kpi.kharkov.ua">omsroot@kpi.kharkov.ua</a>
        </div>
    </address>
</div>

    <div class="row w-100 align-items-center">
        <div class="col justify-content-around"
id="footer-copyright">
            <p class="p-0 m-0">Copyright &copy; 2024 All Rights
Reserved</p>
        </div>
    </div>
</footer>

```

Б.6 Код компонента ErrorToast.html

```

<div th:fragment="errorToast" th:if="{errorMessage != null}">
    <div role="alert" class="alert alert-danger alert-dismissible
fade show z-1000 fixed-top top-0 start-50
    translate-middle-x border-2 border-danger-subtle">
        <div class="alert-heading">
            <p class="text-center fw-bold">
                <i class="bi bi-exclamation-octagon-fill"></i>

```

```

        <span class="ms-2" th:switch="{mode}">
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).ADD}"
                th:text="'Failed to save'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).EDIT}"
                th:text="'Failed to update'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).DELETE}"
                th:text="'Failed to delete'"></span>
            <span th:case="*"
                th:text="'Action Unsuccessful'"></span>
        </span>
    </p>

    <button type="button" class="btn-close"
data-bs-dismiss="alert" aria-label="Close"></button>
</div>
<hr>

    <p class="mb-0 text-center" th:text="{errorMessage}"></p>
</div>
</div>

```

Б.7 Код сторінки SuccessToast.java

```

<div th:fragment="successToast" th:if="{successMessage != null}">
    <div role="alert" class="alert alert-success alert-dismissible
fade show z-1000 fixed-top top-0 start-50
        translate-middle-x border-2 border-success-subtle">
        <div class="alert-heading">

```

```

        <p class="text-center fw-bold">
            <i class="bi bi-check-circle-fill"></i>

            <span class="ms-2" th:switch="{mode}">
                <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).EDIT}"
                    th:text="{isSaved != null && isSaved ?
'Successfully saved' : 'Successfully updated'}"></span>
                <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).VIEW_TABLE}"
                    th:text="'Successfully deleted'"></span>
                <span th:case="*"
                    th:text="'Action Successful'"></span>
            </span>
        </p>

        <button type="button" class="btn-close"
data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
    <hr>

    <p class="mb-0 text-center"
th:text="{successMessage}"></p>
</div>
</div>

```

Б.8 Код сторінки ErrorPage.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">

```

```

    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title th:text="{errorTitle}">Error</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"

integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body class="d-flex flex-column min-vh-100">

<div th:if="{errorCode != 401}">
    <header th:replace="~{component/NavigationBar :: header}"
class="flex-shrink-0"></header>
</div>

<main class="container-fluid flex-fill d-flex align-items-center
justify-content-center">
    <div class="row w-100">
        <div class="card border border-2 border-danger-subtle
            shadow bg-danger-subtle rounded mx-auto my-auto
w-50 p-0">
            <div class="card-header bg-danger-subtle border-bottom
border-2 border-danger-subtle
                font-monospace text-center
text-secondary-emphasis fw-bolder">

```



```

        <span class="badge text-bg-danger"
th:text="{errorCode}">500</span>
        <span th:text="{errorTitle}">Error</span>
    </div>

    <div class="card-body bg-light font-monospace">
        <div class="card-text text-secondary-emphasis
fw-light text-center">
            <p th:text="{errorMessage}">Oops! Something
went wrong.</p>
            <p th:text="{errorException}"
th:if="{errorException != null}"></p>
        </div>
    </div>
</div>

<div class="d-flex justify-content-center mt-5"
th:switch="{errorCode}">
    <button th:case="401" class="btn btn-outline-info"
type="button" onclick="window.history.back()">
        <i class="bi bi-box-arrow-in-left"></i>
    </button>
    <a th:case="*" th:href="{@{/home}}" class="btn
btn-outline-info" type="button">
        <i class="bi bi-house"></i>
    </a>
</div>
</div>
</main>

<footer th:replace="{~{component/Footer :: footer}}"
class="flex-shrink-0"></footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

```

```

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"
        crossorigin="anonymous"></script>
</body>
</html>

```

Б.9 Код сторінки EntrantTable.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Entrant Table</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

```

```

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="mx-3 mt-3 mb-5 font-monospace">
  <div class="row">
    <h1 class="text-info fs-1 my-3 fw-semibold
text-center">Entrant Table</h1>
  </div>

  <div class="row align-items-start">
    <div class="col-2 mx-2">
      <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
          <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0
m-0">Search</p>
            <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
              Search by case number, full name
              (matching start, case-insensitive)
            </p>
          </div>
          <div class="card-text col mt-3"
id="formSearch"
          th:method="'get'"
          th:action="@{/entrants}">
            <div class="row mb-3">
              <label for="inputSearch"
class="visually-hidden">Search</label>
              <input type="search"
class="form-control border border-2 border-info-subtle"
id="inputSearch" name="search"
placeholder="Type to search...">

```

```

        </div>

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button" id="btn-search"
onclick="applySearch(this) "
                class="btn btn-outline-info
border-2" th:alt-title="'Apply Search'">
                <i class="bi bi-search"></i>
            </a>

            <a type="button" id="btn-search-reset"
onclick="resetSearch(this) "
                class="btn btn-outline-info border-2
button-reset" th:title="'Reset Search'">
                <i class="bi bi-x-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Filters</p>
            </div>

            <div class="card-subtitle row mt-2"
id="filterBirthday">
                <p class="text-info fs-6 fw-bold p-0
m-0">Birthday</p>
            </div>

```

```

        <form class="card-text col mt-2"
id="formFilterBirthday"
        th:method="'get'"
th:action="@{/entrants}">
        <div class="row mb-2">
            <label for="inputBirthdayStart"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                style="font-size:
0.8rem;">Start</label>
            <input type="date" class="form-control
border border-2 border-info-subtle"
                id="inputBirthdayStart"
name="birthdayStart">
        </div>

        <div class="row mb-3">
            <label for="inputBirthdayEnd"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                style="font-size:
0.8rem;">End</label>
            <input type="date" class="form-control
border border-2 border-info-subtle"
                id="inputBirthdayEnd"
name="birthdayEnd">
        </div>

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button"
id="btn-filter-birthday" onclick="applyFilterBirthday(this)"
                class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
                <i class="bi bi-funnel"></i>
            </a>

```

```

        <a type="button"
id="btn-filter-birthday-reset" onclick="resetFilterBirthday(this)"
        class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
            <i class="bi bi-x-lg"></i>
        </a>
    </div>
</form>

    <div class="card-subtitle row mt-4"
id="filterRatingScore">
        <p class="text-info fs-6 fw-bold p-0
m-0">RatingScore</p>
    </div>

    <form class="card-text col mt-2"
id="formFilterRatingScore"
        th:method="'get'"
th:action="@{/entrants}">
        <div class="row mb-2">
            <label for="inputRatingScoreMin"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"
                style="font-size:
0.8rem;">Start</label>
            <input type="number"
class="form-control border border-2 border-info-subtle"
                id="inputRatingScoreMin"
name="ratingScoreMin"
                th:min="120.001"
th:max="200.000" th:step="0.001">
        </div>

        <div class="row mb-3">
            <label for="inputRatingScoreMax"
class="form-label text-secondary fw-lighter py-0 px-1 m-0"

```



```

        <p class="text-info fs-5 fw-bold p-0
m-0">Add record</p>

        <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
            Add new record to the end of the table
        </p>
    </div>

    <form class="card-text col mt-3"
id="formInsert"
        th:method="'get'"
th:action="@{/entrants}">
        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a th:href="@{/entrants/add}"
id="btn-add-entrant"
                class="btn btn-outline-info
border-2" role="button" th:title="'Add record'">
                <i class="bi bi-plus-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>
</div>

<div class="col">
    <table id="entrantTable" class="table table-sm
table-hover font-monospace
        table-bordered border border-3
border-info-subtle shadow-lg">
        <thead class="table-info text-center fw-medium
text-secondary-emphasis">
            <tr>
                <th class="text-center align-content-center
text-info">#</th>

```



```

        <div class="p-0 m-0" th:each="fieldName :
${fields}">
            <th class="text-center
align-content-center">
                <div class="d-flex p-0 m-0">
                    <div class="flex-grow-1
align-content-center justify-content-center p-0 m-0">
                        <span class="">
                            <a class="sortLink
link-info link-offset-2 link-offset-3-hover link-underline
link-underline-opacity-0 link-underline-opacity-75-hover"
th:id="${fieldName[0]}"
th:text="${fieldName[1]}"
onclick="applySort(this)"></a>
                        </span>
                    </div>
                    <div class="d-none flex-shrink-1
align-content-center justify-content-end ms-2 p-0 m-0"
style="width: 20px;">
                        <button type="button"
class="btn bg-transparent p-0 m-0" title="Sort records">
                            <i class="bi
bi-sort-down-alt sortingAsc" th:title="'Ascending Sort'"></i>
                            <i class="bi bi-sort-down
sortingDesc" th:title="'Descending Sort'"></i>
                        </button>
                    </div>
                </div>
            </th>
        </div>

        <th class="text-center align-content-center
text-info">Operations</th>

```

```

        </tr>
    </thead>

    <tbody class="table-light fw-light
text-primary-emphasis" th:each="entrant, iterStat : ${entrants}">
        <tr>
            <td class="text-center align-content-center"
th:text="${iterStat.index + 1}"></td>
            <td class="text-start align-content-center"
th:text="${entrant.caseNumber}"></td>
            <td class="text-start align-content-center"
                th:text="${entrant.surname} + ' ' +
${entrant.name} + ' ' + ${entrant.patronymic}"></td>
            <td class="text-center align-content-center"
th:text="${#temporals.format(entrant.birthday,
'dd.MM.yyyy')}"></td>
            <td class="text-center align-content-center"
                th:style="'color:' + (${entrant.gender} ?
'#0d6efd' : '#d63384')"
                th:text="${entrant.gender} ? 'чоловік' :
'жінка'"></td>
            <td class="text-end align-content-center"
th:text="${#numbers.formatDecimal(entrant.ratingScore, 3, 3,
'COMMA')}"></td>
            <td class="text-center align-content-center">
                <div class="d-flex justify-content-evenly
p-0 m-0">
                    <a type="button"
th:href="@{/students/___${entrant.studentId}___/view}"
                    th:if="${entrant.studentId != null}"
                    class="col-1 p-0 m-0 border border-0
bg-transparent button-linked-entrant"
                    th:title="'Student info'">

```

```

                <i class="bi bi-person-rolodex
text-info"></i>

                </a>

                <span th:if="{entrant.studentId ==
null}" class="col-1"></span>

                <a type="button"
th:href="{/entrants/__${entrant.id}__/edit}"
                class="col-1 p-0 m-0 border border-0
bg-transparent button-update"
                th:title="'Edit record'">
                <i class="bi bi-pencil
text-warning"></i>

                </a>

                <a type="button"
th:href="{/entrants/__${entrant.id}__/delete}"
                class="col-1 p-0 m-0 border border-0
bg-transparent button-delete"
                th:title="'Delete record'">
                <i class="bi bi-trash3
text-danger"></i>

                </a>
            </div>
        </td>
    </tr>
</tbody>
</table>
<br>
</div>
</div>

<div class="d-flex justify-content-center">
    <a th:href="{/home}" class="btn btn-outline-info"
type="button">
        <i class="bi bi-house"></i>
    </a>
</div>

```

```

</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setSearchPatternAttr();
        setSearchInputValidity();

        setBirthdayLimit('inputBirthdayStart');
        setBirthdayLimit('inputBirthdayEnd');
        setBirthdayValidity('inputBirthdayStart');
        setBirthdayValidity('inputBirthdayEnd');

        setRatingScoreValidity('inputRatingScoreMin');
        setRatingScoreValidity('inputRatingScoreMax');

        setInputValueFromGetParams();
    });

    function setSearchPatternAttr() {
        const regexSearch = new
RegExp("[А-ЩЬЮЯЄІЇҐа-щьюяєііґ'\\"-\\d ]{1,200}$");

document.getElementById('inputSearch').setAttribute('pattern',
regexSearch.source);
    }

    function setSearchInputValidity() {
        const message = "Value is a Ukrainian a string between 1
and 200 chars long, optionally including numbers, dash,
apostrophe, space.";

document.getElementById('inputSearch').addEventListener('input',
function () {

```

```

        if (this.validity.patternMismatch) {
            this.setCustomValidity(message);
        } else {
            this.setCustomValidity('');
        }

        this.reportValidity();
    });
}

function setBirthdayLimit(inputId) {
    document.getElementById(inputId).setAttribute('min',
getBirthdayMinValue());

    document.getElementById(inputId).setAttribute('max',
getBirthdayMaxValue());
}

function getBirthdayMinValue() {
    const today = new Date();
    return new Date(
        today.getFullYear() - 110,
        today.getMonth(),
        today.getDate())
        .toISOString().split('T')[0];
}

function getBirthdayMaxValue() {
    const today = new Date();
    return new Date(today.getFullYear() - 16,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
}

function setBirthdayValidity(inputId) {
    const regexpDate = new
RegExp("(?=.{10}$)\\d{4}\\-\\d{2}\\-\\d{2}");
    const messages = [

```

```

        "Invalid date format. Check the entered value.",
        "Value is a date from 110 to 16 years ago today."
    ];

document.getElementById(inputId).addEventListener('mouseleave',
function () {
    const minDate = new Date(this.min);
    const maxDate = new Date(this.max);
    const selectedDate = regexpDate.test(this.value) ? new
Date(this.value) : null;

    if (!selectedDate) {
        this.setCustomValidity(messages[0]);
    } else if (selectedDate < minDate || selectedDate >
maxDate) {
        this.setCustomValidity(messages[1]);
    } else {
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setRatingScoreValidity(inputId) {
    const message = "Value is a decimal number in range from
120.001 to 200.000 inclusive.";

    document.getElementById(inputId).addEventListener('input',
function () {
        const minVal = parseFloat(this.min);
        const maxVal = parseFloat(this.max);
        const selectedVal = parseFloat(this.value);

        if (selectedVal < minVal || selectedVal > maxVal) {

```

```

        this.setCustomValidity(message);
    } else {
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setInputValueFromGetParams() {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);
    const inputFields = document.getElementsByTagName("input");

    for (const input of inputFields) {
        if (params.has(input.name)) {
            input.value = params.get(input.name);
        }
    }
}

function applySort(element) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const paramName = "sort";
    const paramFieldName = element.id;
    let paramSortOrder = 'asc';
    const currentSortValue = params.get(paramName);

    if (currentSortValue) {
        const currentSortFieldName =
currentSortValue.split("-")[0];

```

```

        const currentSortOrder = currentSortFieldName ===
paramFieldName ? currentSortValue.split("-")[1] : '';
        paramSortOrder = currentSortOrder === 'asc' ? 'desc' :
'asc';
    }

    params.set(paramName, paramFieldName + '-' +
paramSortOrder);

    const newURL = currentURL.pathname + '?' +
params.toString();
    element.setAttribute('href', newURL);
}

function applySearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    setFilteringParameter(params,
document.getElementById("inputSearch"));

    const newURL = currentURL.pathname + '?' +
params.toString();
    linkButton.setAttribute('href', newURL);
}

function resetSearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    resetFilteringParameter(params,
document.getElementById("inputSearch"));

```



```

        const newURL = currentURL.pathname + '?' +
params.toString();

        document.getElementById("formSearch").reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterBirthday(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        setFilteringParameter(params,
document.getElementById("inputBirthdayStart"));
        setFilteringParameter(params,
document.getElementById("inputBirthdayEnd"));

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterBirthday(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById("inputBirthdayStart"));
        resetFilteringParameter(params,
document.getElementById("inputBirthdayEnd"));

        const newURL = currentURL.pathname + '?' +
params.toString();
        document.getElementById("formFilterBirthday").reset();
        linkButton.setAttribute('href', newURL);
    }

```

```

function applyFilterRatingScore(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    setFilteringParameter(params,
document.getElementById("inputRatingScoreMin"));
    setFilteringParameter(params,
document.getElementById("inputRatingScoreMax"));

    const newURL = currentURL.pathname + '?' +
params.toString();
    linkButton.setAttribute('href', newURL);
}

function resetFilterRatingScore(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    resetFilteringParameter(params,
document.getElementById("inputRatingScoreMin"));
    resetFilteringParameter(params,
document.getElementById("inputRatingScoreMax"));

    const newURL = currentURL.pathname + '?' +
params.toString();
    document.getElementById("formFilterRatingScore").reset();
    linkButton.setAttribute('href', newURL);
}

function setFilteringParameter(params, inputElement) {
    const paramName = inputElement.name;
    const paramValue = inputElement.value.toString().trim();

```

```

        if (paramValue.length > 0) {
            params.set(paramName, paramValue);
        } else {
            params.delete(paramName);
        }
    }

    function resetFilteringParameter(params, inputElement) {
        const paramName = inputElement.name;
        params.delete(paramName);
    }
}

</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

</body>
</html>

```

Б.10 Код сторінки EntrantForm.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title th:if="${mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).ADD}"
th:text="'Add Entrant'"></title>

```

```

<title th:if="${mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}"
    th:text="'View Entrant'"></title>
<title th:if="${mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT}"
    th:text="'Edit Entrant'"></title>
<title th:if="${mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}"
    th:text="'Delete Entrant'"></title>
<link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
    rel="stylesheet"
    integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
    crossorigin="anonymous">
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-50 text-center justify-content-center">
    <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0">

```

```

        <div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder"
        th:switch="{mode}">
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).ADD}"
                th:text="'Add Entrant'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).VIEW_FORM}"
                th:text="'View Entrant'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).EDIT}"
                th:text="'Edit Entrant'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).DELETE}"
                th:text="'Delete Entrant'"></span>
            <span th:case="*"
                th:text="'Entrant Form'"></span>
        </div>

        <form class="card-body bg-light font-monospace p-5 m-0"
id="entrantForm"
        th:object="{entrant}"
        th:with="actionUrl=
            {mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).ADD ?
'/entrants/add' :
            (mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
? '/entrants/view' :

```

```

        (mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT ?
('/entrants/' + entrant.id + '/edit') :
        (mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE ?
('/entrants/' + entrant.id + '/delete') : '')))}"
        th:action="@${actionUrl}" th:method="'post'">

        <div class="d-none card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded ">
                <label for="inputEntrantId" class="col-3
input-group-text text-info">Id</label>
                <input id="inputEntrantId" class="col-auto
form-control text-dark" type="hidden" required
                    th:field="*{id}" th:minlength="1"
th:maxlength="100"
                    th:value="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
? entrant.id : null}"
                    th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

            </div>
        </div>

        <div class="card-text mb-4">
            <div class="input-group border border-1
border-info-subtle rounded ">
                <label for="inputCaseNumber" class="col-3
input-group-text text-info">Case Number</label>

```

```

        <input id="inputCaseNumber" class="col-auto
form-control text-dark" type="text" required
        th:field="*{caseNumber}"
th:minlength="5" th:maxlength="20"
        th:value="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
        or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
        ? entrant.caseNumber :
null}"
        th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
        or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

    </div>
</div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputSurname" class=" col-3
input-group-text text-info">Surname</label>
            <input id="inputSurname" class="col-auto
form-control text-dark" type="text" required
                th:field="*{surname}" th:minlength="1"
th:maxlength="50"
                th:value="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
                or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                ? entrant.surname : null}"
                th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE

```

```

                                or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputName" class=" col-3
input-group-text text-info">Name</label>
            <input id="inputName" class="col-auto
form-control text-dark" type="text" required
                th:field="*{name}" th:minlength="1"
th:maxlength="50"
                th:value="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
                                or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                ? entrant.name : null}"
                th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputPatronymic" class=" col-3
input-group-text text-info">Patronymic</label>
            <input id="inputPatronymic" class="col-auto
form-control text-dark" type="text" required

```



```

        th:field="*{patronymic}"
th:minlength="1" th:maxlength="50"
        th:value="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
        or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
        ? entrant.patronymic :
null}"

        th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
        or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputBirthday" class="col-3
input-group-text text-info">Birthday</label>
        <input id="inputBirthday" class="col-auto
form-control text-dark" type="date" required
        th:name="{ 'birthday' }"
        th:value="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
        or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
        ?
#temporals.format(entrant.birthday, 'yyyy-MM-dd') : null}"
        th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
        or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

    </div>

```

```

        </div>

        <div class="card-text d-flex align-content-center
justify-content-start mb-4">
            <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
                <span class="d-flex flex-fill input-group-text
text-info">Gender</span>
            </div>

            <div class="col-auto btn-group" role="group"
aria-label="Gender radio toggle button group">
                <input type="hidden"
                    th:field="*{gender}"
                    th:value="{entrant.gender}"
                    th:if="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}">

                <input id="inputGenderMale" type="radio"
class="btn-check" autocomplete="off"
                    th:field="*{gender}"
                    th:value="{ 'true' }"
                    th:disabled="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

                <label class="btn btn-outline-info"
for="inputGenderMale">Male</label>

                <input id="inputGenderFemale" type="radio"
class="btn-check" autocomplete="off"
                    th:field="*{gender}"
                    th:value="{ 'false' }"
                    th:disabled="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE

```

```

                                or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        <label class="btn btn-outline-info"
for="inputGenderFemale">Female</label>

        </div>
    </div>

    <div class="card-text mb-4">
        <div class="input-group border border-1
border-info-subtle rounded ">
            <label for="inputRatingScore" class="col-3
input-group-text text-info">Rating score</label>
            <input id="inputRatingScore" class="col-auto
form-control text-dark" type="number" required
                th:min="120.001" th:max="200.000"
th:step="0.001"
                th:field="*{ratingScore}"
                th:readonly="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                or mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        </div>
    </div>

    <div class="d-flex justify-content-evenly">
        <button type="submit" class="btn btn-outline-info
border-2 button-apply"
            th:title="'Save record'"
            th:if="{mode !=
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                and mode !=
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        <i class="bi bi-floppy"></i>

```

```

        </button>

        <button type="reset" class="btn btn-outline-info
border-2 button-reset"
            onclick="form.reset()"
            th:title="'Reset fields'"
            th:if="{mode !=
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
and mode !=
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

            <i class="bi bi-x-lg"></i>
        </button>

        <button type="button" class="btn btn-outline-info
border-2 button-delete"
            onclick="submitConfirmation()"
            th:title="'Delete record'"
            th:if="{mode ==
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}">

            <i class="bi bi-trash3"></i>
        </button>
    </div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <button class="btn btn-outline-info" type="button"
        th:title="'Go Back'"
        onclick="window.history.back()">
        <i class="bi bi-box-arrow-in-left"></i>
    </button>

    <a class="btn btn-outline-info ms-5" type="button"
        th:title="'Go to Entrant Table'"
        th:href="{@{/entrants}}">

```

```

        <i class="bi bi-table"></i>
    </a>
</div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setAllInputsValidity();
    });

    function setAllInputsValidity() {
        setPatternAttr();
        setBirthdayLimit();

        setStringInputValidity();
        setBirthdayValidity();
        setRatingScoreValidity();

        const form = document.getElementById('entrantForm');

        form.addEventListener('submit', function (event) {
            event.preventDefault();

            if (!form.checkValidity()) {
                event.stopPropagation();
                form.reportValidity();
            } else {
                form.submit();
            }
        });

        function setPatternAttr() {
            const regexCaseNumber = new
RegExp("^(?=. {5,20}$) [А-ЩЮЯІЄІї]{1,10} \\d{2} [\\-] \\d{1,4}$");

```

```

        const regexNames = new
RegExp("^(?=.{1,50}$)[А-ЩЮЯҒҒІİ] (?:[А-ЯҒҒІİа-яҒҒіі']*[А-ЯҒҒІİа-яҒҒіі
і])?(?:[
\\-] [А-ЩЮЯҒҒІİ] (?:[А-ЯҒҒІİа-яҒҒіі']*[А-ЯҒҒІİа-яҒҒіі])?)*$");

document.getElementById('inputCaseNumber').setAttribute('pattern',
regexCaseNumber.source);

document.getElementById('inputSurname').setAttribute('pattern',
regexNames.source);

document.getElementById('inputName').setAttribute('pattern',
regexNames.source);

document.getElementById('inputPatronymic').setAttribute('pattern',
regexNames.source);
    }

    function setBirthdayLimit() {

document.getElementById('inputBirthday').setAttribute('min',
getBirthdayMinValue());

document.getElementById('inputBirthday').setAttribute('max',
getBirthdayMaxValue());

        function getBirthdayMinValue() {
            const today = new Date();
            return new Date(today.getFullYear() - 110,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
        }

        function getBirthdayMaxValue() {
            const today = new Date();

```

```

        return new Date(today.getFullYear() - 16,
today.getMonth(), today.getDate()).toISOString().split('T')[0];
    }
}

function setBirthdayValidity() {
    const regexpDate = new
RegExp("(?=.{10}$)\\d{4}\\-\\d{2}\\-\\d{2}");
    const messages = [
        "Birthday cannot be empty.",
        "Value must be date from 110 to 16 years ago
today."
    ];

document.getElementById('inputBirthday').addEventListener('submit'
, function () {
    const minDate = new Date(this.min);
    const maxDate = new Date(this.max);
    const selectedDate = new Date(this.value);

    if (!regexpDate.test(selectedDate.toString())) {
        this.setCustomValidity(messages[0]);
    } else if (selectedDate < minDate || selectedDate >
maxDate) {
        this.setCustomValidity(messages[1]);
    } else {
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setRatingScoreValidity() {

```

```

        const message = "Value must be decimal number in range
from 120.001 to 200.000 inclusive.";

document.getElementById('inputRatingScore').addEventListener('input', function () {
    const minVal = parseFloat(this.min);
    const maxVal = parseFloat(this.max);
    const selectedVal = parseFloat(this.value);

    if (selectedVal < minVal || selectedVal > maxVal) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity("");
    }

    this.reportValidity();
});
}

function setStringInputValidity() {
    const messages = [
        "Value must be Ukrainian char string between 5 and
20 chars long in format 'LLLLLLLLLLNN-NNNN'.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe.",
        "Value must be Ukrainian char string between 1 and
50 chars long, including space and apostrophe."
    ];

    document.getElementById('entrantForm')
        .querySelectorAll('input[pattern]')
        .forEach((input, i) => {
            input.addEventListener('input', function () {

```



```

        if (this.validity.patternMismatch) {
            this.setCustomValidity(messages[i]);
        } else {
            this.setCustomValidity('');
        }

        this.reportValidity();
    });
});
}

}

function submitConfirmation() {
    const message = 'Are you sure you want to delete that
record?';

    if (confirm(message) === true) {
        document.getElementById('entrantForm').submit();
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

</body>
</html>

```

Б.11 Код класу MainController.java

```
package ntukhpi.csit.semit.riv.webappsrivlab3.controller;

import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.SessionAttributes;

@Controller
@SessionAttributes("userLogin")
public class MainController {

    @GetMapping("/")
    public String showStartPage() {
        return "login/Login";
    }

    private static final String LOGIN = "irepeshko";
    private static final String PASSWORD = "password";

    @PostMapping("/process-login")
    public String processLogin(
        @RequestParam("login") String login,
        @RequestParam("password") String password,
        HttpSession session,
        Model model) {

        if (LOGIN.equals(login) && PASSWORD.equals(password)) {
            session.setAttribute("userLogin", login);
            model.addAttribute("userLogin", login);

            return "redirect:/home";
        }
    }
}
```

```

        } else {
            return "redirect:/unauthorized-error";
        }
    }

    @GetMapping("/home")
    public String showHomePage(Model model) {
        return "home/Home";
    }

    @GetMapping("/unauthorized-error")
    public String showUnauthorizedErrorPage(Model model) {
        model.addAttribute("errorTitle", "Unauthorized Error");
        model.addAttribute("errorCode", 401);
        model.addAttribute("errorMessage", "Unauthorized access.
Please check your credentials.");

        return "error/ErrorPage";
    }
}

```

Б.12 Код класу EntrantController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.controller;

import jakarta.validation.Valid;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.stereotype.Controller;

```

```

import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import
org.springframework.web.servlet.mvc.support.RedirectAttributes;
import org.springframework.web.util.UriComponentsBuilder;

import java.time.LocalDate;
import java.util.List;

@Controller
public class EntrantController {
    private static final Logger logger =
LoggerFactory.getLogger(EntrantController.class);
    private EntrantService entrantService;

    public EntrantController(EntrantService entrantService) {
        super();
        this.entrantService = entrantService;
    }

    @GetMapping("/entrants")
    public String showEntrantTable(
        @RequestParam(value = "search", required = false)
String search,
        @RequestParam(value = "birthdayStart", required =
false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate
birthdayStart,
        @RequestParam(value = "birthdayEnd", required = false)
@DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate
birthdayEnd,
        @RequestParam(value = "ratingScoreMin", required =
false) Double ratingScoreMin,
        @RequestParam(value = "ratingScoreMax", required =
false) Double ratingScoreMax,

```

```

        @RequestParam(value = "sort", required = false,
defaultValue = "id-asc") String sort,
        Model model) {
            model.addAttribute("fields",
Entrant.getFieldNamesAsFormattedStrings());
            model.addAttribute("mode", Mode.VIEW_TABLE);

            try {
                List<Entrant> entrants =
entrantService.getFilteredAndSortedEntrants(
                    search,
                    birthdayStart,
                    birthdayEnd,
                    ratingScoreMin,
                    ratingScoreMax,
                    sort);

                model.addAttribute("entrants", entrants);
            } catch (Exception e) {
                model.addAttribute("action", "load table");
                model.addAttribute("errorMessage", e.getMessage());
            }

            return "entity/entrant/EntrantTable";
        }

@GetMapping("/entrants/add")
public String showCreateEntrantForm(Model model) {
    /*Entrant entrant = new Entrant("KH22-0", "Репешко",
"Інесса", "Віталіївна", "01.01.2000", "FALSE", "120.001");*/
    Entrant entrant = new Entrant();

    model.addAttribute("entrant", entrant);
    model.addAttribute("mode", Mode.ADD);

    return "entity/entrant/EntrantForm";
}

```

```

    }

    @PostMapping("/entrants/add")
    public String saveEntrant(@Valid @ModelAttribute("entrant")
Entrant entrantToSave,

                                BindingResult result,
                                RedirectAttributes
redirectAttributes,

                                Model model) {
        model.addAttribute("mode", Mode.ADD);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
IllegalArgumentException(result.getAllErrors().toString());
            }

            Entrant savedEntrant =
entrantService.saveEntrant(entrantToSave);

            redirectAttributes.addFlashAttribute("entrant",
savedEntrant);
            redirectAttributes.addFlashAttribute("successMessage",
"The record successfully saved to the database.");
            redirectAttributes.addFlashAttribute("isSaved", true);

            String redirectUrl =
UriComponentsBuilder.fromPath("/entrants/{id}/edit")
                        .buildAndExpand(savedEntrant.getId())
                        .toUriString();

            return "redirect:" + redirectUrl;
        } catch (Exception e) {
            model.addAttribute("entrant", entrantToSave);

```

```

        model.addAttribute("action", "save");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/entrant/EntrantForm";
    }
}

@GetMapping("/entrants/{id}/view")
public String showViewEntrantForm(@PathVariable Long id,
                                   Model model) {
    model.addAttribute("mode", Mode.VIEW_FORM);

    try {
        Entrant entrantToView =
entrantService.findEntrantById(id);

        model.addAttribute("entrant", entrantToView);

        return "entity/entrant/EntrantForm";
    } catch (Exception e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");
        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@GetMapping("/entrants/{id}/edit")
public String showUpdateEntrantForm(@PathVariable Long id,
                                   Model model) {
    model.addAttribute("mode", Mode.EDIT);

```

```

        try {
            Entrant entrantToUpdate =
entrantService.findEntrantById(id);

            model.addAttribute("entrant", entrantToUpdate);

            return "entity/entrant/EntrantForm";
        } catch (Exception e) {
            model.addAttribute("errorTitle", "Page Not Found
Error");

            model.addAttribute("errorCode", 404);
            model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
            model.addAttribute("errorException", e.getMessage());

            return "error/ErrorPage";
        }
    }

    @PostMapping("/entrants/{id}/edit")
    public String updateEntrant(@Valid @ModelAttribute("entrant")
Entrant entrantToUpdate,

                                BindingResult result,
                                Model model) {
        model.addAttribute("mode", Mode.EDIT);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
IllegalArgumentException(result.getAllErrors().toString());
            }

            Entrant updatedEntrant =
entrantService.updateEntrant(entrantToUpdate);

```



```

        model.addAttribute("entrant", updatedEntrant);
        model.addAttribute("successMessage", "The record
successfully updated in the database.");

        return "entity/entrant/EntrantForm";
    } catch (Exception e) {
        model.addAttribute("entrant", entrantToUpdate);
        model.addAttribute("action", "update");
        model.addAttribute("errorMessage", e.getMessage());

        return "entity/entrant/EntrantForm";
    }
}

@GetMapping("/entrants/{id}/delete")
public String showDeleteEntrantForm(@PathVariable Long id,
                                    Model model) {
    model.addAttribute("mode", Mode.DELETE);

    try {
        Entrant entrantToDelete =
entrantService.findEntrantById(id);

        model.addAttribute("entrant", entrantToDelete);

        return "entity/entrant/EntrantForm";
    } catch (Exception e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");
        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

```

```

    }

    }

    @PostMapping("/entrants/{id}/delete")
    public String deleteEntrant(@Valid @ModelAttribute("entrant")
Entrant entrantToDelete,

                                BindingResult result,
                                RedirectAttributes
redirectAttributes,

                                Model model) {
        model.addAttribute("mode", Mode.DELETE);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
IllegalArgumentException(result.getAllErrors().toString());
            }

            entrantService.deleteEntrant(entrantToDelete);

            redirectAttributes.addFlashAttribute("successMessage",
"The record successfully deleted from the database.");

            return "redirect:/entrants";
        } catch (Exception e) {
            model.addAttribute("entrant", entrantToDelete);
            model.addAttribute("action", "delete");
            model.addAttribute("errorMessage", e.getMessage());

            return "entity/entrant/EntrantForm";
        }
    }
}

```

Б.13 Результати роботи вебзастосунку відповідно до реалізації коду до пунктів 1 та 2 лабораторної роботи

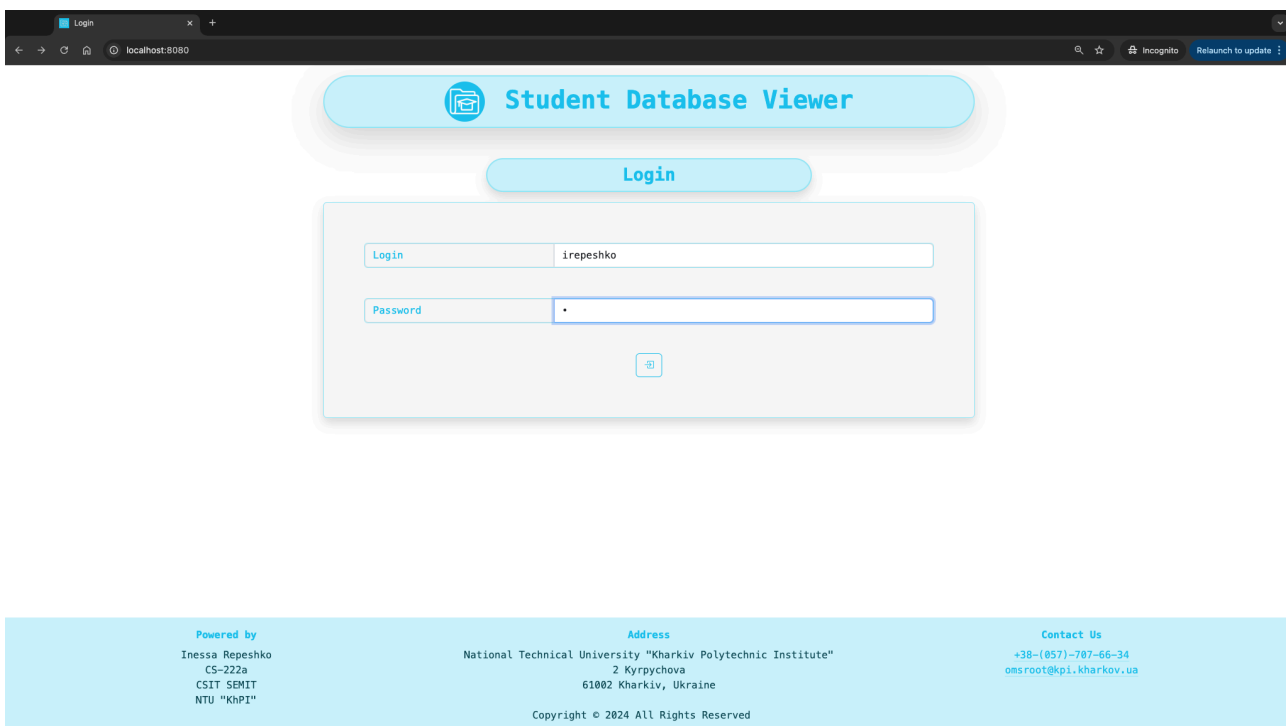


Рисунок Б.13.1 – Сторінка авторизації на URL “`http://localhost:8080/`”

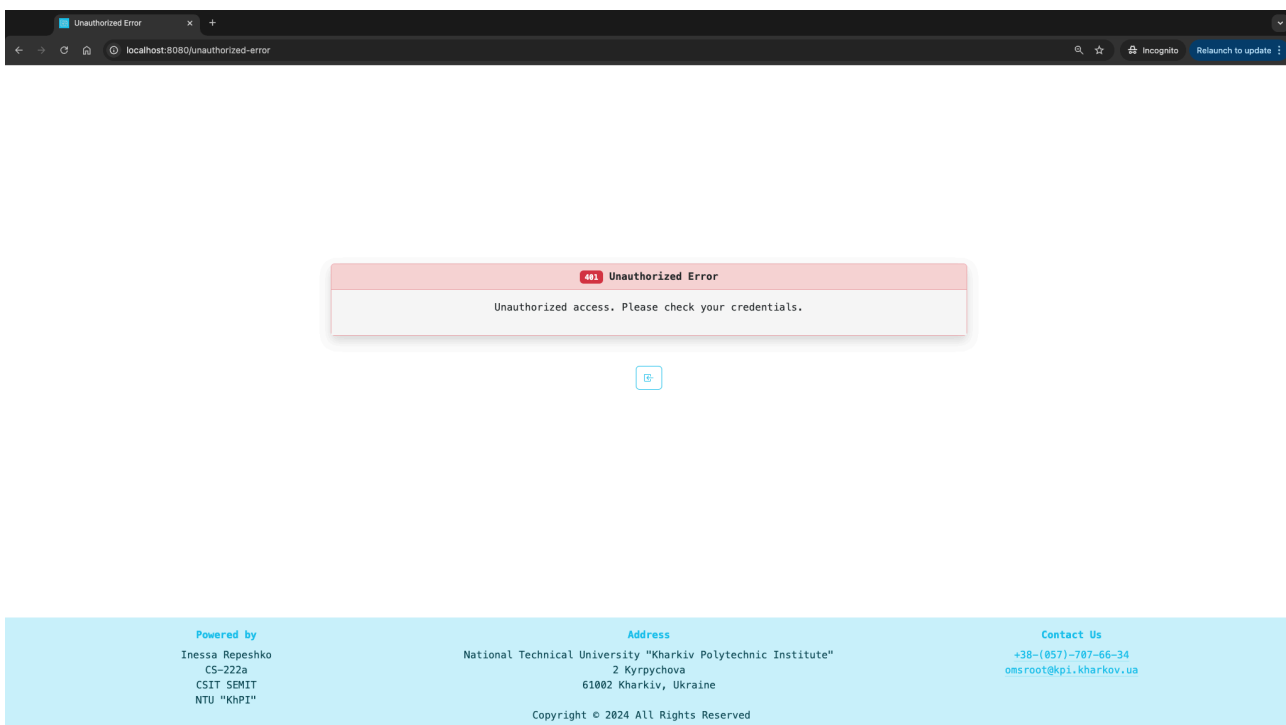


Рисунок Б.13.2 – Сторінка помилки авторизації на URL “`http://localhost:8080/unauthorized-error`”

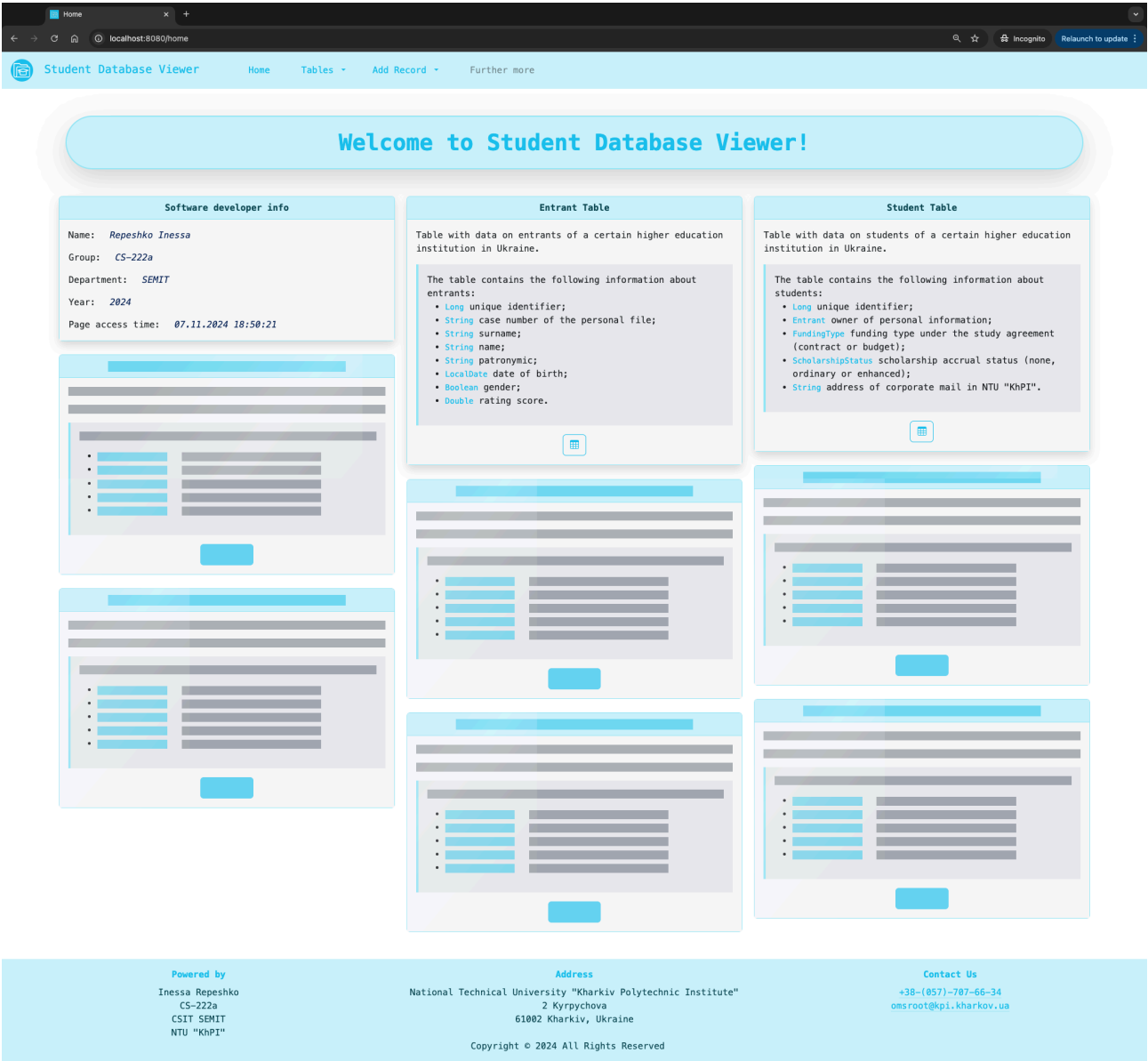


Рисунок Б.13.3 – Домашня сторінка на URL “http://localhost:8080/home”

Entrant Table

localhost:8080/entrants

Student Database Viewer

HomeTablesAdd RecordFurther more

Search

Search by case number, full name
(matching start, case-insensitive)

Type to search...

Q

X

Filters

Birthday

Start

dd.mm.yyyy

End

dd.mm.yyyy

▽

X

RatingScore

Start

End

▽

X

Add record

Add new record to the end of the table

+

Entrant Table

#	Case Number	Full Name	Birthday	Gender	Rating Score	Operations
1	KN22-4814	Бабічев Олексій Вікторович	22.05.2005	чоловік	168,111	
2	KN22-5835	Волков Дмитро Олександрович	16.07.2005	чоловік	151,202	
3	KN22-5511	Гєєнко Анна Олексіївна	03.12.2005	жінка	150,013	
4	KN22-9299	Голоп'юров Микита Сергійович	07.11.2005	чоловік	161,484	
5	KN22-9012	Гриценко Максим Георгійович	04.12.2005	чоловік	191,945	
6	KN22-6636	Жарий Вадим Віталійович	25.10.2005	чоловік	128,346	
7	KN22-5140	Жупанов Андрій Володимирович	22.06.2005	чоловік	183,627	
8	KN22-4896	Кльоз Михайло Романович	15.01.2005	чоловік	127,288	
9	KN22-4694	Коваль Микита Артемович	11.10.2005	чоловік	139,739	
10	KN22-5529	Ковтун Єгор Вячеславович	10.04.2005	чоловік	187,790	
11	KN22-6874	Колесниченко Денис Євгенович	26.03.2005	чоловік	165,641	
12	KN22-2947	Ласкевич Андрій Сергійович	05.09.2005	чоловік	175,392	
13	KN22-3856	Лукаш Антон Олександрович	24.11.2005	чоловік	184,473	
14	KN22-7534	Палій Нікіта Андрійович	05.05.2005	чоловік	168,004	
15	KN22-3843	Прокопов Владислав Сергійович	12.12.2004	чоловік	164,665	
16	KN22-7819	Репешко Інесса Віталіївна	06.05.2005	жінка	185,306	
17	KN22-9115	Скиба Андрій Григорович	26.02.2004	чоловік	125,907	
18	KN22-3812	Цибань Ольга Юріївна	19.05.2004	жінка	190,298	
19	KN22-1828	Шевченко Данило Валерійович	25.10.2004	чоловік	166,849	
20	KN22-9514	Антипенко Катерина Віталіївна	06.01.2005	жінка	168,460	
21	KN22-2929	Багрянцева Марина Русланівна	01.10.2005	жінка	157,631	
22	KN22-1637	Бондаренко Ярослав Юрійович	24.04.2005	чоловік	199,532	
23	KN22-6681	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683	
24	KN22-7557	Васильєв Михайло Олександрович	25.01.2005	чоловік	170,864	
25	KN22-8759	Деркач Дарина Євгенівна	03.03.2005	жінка	171,715	
26	KN22-7309	Ілехін Тимур Ігорович	24.01.2005	чоловік	125,776	
27	KN22-8254	Криженко Аліна Степанівна	29.08.2005	жінка	138,747	
28	KN22-3825	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718	
29	KN22-8153	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309	
30	KN22-4788	Сизоненко Єлизавета Олександрівна	17.02.1997	жінка	124,240	
31	KN22-8255	Криженко Аліна Степанівна	29.08.2005	жінка	138,747	
32	KN22-3826	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718	
33	KN22-8154	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309	
34	KN22-6682	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683	
35	KN22-9998	Марчук Періна Віталіївна	06.06.1996	жінка	200,000	

Powered by

Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KHPI"

Address

National Technical University "Kharkiv Polytechnic Institute"
2 Kyrpychova
61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34
onsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Б.13.4 – Сторінка таблиці абітурієнтів на URL “http://localhost:8080/entrants”

Student Database Viewer

Home Tables Add Record Further more

Add Entrant

Case Number KH22-4242

Surname Якімов

Name Іван

Patronymic Романович

Birthday 01.01.1990

Gender ☒ Male ☐ Female

Rating score 200

Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU "KhPI"

Address National Technical University "Kharkiv Polytechnic Institute" 2 Kyrychova 61002 Kharkiv, Ukraine

Contact Us +38-(057)-707-66-34 omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Б.13.5 – Сторінка форми додавання абітурієнта на URL
“http://localhost:8080/entrants/add”

Student Database Viewer

Home

Successfully saved

The record successfully saved to the database.

Edit Entrant

Case Number KH22-4242

Surname Якімов

Name Іван

Patronymic Романович

Birthday 01.01.1990

Gender ☒ Male ☐ Female

Rating score 200,0

Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU "KhPI"

Address National Technical University "Kharkiv Polytechnic Institute" 2 Kyrychova 61002 Kharkiv, Ukraine

Contact Us +38-(057)-707-66-34 omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Б.13.6 – Сторінка форми додавання абітурієнта після збереження даних
на URL “http://localhost:8080/entrants/36/edit”

Student Database Viewer

Home Tables Add Record Further more

Edit Entrant

Case Number KH22-4814

Surname Бабічев

Name Олексій

Patronymic Вікторович

Birthday 22.05.2005

Gender ☒ Male ☐ Female

Rating score 168,111

Save Cancel

Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU "KhPI"

Address National Technical University "Kharkiv Polytechnic Institute" 2 Kyrychova 61002 Kharkiv, Ukraine

Contact Us +38-(057)-707-66-34 omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Б.13.7 – Сторінка форми редагування абітурієнта на URL
“http://localhost:8080/entrants/1/edit”

Student Database Viewer

Home

Successfully updated
The record successfully updated in the database.

Edit Entrant

Case Number KH22-3090

Surname Барвний

Name Лук'ян

Patronymic Ярославович

Birthday 01.01.2005

Gender ☒ Male ☐ Female

Rating score 170,0

Save Cancel

Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU "KhPI"

Address National Technical University "Kharkiv Polytechnic Institute" 2 Kyrychova 61002 Kharkiv, Ukraine

Contact Us +38-(057)-707-66-34 omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Б.13.8 – Сторінка форми редагування абітурієнта після збереження
даних на URL “http://localhost:8080/entrants/1/edit”

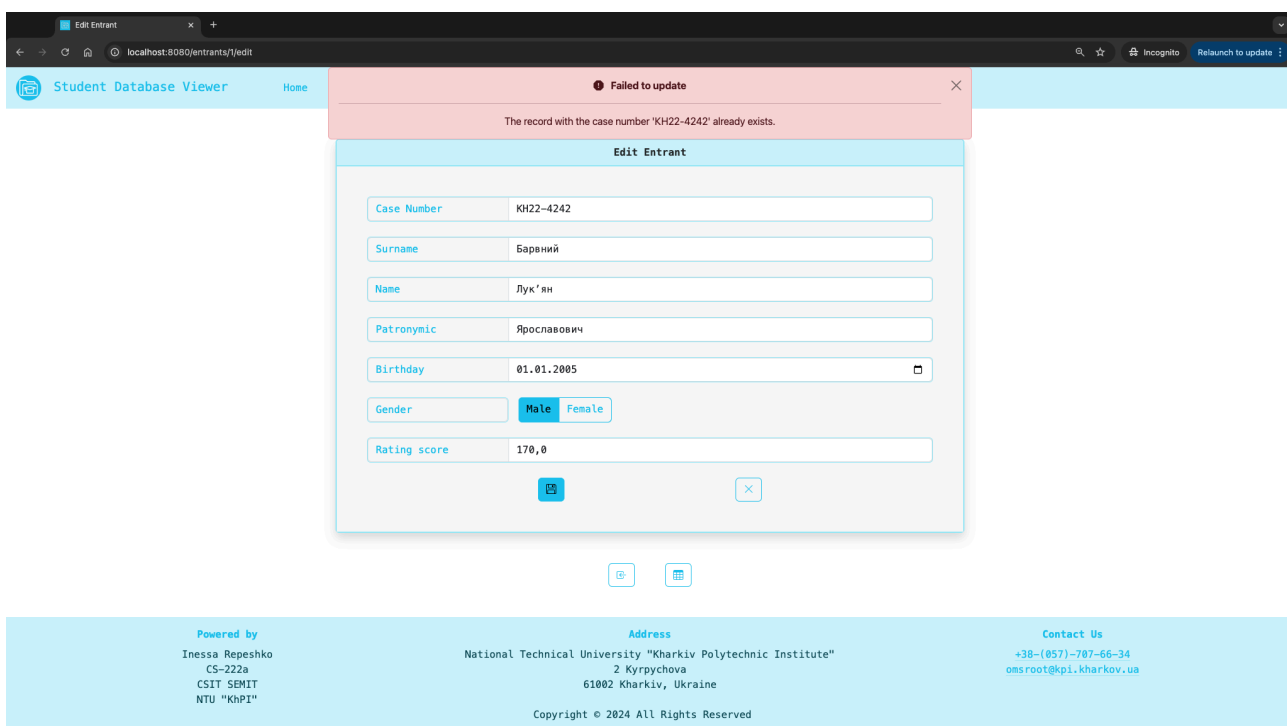


Рисунок Б.13.9 – Сторінка форми редагування абітурієнта на URL з помилкою дубліката “http://localhost:8080/entrants/1/edit”

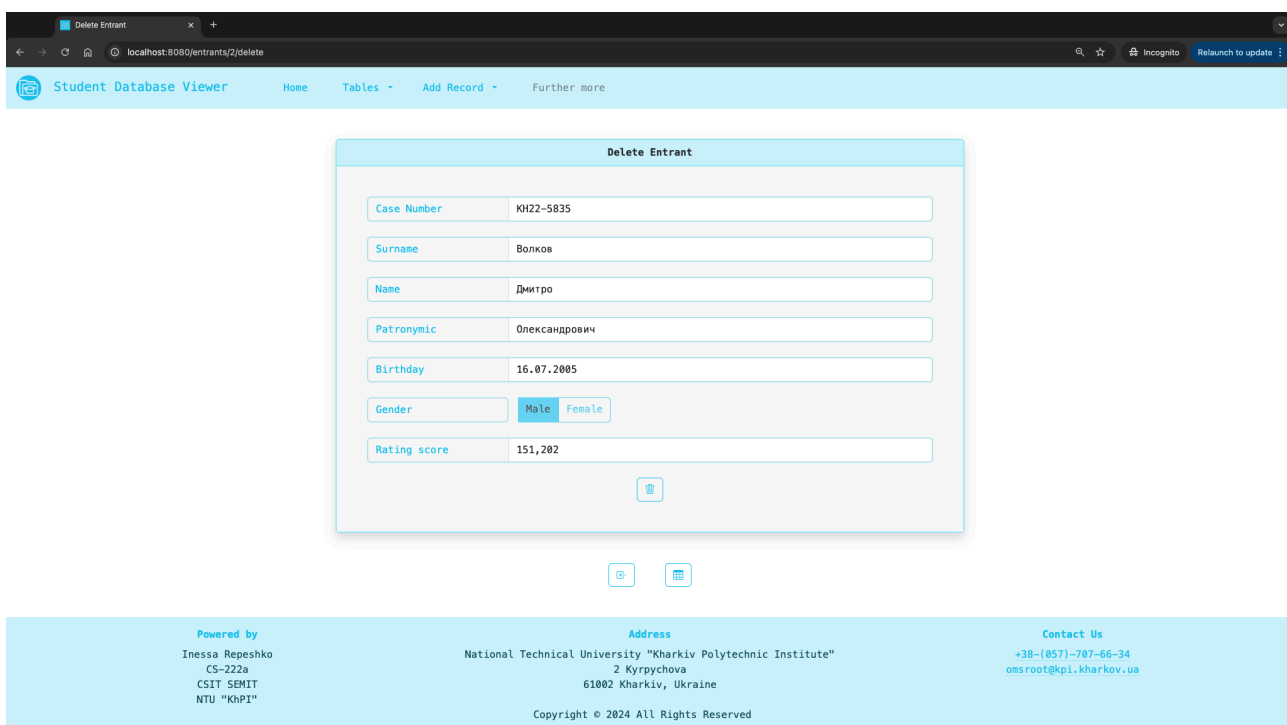


Рисунок Б.13.10 – Сторінка форми видалення абітурієнта на URL “http://localhost:8080/entrants/2/delete”

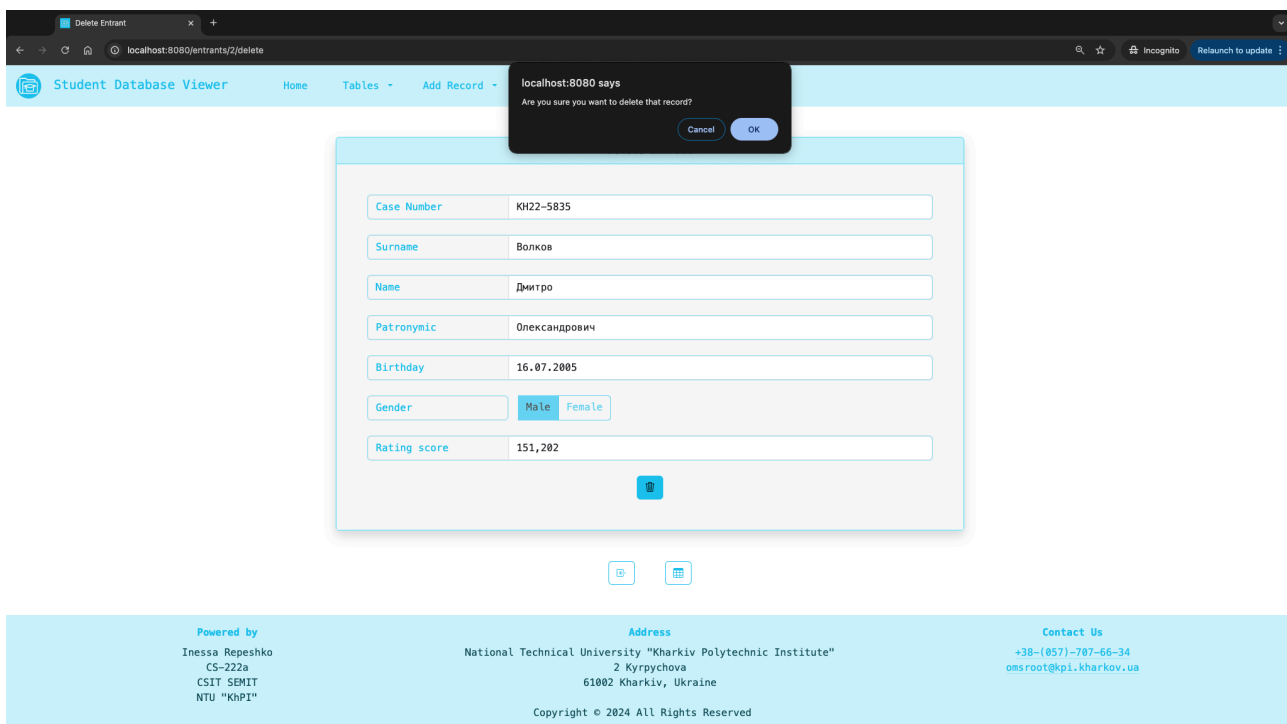


Рисунок Б.13.11 – Сторінка форми видалення абітурієнта з запитом підтвердження на URL “`http://localhost:8080/entrants/2/delete`”

Student Database Viewer

Successfully deleted

The record successfully deleted from the database.

Entrant Table

Search

Search by case number, full name (matching start, case-insensitive)

Type to search...

Q

X

Filters

Birthday

Start

dd.mm.yyyy

End

dd.mm.yyyy

▽

X

RatingScore

Start

End

▽

X

Add record

Add new record to the end of the table

+

#	Case Number	Full Name	Birthday	Gender	Rating Score	Operations
1	KH22-3090	Барвний Лук'ян Ярославович	01.01.2005	чоловік	170,000	
2	KH22-5511	Гесенко Анна Олександрівна	03.12.2005	жінка	150,013	
3	KH22-9299	Голоп'яров Микита Сергійович	07.11.2005	чоловік	161,484	
4	KH22-9812	Грищенко Максим Георгійович	04.12.2005	чоловік	191,945	
5	KH22-6636	Жарий Вадим Віталійович	25.10.2005	чоловік	128,346	
6	KH22-5140	Жупанов Андрій Володимирович	22.06.2005	чоловік	183,627	
7	KH22-4896	Кльоз Михайло Романович	15.01.2005	чоловік	127,288	
8	KH22-4694	Коваль Микита Артемович	11.10.2005	чоловік	139,739	
9	KH22-5529	Ковтун Єгор Вячеславович	10.04.2005	чоловік	187,790	
10	KH22-6874	Колесниченко Денис Євгенович	26.03.2005	чоловік	165,641	
11	KH22-2947	Ласкевич Андрій Сергійович	05.09.2005	чоловік	175,392	
12	KH22-3856	Лукаш Антон Олександрович	24.11.2005	чоловік	184,473	
13	KH22-7534	Палій Нікіта Андрійович	05.05.2005	чоловік	168,004	
14	KH22-3843	Прокопов Владислав Сергійович	12.12.2004	чоловік	164,665	
15	KH22-7819	Репешко Інесса Віталіївна	06.05.2005	жінка	185,306	
16	KH22-9115	Скиба Андрій Григорович	26.02.2004	чоловік	125,907	
17	KH22-3812	Цибань Ольга Фріівна	19.05.2004	жінка	190,290	
18	KH22-1828	Шевченко Данило Валерійович	25.10.2004	чоловік	166,849	
19	KH22-9514	Антипенко Катерина Віталіївна	06.01.2005	жінка	168,460	
20	KH22-2929	Багрянцева Марина Русланівна	01.10.2005	жінка	157,631	
21	KH22-1637	Бондаренко Ярослав Юрійович	24.04.2005	чоловік	199,532	
22	KH22-6681	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683	
23	KH22-7557	Васильєв Михайло Олександрович	25.01.2005	чоловік	170,864	
24	KH22-8759	Держак Дарина Євгенівна	03.03.2005	жінка	171,715	
25	KH22-7309	Ілюхін Тимур Ігорович	24.01.2005	чоловік	125,776	
26	KH22-8254	Криженко Аліна Степанівна	29.08.2005	жінка	138,747	
27	KH22-3825	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718	
28	KH22-8153	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309	
29	KH22-4788	Сизоненко Єлизавета Олександрівна	17.02.1997	жінка	124,240	
30	KH22-8255	Криженко Аліна Степанівна	29.08.2005	жінка	138,747	
31	KH22-3826	Посмашний Ярослав Дмитрович	25.02.2000	чоловік	160,718	
32	KH22-8154	Рожкова Вікторія Андріївна	04.10.1993	жінка	188,309	
33	KH22-6682	Булгаков Віктор Олександрович	17.05.2005	чоловік	153,683	
34	KH22-9998	Марчук Регіна Віталіївна	06.06.1996	жінка	200,000	
35	KH22-4242	Якімов Іван Романович	01.01.1990	чоловік	200,000	

Powered by

Inessa Repeshko

CS-222a

CSIT SEMIT

NTU "KHPI"

Address

National Technical University "Kharkiv Polytechnic Institute"

2 Kyrpychova

61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34

omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Б.13.12 – Сторінка таблиці абітурієнтів з повідомленням про видалення запису на URL “http://localhost:8080/entrants”

The screenshot shows a web browser window with the address bar displaying `localhost:8080/students/3/view`. The page title is "View Student". The main content area contains a form with the following fields and values:

- Entrant:** Гегенко Анна Олександрівна, KH22-5511
- Funding Type:** Budget (selected), Contract
- Scholarship Status:** None (selected), Ordinary, Enhanced
- Corporate Email:** anna.hieienko@cs.khpi.edu.ua

Below the form are two small icons: a square with a circle and a square with a plus sign.

The footer contains the following information:

- Powered by:** Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI"
- Address:** National Technical University "Kharkiv Polytechnic Institute", 2 Kyrychova, 61002 Kharkiv, Ukraine
- Contact Us:** +38-(057)-707-66-34, omsroot@kpi.kharkov.ua
- Copyright:** © 2024 All Rights Reserved

Рисунок Б.13.13 – Сторінка форми перегляду студента прив’язаного до обраного абітурієнта на URL “`http://localhost:8080/students/3/view`”

ДОДАТОК В

Реалізація шару бізнес-логіки та тестування методів CRUD для другої сутності

В.1 Код класу Srudent.java

```
package ntukhpi.csit.semit.riv.webappsrivlab3.model.student;

import jakarta.persistence.Table;
import jakarta.persistence.ForeignKey;
import jakarta.persistence.CascadeType;
import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.FundingTypeConve
rter;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.ScholarshipStatu
s;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.ScholarshipStatu
sConverter;
import org.hibernate.annotations.*;
import org.hibernate.validator.constraints.*;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
```

```

import java.util.Arrays;
import java.util.List;

/**
 * The Student class represents an entity of a student who has
 * transitioned from an entrant.
 * It includes details such as the funding type, scholarship
 * status, and corporate email.
 *
 * The class is mapped to the "students" table in the database and
 * uses various JPA and Hibernate annotations for
 * defining relationships, constraints, and filters.
 *
 * Key attributes:
 * - {@link #entrant}: The one-to-one relationship with the {@link
 * Entrant} entity, representing the entrant information.
 * - {@link #fundingType}: The funding type (contract or budget),
 * stored as a string in the database.
 * - {@link #scholarshipStatus}: The scholarship status (ordinary,
 * enhanced, or none), stored as a string in the database.
 * - {@link #corporateEmail}: The corporate email with a validation
 * pattern ensuring it follows a specific format.
 *
 * Relationships:
 * - One-to-one relationship with the {@link Entrant} entity.
 * - A foreign key constraint is defined for the entrant ID, which
 * sets the foreign key to null on deletion or update.
 *
 * @see Entrant
 * @see FundingType
 * @see ScholarshipStatus
 * @see FundingTypeConverter
 * @see ScholarshipStatusConverter
 */

```

```

* @author Inessa Repeshko CS-222a
*/

@Entity
@Table(name = "students")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Student {
    private static final String REGEX_EMAIL =
"^[a-z](\\.?[a-z]+){1,99}@[a-z]{1,10}\\.(k\\hpi\\.edu\\.ua$";

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")

    @Positive(message = "The id must be a positive number.")

    private Long id;

    @OneToOne(cascade = {CascadeType.REFRESH, CascadeType.REMOVE})
    @JoinColumn(name = "entrant_id", referencedColumnName = "id",
        foreignKey = @ForeignKey(
            name = "fk_student_entrant",
            foreignKeyDefinition = "FOREIGN KEY
(entrant_id) REFERENCES entrants(id) ON DELETE CASCADE ON UPDATE
CASCADE"))

    private Entrant entrant;

    @Column(name = "funding_type", nullable = false)
    @Enumerated(EnumType.STRING)

```

```

@Convert(converter = FundingTypeConverter.class)
@ColumnDefault(value = "'CONTRACT'")

@NotNull(message = "Funding type must not be null.")

private FundingType fundingType;

@Column(name = "scholarship_status", nullable = false)
@Enumerated(EnumType.STRING)
@Convert(converter = ScholarshipStatusConverter.class)
@ColumnDefault(value = "'NONE'")

@NotNull(message = "Scholarship status must not be null.")

private ScholarshipStatus scholarshipStatus;

@Column(name = "corporate_email", nullable = false, unique =
true, length = 123)
@Check(constraints = "REGEXP_LIKE(corporate_email, '" +
REGEX_EMAIL + "', 'c') = 1")
@Length(min = 17, max = 123, message = "The corporate email
length should be between 1 to 50 characters.")
@org.hibernate.validator.constraints.Email(regexp =
REGEX_EMAIL, message = "The corporate email should match the
format 'name.surname@faculty.khpi.edu.ua'.")

@NotBlank(message = "The email must not be blank.")
@Size(min = 17, max = 123, message = "The corporate email
length should be between 1 to 50 characters.")
@Pattern(regexp = REGEX_EMAIL, message = "The corporate email
should match the format 'name.surname@faculty.khpi.edu.ua'.")
>Email(regexp = REGEX_EMAIL, message = "The corporate email
should match the format 'name.surname@faculty.khpi.edu.ua'.")

```

```

    private String corporateEmail;

    public Student(Entrant entrant, String fundingType, String
scholarshipStatus, String corporateEmail) {
        setEntrant(entrant);
        setFundingType(FundingType.fromValue(fundingType));

setScholarshipStatus(ScholarshipStatus.fromValue(scholarshipStatus
));

        setCorporateEmail(corporateEmail);
    }

    public static String[][] getFieldNamesAsFormattedStrings() {
        List<String[]> fieldNames = Arrays.asList(
            new String[]{"fullName", "Full Name"},
            new String[]{"fundingType", "Funding Type"},
            new String[]{"scholarshipStatus", "Scholarship
Status"},
            new String[]{"corporateEmail", "Corporate Email"}
        );

        return fieldNames.toArray(String[][]::new);
    }
}

```

B.2 Код класу StudentList.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.model.student;

import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;

import java.util.ArrayList;
import java.util.Arrays;

```



```

import java.util.List;

/**
 * StudentList is a singleton class implemented using an Enum to
 * store a predefined list of {@link Student} objects.
 * It provides a method for initializing the list with sample data
 * and retrieving the list of students.
 * This approach guarantees thread-safety and is protected from
 * issues related to serialization.
 * <p>
 * Key features:
 * - Enum-based singleton for a single instance of StudentList.
 * - Predefined list of students that can be used for testing or
 * demo purposes.
 * - The list includes both valid and logically deleted entrants,
 * as well as examples with complex name formats.
 * <p>
 * This class is primarily used to simulate a database or
 * repository of students.
 *
 * @author Inessa Repeshko CS-222a
 * @see Student
 * @see Entrant
 * @see ArrayList
 */

public enum StudentList {
    INSTANCE;

    private final List<Student> students;

    StudentList() {
        this.students = new ArrayList<>();

        List<String[]> studentData = Arrays.asList(

```

```

        new String[]{"KH22-4814", "Бабічев", "Олексій",
"Вікторович", "2005-05-22", "TRUE", "168.111", "бюджет",
"звичайна", "oleksii.babichiev@cs.khpi.edu.ua"},

        new String[]{"KH22-5835", "Волков", "Дмитро",
"Олександрович", "2005-07-16", "TRUE", "151.202", "бюджет",
"відсутня", "dmytro.o.volkov@cs.khpi.edu.ua"},

        new String[]{"KH22-5511", "Геєнко", "Анна",
"Олексіївна", "2005-12-03", "FALSE", "150.013", "бюджет",
"відсутня", "anna.hieienko@cs.khpi.edu.ua"},

        new String[]{"KH22-9299", "Голоп'яров", "Микита",
"Сергійович", "2005-11-07", "TRUE", "161.484", "бюджет",
"звичайна", "mykyta.holoporov@cs.khpi.edu.ua"},

        new String[]{"KH22-9012", "Гриценко", "Максим",
"Георгійович", "2005-12-04", "TRUE", "191.945", "бюджет",
"підвищена", "maksym.hrytsenko@cs.khpi.edu.ua"},

        new String[]{"KH22-6636", "Жарий", "Вадим",
"Віталійович", "2005-10-25", "TRUE", "128.346", "контракт",
"відсутня", "vadym.zharyi@cs.khpi.edu.ua"},

        new String[]{"KH22-5140", "Жупанов", "Андрій",
"Володимирович", "2005-06-22", "TRUE", "183.627", "бюджет",
"звичайна", "andrii.zhupanov@cs.khpi.edu.ua"},

        new String[]{"KH22-4896", "Кльоз", "Михайло",
"Романович", "2005-01-15", "TRUE", "127.288", "контракт",
"відсутня", "mykhailo.kloz@cs.khpi.edu.ua"},

        new String[]{"KH22-4694", "Коваль", "Микита",
"Артемович", "2005-10-11", "TRUE", "139.739", "бюджет",
"відсутня", "mykyta.koval@cs.khpi.edu.ua"},

        new String[]{"KH22-5529", "Ковтун", "Єгор",
"Вячеславович", "2005-04-10", "TRUE", "187.790", "бюджет",
"підвищена", "yehor.kovtun@cs.khpi.edu.ua"},

        new String[]{"KH22-6874", "Колесниченко", "Денис",
"Євгенович", "2005-03-26", "TRUE", "165.641", "бюджет",
"звичайна", "denys.kolesnychenko@cs.khpi.edu.ua"},

        new String[]{"KH22-2947", "Ласкевич", "Андрій",
"Сергійович", "2005-09-05", "TRUE", "175.392", "бюджет",
"звичайна", "andrii.laskevych@cs.khpi.edu.ua"},

```

```

        new String[]{"KH22-3856", "Лукаш", "Антон",
"Олександрович", "2005-11-24", "TRUE", "184.473", "бюджет",
"звичайна", "anton.lukash@cs.khpi.edu.ua"},

        new String[]{"KH22-7534", "Палій", "Нікіта",
"Андрійович", "2005-05-05", "TRUE", "168.004", "бюджет",
"відсутня", "nikita.palii@cs.khpi.edu.ua"},

        new String[]{"KH22-3843", "Прокопов", "Владислав",
"Сергійович", "2004-12-12", "TRUE", "164.665", "бюджет",
"звичайна", "vladyslav.prokopov@cs.khpi.edu.ua"},

        new String[]{"KH22-7819", "Репешко", "Інесса",
"Віталіївна", "2005-05-06", "FALSE", "185.306", "бюджет",
"підвищена", "inessa.repeshko@cs.khpi.edu.ua"},

        new String[]{"KH22-9115", "Скиба", "Андрій",
"Григорович", "2004-02-26", "TRUE", "125.907", "контракт",
"відсутня", "andrii.h.skyba@cit.khpi.edu.ua"},

        new String[]{"KH22-3812", "Цибань", "Ольга",
"Юріївна", "2004-05-19", "FALSE", "190.298", "бюджет", "відсутня",
"olha.tsyban@cs.khpi.edu.ua"},

        new String[]{"KH22-1828", "Шевченко", "Данило",
"Валерійович", "2004-10-25", "TRUE", "166.849", "бюджет",
"звичайна", "danylo.v.shevchenko@cs.khpi.edu.ua"},

        new String[]{"KH22-9514", "Антипенко", "Катерина",
"Віталіївна", "2005-01-06", "FALSE", "168.460", "бюджет",
"звичайна", "kateryna.antypenko@cs.khpi.edu.ua"},

        new String[]{"KH22-2929", "Багрянцева", "Марина",
"Русланівна", "2005-10-01", "FALSE", "157.631", "бюджет",
"звичайна", "marina.bahriantseva@cs.khpi.edu.ua"},

        new String[]{"KH22-1637", "Бондаренко", "Ярослав",
"Юрійович", "2005-04-24", "TRUE", "199.532", "бюджет",
"підвищена", "yaroslav.y.bondarenko@cs.khpi.edu.ua"},

        new String[]{"KH22-6681", "Булгаков", "Віктор",
"Олександрович", "2005-05-17", "TRUE", "153.683", "бюджет",
"відсутня", "viktor.bulhakov@cs.khpi.edu.ua"},

        new String[]{"KH22-7557", "Васильєв", "Михайло",
"Олександрович", "2005-01-25", "TRUE", "170.864", "бюджет",
"відсутня", "mykhailo.vasyliiev@cs.khpi.edu.ua"},

```

```

        new String[]{"KH22-8759", "Деркач", "Дарина",
"Євгенівна", "2005-03-03", "FALSE", "171.715", "бюджет",
"звичайна", "daryna.derkach@cs.khpi.edu.ua"},
        new String[]{"KH22-7309", "Ілюхін", "Тимур",
"Ігорович", "2005-01-24", "TRUE", "125.776", "контракт",
"відсутня", "tymur.iliukhin@cs.khpi.edu.ua"},
        new String[]{"KH22-8254", "Криженко", "Аліна",
"Степанівна", "2005-08-29", "FALSE", "138.747", "бюджет",
"відсутня", "alina.kryzhenko@cs.khpi.edu.ua"},
        new String[]{"KH22-3825", "Посмашний", "Ярослав",
"Дмитрович", "2000-02-25", "TRUE", "160.718", "бюджет",
"звичайна", "yaroslav.posmashnyi@cs.khpi.edu.ua"},
        new String[]{"KH22-8153", "Рожкова", "Вікторія",
"Андріївна", "1993-10-04", "FALSE", "188.309", "бюджет",
"підвищена", "viktoriia.rozhkova@cs.khpi.edu.ua"},
        new String[]{"KH22-4788", "Сизоненко", "Єлизавета",
"Олександрівна", "1997-02-17", "FALSE", "124.240", "контракт",
"відсутня", "yelyzaveta.syzonenko@cs.khpi.edu.ua"}
    );

    studentData.forEach(
        data -> addStudent(new Student(
            new Entrant(data[0], data[1], data[2],
data[3], data[4], data[5], data[6]),
            data[7], data[8], data[9])));
    }

    public List<Student> getStudents() {
        return new ArrayList<>(students);
    }

    public void addStudent(Student student) {
        students.add(student);
    }

    public void removeStudent(Student student) {

```

```

        students.remove(student);
    }

    public void clearStudents() {
        students.clear();
    }
}

```

B.3 Код інтерфейсу StudentRepository.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.repository;

import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.ScholarshipStatu
s;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.Student;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface StudentRepository extends JpaRepository<Student,
Long> {

    @Query("SELECT s FROM Student s "
        + "JOIN s.entrant e "
        + "WHERE (:search IS NULL OR UPPER(CONCAT(e.surname, '
', e.name, ' ', e.patronymic)) LIKE UPPER(CONCAT(:search, '%')))"

```

```

        + "AND (:fundingType IS NULL OR s.fundingType =
:fundingType) "
        + "AND (:scholarshipStatuses IS NULL OR
s.scholarshipStatus IN :scholarshipStatuses) ")
    List<Student> findFilteredAndSortedStudents(@Param("search")
String search,

@Param("fundingType") FundingType fundingType,

@Param("scholarshipStatuses") List<ScholarshipStatus>
scholarshipStatuses,

Sort sortBy);
}

```

B.4 Код інтерфейсу StudentService.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.service;

import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.Student;

import java.util.List;

public interface StudentService {
    List<Student> getAllStudents();

    List<Student> getFilteredAndSortedStudents(String search,
                                                String fundingType,
                                                String
scholarshipStatus,
                                                String sort);

    void saveAllStudents(List<Student> students);

    Student findStudentById(Long id);
}

```

```

    Student findStudentByExample(Student student);

    Student saveStudent(Student student);

    Student updateStudent(Student student);

    void deleteStudent(Student student);
}

```

B.5 Код класу StudentServiceImpl.java

```

package
ntukhpi.csit.semit.riv.webappsrivlab3.service.implementation;

import jakarta.persistence.EntityNotFoundException;
import jakarta.validation.ConstraintViolationException;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.FundingType;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.enums.ScholarshipStatu
s;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab3.repository.StudentRepository
;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.StudentService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.dao.EmptyResultDataAccessException;

```

```

import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.data.domain.Example;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.ArrayList;
import java.util.List;

@Service
public class StudentServiceImpl implements StudentService {
    private static final Logger logger =
LoggerFactory.getLogger(StudentServiceImpl.class);

    private final StudentRepository studentRepository;
    private final EntrantService entrantService;

    public StudentServiceImpl(StudentRepository studentRepository,
EntrantService entrantService) {
        super();
        this.studentRepository = studentRepository;
        this.entrantService = entrantService;
    }

    @Override
    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    @Override
    @Transactional(rollbackFor =
{DataIntegrityViolationException.class,
ConstraintViolationException.class,
OptimisticLockingFailureException.class})
    public void saveAllStudents(List<Student> students) {
        try {

```



```

        students.forEach(student ->
student.setEntrant(entrantService.findEntrantByExample(student.get
Entrant())));

        studentRepository.saveAll(students);

    } catch (DataIntegrityViolationException e) {
        /* Handling a duplicate or uniqueness exception */
        logger.error("Duplicate or invalid data detected.\nData
integrity violation: " + e.getMessage());
        throw new IllegalArgumentException("Duplicate or
invalid data detected.");
    }

    } catch (ConstraintViolationException e) {
        /* Handling violations of validation constraints */
        logger.error("Validation failed for the
entity.\nConstraint violation: " + e.getMessage());
        throw new IllegalArgumentException("Validation failed
for the entity. Check the input data.");
    }

    } catch (OptimisticLockingFailureException e) {
        /* Handling version conflicts (if optimistic locking is
used) */
        logger.error("Optimistic locking failure: " +
e.getMessage());
        throw new RuntimeException("The entity could not be
saved due to a version conflict.");
    }

    } catch (Exception e) {
        /* General handling of other exceptions */
        logger.error("An error occurred while saving the
entity.\n" + e.getMessage());
        throw new RuntimeException("An error occurred while
saving the entity.");
    }
}

```

```

@Override
public Student findStudentById(Long id) {
    return studentRepository.findById(id)
        .orElseThrow(() -> {
            String errorMessage = "No record exists with
id: " + id + ".";
            logger.error(errorMessage);
            return new
EntityNotFoundException(errorMessage);
        });
}

@Override
public Student findStudentByExample(Student student) {
    Example<Student> example = Example.of(student);
    return studentRepository.findBy(example, query ->
query.first().orElse(null));
}

@Override
public List<Student> getFilteredAndSortedStudents(String
search,
                                                    String
fundingType,
                                                    String
scholarshipStatus,
                                                    String sort)
{
    String sortPattern =
"^(id|caseNumber|fullName|surname|name|patronymic|fundingType|scho
larshipStatus|corporateEmail)-(asc|desc)$";

    if (!sort.trim().matches(sortPattern)) {
        logger.warn("Invalid sort field provided: " + sort + ".
It should be in a format: 'fieldName-direction'.");
    }
}

```

```

        throw new IllegalArgumentException("Invalid sort field
provided: " + sort + ". It should be in a format:
'fieldName-direction'.");
    }

    String[] sortParams = sort.split("-");
    String sortField = sortParams[0];
    Sort.Direction sortDirection;

    try {
        sortDirection =
Sort.Direction.fromString(sortParams[1]);
    } catch (IllegalArgumentException e) {
        logger.error(e.getMessage());
        throw new IllegalArgumentException(e.getMessage());
    }

    List<Sort.Order> sortOrders = new ArrayList<>();

    switch (sortField) {
        case "caseNumber":
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.caseNumber"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.name"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.patronymic"));
            break;

        case "surname":
        case "fullName":
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.surname"));

```

```

        sortOrders.add(new Sort.Order(sortDirection,
"entrant.name"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.patronymic"));
        sortOrders.add(new Sort.Order(sortDirection,
"entrant.caseNumber"));
        break;

        default:
            sortOrders.add(new Sort.Order(sortDirection,
sortField));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.surname"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.name"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.patronymic"));
            sortOrders.add(new Sort.Order(sortDirection,
"entrant.caseNumber"));
            break;
    }

    Sort sortBy = Sort.by(sortOrders);

    String[] statuses = scholarshipStatus != null ?
scholarshipStatus.trim().split("-") : new String[]{};
    List<ScholarshipStatus> scholarshipStatuses = new
ArrayList<>();

    for (String status : statuses) {
scholarshipStatuses.add(ScholarshipStatus.fromValue(status));
    }

    try {
        return studentRepository.findFilteredAndSortedStudents(

```

```

        search,
        fundingType != null ?
FundingType.fromValue(fundingType) : null,
        !scholarshipStatuses.isEmpty() ?
scholarshipStatuses : null,
        sortBy);
    } catch (Exception e) {
        logger.error("An error occurred while getting the
filtered and sorted students: " + e.getMessage());
        throw new RuntimeException("An error occurred while
getting the filtered and sorted students: " + e.getMessage());
    }
}

@Override
@Transactional(rollbackFor =
{DataIntegrityViolationException.class,
ConstraintViolationException.class,
OptimisticLockingFailureException.class})
public Student saveStudent(Student student) {
    try {
        return studentRepository.save(student);
    } catch (DataIntegrityViolationException e) {
        /* Handling a duplicate or uniqueness exception,
foreign key, NOT NULL, CHECK, or length violation */
        logger.error("Duplicate or invalid data detected.\nData
integrity violation: " + e.getMessage());
        throw new IllegalArgumentException(
            "The record with the corporate email '" +
student.getCorporateEmail() + "' already exists.");
    } catch (ConstraintViolationException e) {
        /* Handling violations of validation constraints */
        logger.error("Validation failed for the
entity.\nConstraint violation: " + e.getMessage());
    }
}

```

```

        throw new IllegalArgumentException("Validation failed
for the entity. Check the input data.");

    } catch (OptimisticLockingFailureException e) {
        /* Handling version conflicts (if optimistic locking is
used) */
        logger.error("Optimistic locking failure: " +
e.getMessage());
        throw new RuntimeException("The entity could not be
saved due to a version conflict.");

    } catch (Exception e) {
        /* General handling of other exceptions */
        logger.error("An error occurred while saving the
entity.\n" + e.getMessage());
        throw new RuntimeException("An error occurred while
saving the entity." + e.getMessage());
    }
}

@Override
@Transactional(rollbackFor =
{DataIntegrityViolationException.class,
ConstraintViolationException.class,
OptimisticLockingFailureException.class})
public Student updateStudent(Student student) {
    if (student.getId() == null) {
        logger.error("Cannot update a record without an id.");
        throw new IllegalArgumentException("Cannot update a
record without an id.");
    }

    if (!studentRepository.existsById(student.getId())) {
        logger.error("No record exists to update.");
        throw new IllegalArgumentException("No record exists to
update.");
    }
}

```

```

    }

    return saveStudent(student);
}

@Override
@Transactional(rollbackFor =
{DataIntegrityViolationException.class,
EmptyResultDataAccessException.class,
OptimisticLockingFailureException.class})
public void deleteStudent(Student student) {
    try {
        studentRepository.delete(student);

    } catch (DataIntegrityViolationException e) {
        /* Handling data integrity violations, for example,
when foreign keys are violated */
        logger.error("Data integrity violation occurred: " +
e.getMessage());
        throw new IllegalArgumentException("Unable to delete
the entity due to data integrity constraints.");
    } catch (EmptyResultDataAccessException e) {
        /* Handling the case when an attempt to delete a
non-existent record is made */
        logger.error("No entity found for deletion: " +
e.getMessage());
        throw new IllegalArgumentException("The entity does not
exist and cannot be deleted.");
    } catch (OptimisticLockingFailureException e) {
        /* Handling an error when trying to delete a record
with a version conflict */
        logger.error("Optimistic locking failure: " +
e.getMessage());
    }
}

```

```

        throw new IllegalArgumentException("The entity could
not be deleted due to a version conflict.");

    } catch (Exception e) {
        /* General handling of other exceptions */
        logger.error("An unexpected error occurred while
deleting the entity: " + e.getMessage());
        throw new RuntimeException("An unexpected error
occurred while deleting the entity.");
    }
}
}
}

```

B.6 Код тесту StudentServiceTest.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3;

import static org.junit.jupiter.api.Assertions.*;
import static org.junit.jupiter.api.Assertions.assertThrows;

import jakarta.persistence.EntityNotFoundException;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.StudentService;

```



```

@SpringBootTest
class StudentServiceTest {

    @Autowired
    private StudentService studentService;

    @Autowired
    private EntrantService entrantService;

    private Student testStudent;

    private Entrant testEntrant;

    @BeforeEach
    void setUp() {
        testEntrant = new Entrant("KH22-9876", "Франко", "Іван",
            "Якович", "2002-01-15", "TRUE", "150.0");
        testStudent = new Student(testEntrant, "бюджет",
            "звичайна", "franko.ivan@cs.khpi.edu.ua");
        entrantService.saveEntrant(testEntrant);
    }

    @AfterEach
    void reset() {
        studentService.deleteStudent(testStudent);
    }

    @Test
    void testCreateStudent() {
        Student savedStudent =
            studentService.saveStudent(testStudent);
        assertNotNull(savedStudent.getId(), "Студент повинен мати
            ID після збереження.");
    }
}

```

```

@Test
void testReadStudent() {
    Student savedStudent =
studentService.saveStudent(testStudent);
    Student foundStudent =
studentService.findStudentById(savedStudent.getId());
    assertNotNull(foundStudent, "Студента має бути знайдено за
ID.");
    assertEquals(savedStudent.getCorporateEmail(),
foundStudent.getCorporateEmail(), "Корпоративна пошта повинна
збігатися.");
}

@Test
void testUpdateStudent() {
    String newEmail = "shevchenko.taras@cs.khpi.edu.ua";
    Student savedStudent =
studentService.saveStudent(testStudent);
    savedStudent.setCorporateEmail(newEmail);
    Student updatedStudent =
studentService.saveStudent(savedStudent);

    Student foundStudent =
studentService.findStudentById(savedStudent.getId());
    assertNotNull(foundStudent, "Оновленого студента має бути
знайдено за ID.");
    assertEquals(newEmail, foundStudent.getCorporateEmail(),
"Корпоративна пошта має бути оновлена.");
}

@Test
void testDeleteStudent() {
    Student savedStudent =
studentService.saveStudent(testStudent);
    Long id = savedStudent.getId();
    studentService.deleteStudent(savedStudent);
}

```

```
        assertThrows(EntityNotFoundException.class, () -> {
            studentService.findStudentById(id);
        }, "Студента не повинно бути знайдено після видалення.");
    }

    @Test
    void testSaveStudentWithDuplicateCorporateEmail() {
        studentService.saveStudent(testStudent);
        Student duplicateStudent = new Student(testEntrant,
        "бюджет", "звичайна", "franko.ivan@cs.khpi.edu.ua");

        assertThrows(IllegalArgumentException.class, () -> {
            studentService.saveStudent(duplicateStudent);
        }, "Дубльована корпоративна пошта має викликати
        IllegalArgumentException.");
    }
}
```

ДОДАТОК Г

Реалізація шару вебінтерфейсу для другої сутності

Г.1 Код сторінки StudentTable.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Student Table</title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">
</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>

```

```

<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="mx-3 mt-3 mb-5 font-monospace">
  <div class="row">
    <h1 class="text-info fs-1 my-3 fw-semibold text-center">Student Table</h1>
  </div>

  <div class="row align-items-start">
    <div class="col-2 mx-2">
      <div class="card row p-2 mb-4 border border-3 border-info-subtle bg-light shadow">
        <div class="card-body col">
          <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0 m-0">Search</p>
            <p class="text-secondary fw-lighter p-0 m-0" style="font-size: 0.8rem;">
              Search by full name (matching start, case-insensitive)
            </p>
          </div>
        </div>

        <form class="card-text col mt-3" id="formSearch"
          th:method="'get'"
          th:action="@{/students}">
          <div class="row mb-3">
            <label for="inputSearch" class="visually-hidden">Search</label>
            <input type="search" class="form-control border border-2 border-info-subtle"
              id="inputSearch" name="search" placeholder="Type to search...">
          </div>
        </form>
      </div>
    </div>
  </div>
</div>

```

```

        <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
            <a type="button" id="btn-search"
onclick="applySearch(this) "
                class="btn btn-outline-info
border-2" th:alt-title="'Apply Search'">
                <i class="bi bi-search"></i>
            </a>

            <a type="button" id="btn-search-reset"
onclick="resetSearch(this) "
                class="btn btn-outline-info border-2
button-reset" th:title="'Reset Search'">
                <i class="bi bi-x-lg"></i>
            </a>
        </div>
    </form>
</div>
</div>

    <div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
        <div class="card-body col">
            <div class="card-title row">
                <p class="text-info fs-5 fw-bold p-0
m-0">Filters</p>
            </div>

            <div class="card-subtitle row mt-2"
id="filterFundingType">
                <p class="text-info fs-6 fw-bold p-0
m-0">Funding Type</p>
            </div>

            <form class="card-text col mt-2"
id="formFilterFundingType"

```

```

th:method="'get'"
th:action="@{/students}">
    <div class="mt-2 mb-3">
        <div class="form-check">
            <input class="form-check-input"
type="radio" id="fundingTypeBudget"
name="fundingType"
th:value="'бюджет'">
                <label class="form-check-label"
for="fundingTypeBudget" style="color: #6f42c1;">
                    Budget
                </label>
            </div>

            <div class="form-check">
                <input class="form-check-input"
type="radio" id="fundingTypeContract"
name="fundingType"
th:value="'контракт'">
                    <label class="form-check-label"
for="fundingTypeContract" style="color: #fd7e14;">
                        Contract
                    </label>
                </div>
            </div>

            <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                <a type="button"
id="btn-filter-funding-type"
onclick="applyFilterFundingType(this)"
class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
                    <i class="bi bi-funnel"></i>
                </a>

```

```

        <a type="button"
id="btn-filter-funding-type-reset"
onclick="resetFilterFundingType(this)"
        class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
            <i class="bi bi-x-lg"></i>
        </a>
    </div>
</form>

    <div class="card-subtitle row mt-4"
id="filterScholarshipStatus">
        <p class="text-info fs-6 fw-bold p-0
m-0">Scholarship Status</p>
    </div>

    <form class="card-text col mt-2"
id="formFilterScholarshipStatus"
        th:method="'get'"
th:action="@{/students}">
        <div class="mt-2 mb-3">
            <div class="form-check">
                <input class="form-check-input"
type="checkbox" id="scholarshipStatusEnhanced"
                    name="scholarshipStatus"
th:value="'підвищена'">
                <label class="form-check-label
fw-lighter py-0 px-1 m-0" for="scholarshipStatusEnhanced"
                    style="color: #20c997;">
                    Enhanced
                </label>
            </div>
            <div class="form-check">
                <input class="form-check-input"
type="checkbox" id="scholarshipStatusOrdinary"

```



```

name="scholarshipStatus"
th:value="'звичайна'"
        <label class="form-check-label
fw-lighter py-0 px-1 m-0" for="scholarshipStatusOrdinary"
        style="color: #0d6efd;">
            Ordinary
        </label>
    </div>
    <div class="form-check">
        <input class="form-check-input"
type="checkbox" id="scholarshipStatusNone"
        name="scholarshipStatus"
th:value="'відсутня'"
        <label class="form-check-label
fw-lighter py-0 px-1 m-0" for="scholarshipStatusNone"
        style="color: #6c757d;">
            None
        </label>
    </div>
</div>

<div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
    <a type="button"
id="btn-filter-scholarship-status"
onclick="applyFilterScholarshipStatus(this)"
        class="btn btn-outline-info
border-2" th:title="'Apply Filter'">
        <i class="bi bi-funnel"></i>
    </a>

    <a type="button"
id="btn-filter-scholarship-status-reset"
onclick="resetFilterScholarshipStatus(this)"

```

```

class="btn btn-outline-info border-2
button-reset" th:title="'Reset Filter'">
    <i class="bi bi-x-lg"></i>
</a>
</div>
</form>
</div>
</div>

<div class="card row p-2 mb-4 border border-3
border-info-subtle bg-light shadow">
    <div class="card-body col">
        <div class="card-title row">
            <p class="text-info fs-5 fw-bold p-0
m-0">Add record</p>
            <p class="text-secondary fw-lighter p-0
m-0" style="font-size: 0.8rem;">
                Add new record to the end of the table
            </p>
        </div>

        <form class="card-text col mt-3"
id="formInsert"
            th:method="'get'"
th:action="@{/students}">
            <div class="col d-flex
justify-content-evenly align-content-center text-center p-0 m-0">
                <a th:href="@{/students/add}"
id="btn-add-student"
                    class="btn btn-outline-info
border-2" role="button" th:title="'Add record'">
                    <i class="bi bi-plus-lg"></i>
                </a>
            </div>
        </form>
    </div>

```

```

        </div>
    </div>

    <div class="col">
        <table id="studentTable" class="table table-sm
table-hover font-monospace
        table-bordered border border-3
border-info-subtle shadow-lg">
            <thead class="table-info text-center fw-medium
text-secondary-emphasis">
                <tr>
                    <th class="text-center align-content-center
text-info">#</th>

                    <div class="p-0 m-0" th:each="fieldName :
${fields}">
                        <th class="text-center
align-content-center">
                            <div class="d-flex p-0 m-0">
                                <div class="flex-grow-1
align-content-center justify-content-center p-0 m-0">
                                    <span class="">
                                        <a class="sortLink
link-info link-offset-2 link-offset-3-hover link-underline
link-underline-opacity-0 link-underline-opacity-75-hover"
th:id="${fieldName[0]}"
th:text="${fieldName[1]}"
onclick="applySort(this)"></a>
                                    </span>
                                </div>
                                <div class="d-none flex-shrink-1
align-content-center justify-content-end ms-2 p-0 m-0"
style="width: 20px;">

```

```

<button type="button"
class="btn bg-transparent p-0 m-0" title="Sort records">
    <i class="bi
bi-sort-down-alt sortingAsc" th:title="'Ascending Sort'"></i>
    <i class="bi bi-sort-down
sortingDesc" th:title="'Descending Sort'"></i>
</button>
</div>
</div>
</th>
</div>

<th class="text-center align-content-center
text-info">Operations</th>
</tr>
</thead>

<tbody class="table-light fw-light
text-primary-emphasis" th:each="student, iterStat : ${students}">
<tr>
<td class="text-center align-content-center"
th:text="${iterStat.index + 1}"></td>
<td class="text-start align-content-center"
th:text="${student.entrant.surname} + ' ' +
${student.entrant.name} + ' ' +
${student.entrant.patronymic}"></td>
<td class="text-center align-content-center"
th:switch="${student.fundingType.getValue()}">
    <span th:case="'контракт'" style="color:
#fd7e14;" th:text="${student.fundingType.getValue()}">test1</span>
    <span th:case="'бюджет'" style="color:
#6f42c1;" th:text="${student.fundingType.getValue()}">test2</span>
</td>
<td class="text-center align-content-center"
th:switch="${student.scholarshipStatus.getValue()}">

```

```

        <span th:case="'підвищена'" style="color:
#20c997;"

th:text="${student.scholarshipStatus.getValue()}">test1</span>
        <span th:case="'звичайна'" style="color:
#0d6efd;"

th:text="${student.scholarshipStatus.getValue()}">test2</span>
        <span th:case="'відсутня'" style="color:
#6c757d;"

th:text="${student.scholarshipStatus.getValue()}">test3</span>
    </td>
    <td class="text-start align-content-center"
th:text="${student.corporateEmail}"></td>
    <td class="text-center align-content-center">
        <div class="d-flex justify-content-evenly
p-0 m-0">
            <a type="button"
th:href="@{/entrants/__$${student.entrant.id}__}/view}"
                class="p-0 m-0 border border-0
bg-transparent button-linked-entrant"
                th:title="'Entrant info'">
                    <i class="bi bi-person-rolodex
text-info"></i>
                </a>
            <a type="button"
th:href="@{/students/__$${student.id}__}/edit}"
                class="p-0 m-0 border border-0
bg-transparent button-update" th:title="'Edit record'">
                    <i class="bi bi-pencil
text-warning"></i>
                </a>
            <a type="button"
th:href="@{/students/__$${student.id}__}/delete}"

```

```

class="p-0 m-0 border border-0
bg-transparent button-delete" th:title="'Delete record'">
    <i class="bi bi-trash3
text-danger"></i>
    </a>
</div>
</td>
</tr>
</tbody>
</table>
<br>
</div>
</div>

<div class="d-flex justify-content-center">
    <a th:href="@{/home}" class="btn btn-outline-info"
type="button">
        <i class="bi bi-house"></i>
    </a>
</div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setSearchPatternAttr();
        setSearchInputValidity();

        setInputValueFromGetParams();
    });

    function setSearchPatternAttr() {
        const regexSearch = new
RegExp("^[A-ЩЬЮЯЄІЇҐа-щьюяєіїґ'\\"-\\d ]{1,200}$");

```

```

document.getElementById('inputSearch').setAttribute('pattern',
regexSearch.source);
}

function setSearchInputValidity() {
    const message = "Value is a Ukrainian a string between 1
and 200 chars long, optionally including numbers, dash,
apostrophe, space.";

document.getElementById('inputSearch').addEventListener('input',
function () {
    if (this.validity.patternMismatch) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity('');
    }

    this.reportValidity();
});
}

function setInputValueFromGetParams() {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);
    const inputFields = document.getElementsByTagName('input');

    for (const input of inputFields) {
        if (params.has(input.name)) {
            switch (input.type) {
                case 'radio':
                    if (input.value ===
params.get(input.name).trim()) {
                        input.setAttribute('checked', 'true');

```

```

        }
        break;
        case 'checkbox':
            const paramValues =
params.get(input.name).trim().split('-');
            paramValues.forEach(value => {
                if (input.value === value) {
                    input.setAttribute('checked',
'true');
                }
            });
            break;
        default:
            input.value = params.get(input.name);
            break;
    }
}
}
}

function applySort(element) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    const paramName = 'sort';
    const paramFieldName = element.id;
    let paramSortOrder = 'asc';
    const currentSortValue = params.get(paramName);

    if (currentSortValue) {
        const currentSortFieldName =
currentSortValue.split('-')[0];
        const currentSortOrder = currentSortFieldName ===
paramFieldName ? currentSortValue.split('-')[1] : '';
    }
}

```



```

        paramSortOrder = currentSortOrder === 'asc' ? 'desc' :
'asc';
    }

    params.set(paramName, paramFieldName + '-' +
paramSortOrder);

    const newURL = currentURL.pathname + '?' +
params.toString();
    element.setAttribute('href', newURL);
}

function applySearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    setFilteringParameter(params,
document.getElementById('inputSearch'));

    const newURL = currentURL.pathname + '?' +
params.toString();
    linkButton.setAttribute('href', newURL);
}

function resetSearch(linkButton) {
    const currentURL = new URL(window.location.href);
    const params = new
URLSearchParams(currentURL.searchParams);

    resetFilteringParameter(params,
document.getElementById('inputSearch'));

    const newURL = currentURL.pathname + '?' +
params.toString();
    document.getElementById('formSearch').reset();
}

```

```

        linkButton.setAttribute('href', newURL);
    }

    function applyFilterFundingType(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        const inputContract =
document.getElementById('fundingTypeContract');
        const inputBudget =
document.getElementById('fundingTypeBudget');

        console.log("inputContract.checked: ",
inputContract.checked);
        console.log("inputBudget.checked: ", inputBudget.checked);

        if (inputContract.checked) setFilteringParameter(params,
inputContract);
        if (inputBudget.checked) setFilteringParameter(params,
inputBudget);

        const newURL = currentURL.pathname + '?' +
params.toString();
        console.log(newURL);
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterFundingType(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById('fundingTypeContract'));
    }

```

```

        resetFilteringParameter(params,
document.getElementById('fundingTypeBudget'));

        const newURL = currentURL.pathname + '?' +
params.toString();

        document.getElementById('formFilterFundingType').reset();
        linkButton.setAttribute('href', newURL);
    }

    function applyFilterScholarshipStatus(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        function setFilteringParameterForScholarship(params,
inputElement) {
            const paramName = inputElement.name;
            const paramValue =
inputElement.value.toString().trim();

            if (paramValue.length > 0) {
                if (params.has(paramName)) {
                    const newParamValue = params.get(paramName) +
"-" + paramValue;
                    params.set(paramName, newParamValue);
                } else {
                    params.set(paramName, paramValue);
                }
            } else {
                params.delete(paramName);
            }
        }

        const inputEnhanced =
document.getElementById('scholarshipStatusEnhanced');

```

```

        const inputOrdinary =
document.getElementById('scholarshipStatusOrdinary');
        const inputNone =
document.getElementById('scholarshipStatusNone');

        params.delete(inputEnhanced.name);

        if (inputEnhanced.checked)
setFilteringParameterForScholarship(params, inputEnhanced);
        if (inputOrdinary.checked)
setFilteringParameterForScholarship(params, inputOrdinary);
        if (inputNone.checked)
setFilteringParameterForScholarship(params, inputNone);

        const newURL = currentURL.pathname + '?' +
params.toString();
        linkButton.setAttribute('href', newURL);
    }

    function resetFilterScholarshipStatus(linkButton) {
        const currentURL = new URL(window.location.href);
        const params = new
URLSearchParams(currentURL.searchParams);

        resetFilteringParameter(params,
document.getElementById('scholarshipStatusEnhanced'));
        resetFilteringParameter(params,
document.getElementById('scholarshipStatusOrdinary'));
        resetFilteringParameter(params,
document.getElementById('scholarshipStatusNone'));

        const newURL = currentURL.pathname + '?' +
params.toString();

document.getElementById('formFilterScholarshipStatus').reset();
        linkButton.setAttribute('href', newURL);
    }

```

```

    }

    function setFilteringParameter(params, inputElement) {
        const paramName = inputElement.name;
        const paramValue = inputElement.value.toString().trim();

        if (paramValue.length > 0) {
            params.set(paramName, paramValue);
        } else {
            params.delete(paramName);
        }
    }

    function resetFilteringParameter(params, inputElement) {
        const paramName = inputElement.name;
        params.delete(paramName);

        if (inputElement.type === 'radio' || inputElement.type ===
'checkbox') {
            inputElement.removeAttribute('checked');
        }
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>

</body>
</html>

```

Г.2 Код сторінки StudentForm.html

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title th:if="${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).ADD}"
        th:text="'Add Student'"></title>
    <title th:if="${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}"
        th:text="'View Student'"></title>
    <title th:if="${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT}"
        th:text="'Edit Student'"></title>
    <title th:if="${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}"
        th:text="'Delete Student'"></title>
    <link rel="icon" type="image/x-icon"
th:href="@{/image/icon/favicon_square.png}">
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
    <link rel="stylesheet"
href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.11.3/font/boo
tstrap-icons.min.css">

```

```

</head>
<body>

<header th:replace="~{component/NavigationBar ::
header}"></header>

<div th:replace="~{component/ErrorToast :: errorToast}"></div>
<div th:replace="~{component/SuccessToast :: successToast}"></div>

<div class="container w-75 text-center justify-content-center">
    <div class="card border border-3 border-info-subtle shadow
bg-body-tertiary rounded m-0 mt-5 p-0">
        <div class="card-header bg-info-subtle border-bottom
border-3 border-info-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder" id="formHeader"
            th:switch="{mode}">
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).ADD}"
                th:text="'Add Student'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).VIEW_FORM}"
                th:text="'View Student'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).EDIT}"
                th:text="'Edit Student'"></span>
            <span
th:case="{T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode
).DELETE}"
                th:text="'Delete Student'"></span>
            <span th:case="*"
                th:text="'Student Form'"></span>
        </div>
    </div>
</div>

```

```

        <form class="card-body bg-light font-monospace p-5 m-0"
id="studentForm"
        th:object="{student}"
        th:with="actionUrl=
                {mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).ADD ?
'/students/add' :
                (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
? '/students/view' :
                (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT ?
('/students/' + student.id + '/edit') :
                (mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE ?
('/students/' + student.id + '/delete') : '')}}}"
        th:action="@{{{actionUrl}}}"
        th:method="'post'">

        <div class="d-none card-text mb-4">
                <div class="input-group border border-1
border-info-subtle rounded ">
                        <label for="inputStudentId" class="col-4
input-group-text text-info">Id</label>
                        <input id="inputStudentId" class="col-auto
form-control text-dark" type="hidden" required
                                th:field="*{id}"
                                th:minlength="1" th:maxlength="100"
                                th:value="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                ? student.id : null}"
                                th:readonly="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE

```



```

                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        </div>
    </div>

    <div class="card-text d-flex align-content-center
justify-content-start mb-4">
        <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
            <span class="d-flex flex-fill input-group-text
text-info">Entrant</span>
        </div>

        <div class="col">
            <select class="form-select border border-1
border-info-subtle" aria-label="Entrant list select"
                th:field="*{entrant.id}" required
                th:disabled="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

                <option th:selected="true"
                    th:value="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).ADD
                                ? student.entrant.id :
null}"
                    th:text="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).ADD
                                ? student.entrant.surname + '
' + student.entrant.name + ' ' + student.entrant.patronymic + ', '
+ student.entrant.caseNumber
                                : 'Select an Entrant'}">

                    Select an Entrant
                </option>

```

```

        <div th:each="entrant : ${entrantList}">
            <option th:value="${entrant.id}"
                    th:text="${entrant.surname + ' ' + entrant.name + ' ' + entrant.patronymic + ', ' + entrant.caseNumber}">
                Entrant
            </option>
        </div>
    </select>
</div>
</div>

<div class="card-text d-flex align-content-center justify-content-start mb-4">
    <div class="col-3 border border-1 border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
        <span class="d-flex flex-fill input-group-text text-info">Funding Type</span>
    </div>

    <div class="col-auto">
        <input type="hidden"
              th:field="*{fundingType}"
              th:value="${student.fundingType.getValue()}"
              th:if="${mode eq T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}">

        <input id="fundingTypeBudget" type="radio"
              class="btn-check" autocomplete="off" required
              th:field="*{fundingType}"
              th:value="${'BUDGET'}"
              th:disabled="${mode eq T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}

```

```

                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}"

                                onclick="setScholarshipStatus(this)">
                                <label for="fundingTypeBudget" class="btn
btn-outline-info">Budget</label>

                                <input id="fundingTypeContract" type="radio"
class="btn-check" autocomplete="off" required
                                th:field="*{fundingType}"
                                th:value="${ 'CONTRACT' }"
                                th:disabled="${mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}"

                                onclick="setScholarshipStatus(this)">
                                <label for="fundingTypeContract" class="btn
btn-outline-info">Contract</label>
                                </div>
                                </div>

                                <div class="card-text d-flex align-content-center
justify-content-start mb-4">
                                <div class="col-3 border border-1
border-info-subtle rounded d-inline-flex p-0 m-0 me-3">
                                <span class="d-flex flex-fill input-group-text
text-info">Scholarship Status</span>
                                </div>

                                <div class="col-auto">
                                <input type="hidden"
                                th:field="*{scholarshipStatus}"
                                th:value="${student.scholarshipStatus.getValue()}"

```

```

th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}">

    <input id="scholarshipStatusNone" type="radio"
class="btn-check" autocomplete="off" required
th:field="{scholarshipStatus}"
th:value="{ 'NONE' }"
th:disabled="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

    <label for="scholarshipStatusNone" class="btn
btn-outline-info">None</label>

    <input id="scholarshipStatusOrdinary"
type="radio" class="btn-check" autocomplete="off" required
th:field="{scholarshipStatus}"
th:value="{ 'ORDINARY' }"
th:disabled="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

    <label for="scholarshipStatusOrdinary"
class="btn btn-outline-info">Ordinary</label>

    <input id="scholarshipStatusEnhanced"
type="radio" class="btn-check" autocomplete="off" required
th:field="{scholarshipStatus}"
th:value="{ 'ENHANCED' }"
th:disabled="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

```

```

        <label for="scholarshipStatusEnhanced"
class="btn btn-outline-info">Enhanced</label>
    </div>
</div>

<div class="card-text mb-4">
    <div class="input-group border border-1
border-info-subtle rounded ">
        <label for="inputCorporateEmail" class="col-3
input-group-text text-info">
            Corporate Email
        </label>
        <input id="inputCorporateEmail" class="col-auto
form-control text-dark" type="text" required
            th:field="*{corporateEmail}"
            th:minlength="17" th:maxlength="123"
            th:value="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).EDIT
or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
? student.corporateEmail :
null}"
            th:readonly="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
or mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">
    </div>
</div>

<div class="d-flex justify-content-evenly">
    <button type="submit" class="btn btn-outline-info
border-2 button-apply"
        th:title="'Save record'"
        th:if="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE

```

```

                                and mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        <i class="bi bi-floppy"></i>
    </button>

    <button type="reset" class="btn btn-outline-info
border-2 button-reset"
        onclick="form.reset()"
        th:title="'Reset fields'"
        th:if="{mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE
                                and mode ne
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).VIEW_FORM
}">

        <i class="bi bi-x-lg"></i>
    </button>

    <button type="button" class="btn btn-outline-info
border-2 button-delete"
        onclick="submitConfirmation()"
        th:title="'Delete record'"
        th:if="{mode eq
T(ntukhpi.csit.semit.riv.webappsrivlab3.controller.Mode).DELETE}">
        <i class="bi bi-trash3"></i>
    </button>
</div>
</form>
</div>

<div class="d-flex justify-content-center mt-5">
    <button class="btn btn-outline-info" type="button"
        th:title="'Go Back'"
        onclick="window.history.back()">
        <i class="bi bi-box-arrow-in-left"></i>
    </button>

```

```

        <a class="btn btn-outline-info ms-5" type="button"
            th:title="'Go to Student Table'"
            th:href="@{/students}">
            <i class="bi bi-table"></i>
        </a>
    </div>
</div>

<footer th:replace="~{component/Footer :: footer}"></footer>

<script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function () {
        setAllInputsValidity();

        const title =
document.getElementById('formHeader').innerText;

        if (title.match(new RegExp("Add Student|Edit Student"))) {
setScholarshipStatus(document.getElementById('fundingTypeBudget'))
;

setScholarshipStatus(document.getElementById('fundingTypeContract'
));
        }
    });

    function setAllInputsValidity() {
        setEmailInputValidity();

        const form = document.getElementById('studentForm');

        form.addEventListener('submit', function (event) {
            event.preventDefault();

```

```

        if (!form.checkValidity()) {
            event.stopPropagation();
            form.reportValidity();
        } else {
            form.submit();
        }
    });

    function setEmailInputValidity() {
        const regexCorporateEmail = new
RegExp("^([a-z](\\.|?([a-z]+){1,99})@[a-z]{1,10}\\.khpi\\.edu\\.ua$");

document.getElementById('inputCorporateEmail').setAttribute('patte
rn', regexCorporateEmail.source);

        const message = "The corporate email should match the
format 'name.surname@faculty.khpi.edu.ua'.";

document.getElementById('inputCorporateEmail').addEventListener('i
nput', function () {
    if (this.validity.patternMismatch) {
        this.setCustomValidity(message);
    } else {
        this.setCustomValidity('');
    }

    this.reportValidity();
});
}

function setScholarshipStatus(inputElement) {
    const inputId = inputElement.id;
    const enhancedId = 'scholarshipStatusEnhanced';
    const ordinaryId = 'scholarshipStatusOrdinary';

```



```

        const inputEnhanced = document.getElementById(enhancedId);
        const inputOrdinary = document.getElementById(ordinaryId);
        const lableEnhanced =
document.querySelector(`label[for="${enhancedId}"]`);
        const lableOrdinary =
document.querySelector(`label[for="${ordinaryId}"]`);

        if (inputId === 'fundingTypeContract' &&
inputElement.checked) {
            inputEnhanced.checked = false;
            inputEnhanced.disabled = true;
            lableEnhanced.classList.replace('btn-outline-info',
'btn-outline-secondary');

            inputOrdinary.checked = false;
            inputOrdinary.disabled = true;
            lableOrdinary.classList.replace('btn-outline-info',
'btn-outline-secondary');

document.getElementById('scholarshipStatusNone').checked = true;
        }

        if (inputId === 'fundingTypeBudget' &&
inputElement.checked) {
            inputEnhanced.disabled = false;

lableEnhanced.classList.replace('btn-outline-secondary',
'btn-outline-info');

            inputOrdinary.disabled = false;

lableOrdinary.classList.replace('btn-outline-secondary',
'btn-outline-info');
        }
    }
}

```

```

    function submitConfirmation() {
        const message = 'Are you sure you want to delete that
record?';

        if (confirm(message) === true) {
            document.getElementById('studentForm').submit();
        }
    }
}
</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>
</body>
</html>

```

Г.3 Код класы StudentController.java

```

package ntukhpi.csit.semit.riv.webappsrivlab3.controller;

import jakarta.validation.Valid;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.entrant.Entrant;
import
ntukhpi.csit.semit.riv.webappsrivlab3.model.student.Student;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.EntrantService;
import
ntukhpi.csit.semit.riv.webappsrivlab3.service.StudentService;
import org.slf4j.Logger;

```

```

import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import
org.springframework.web.servlet.mvc.support.RedirectAttributes;
import org.springframework.web.util.UriComponentsBuilder;

import java.util.List;

@Controller
public class StudentController {
    private static final Logger logger =
LoggerFactory.getLogger(StudentController.class);
    private StudentService studentService;
    private EntrantService entrantService;

    public StudentController(StudentService studentService,
EntrantService entrantService) {
        super();
        this.studentService = studentService;
        this.entrantService = entrantService;
    }

    @GetMapping("/students")
    public String showStudentTable(
        @RequestParam(value = "search", required = false)
String search,
        @RequestParam(value = "fundingType", required = false)
String fundingType,
        @RequestParam(value = "scholarshipStatus", required =
false) String scholarshipStatus,
        @RequestParam(value = "sort", required = false,
defaultValue = "id-asc") String sort,
        Model model) {

```

```

        model.addAttribute("fields",
Student.getFieldNamesAsFormattedStrings());

        model.addAttribute("mode", Mode.VIEW_TABLE);

        try {
            List<Student> students =
studentService.getFilteredAndSortedStudents(
                search,
                fundingType,
                scholarshipStatus,
                sort);

            model.addAttribute("students", students);
        } catch (Exception e) {
            model.addAttribute("action", "load table");
            model.addAttribute("errorMessage", e.getMessage());
        }

        return "entity/student/StudentTable";
    }

@GetMapping("/students/add")
public String showCreateStudentForm(Model model) {
    Student student = new Student();
    /*student.setEntrant(entrantService.findEntrantById(35L));
    student.setFundingType(FundingType.BUDGET);
    student.setScholarshipStatus(ScholarshipStatus.ORDINARY);

student.setCorporateEmail("regina.marchyk@cs.khpi.edu.ua");*/

    List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

    model.addAttribute("student", student);
    model.addAttribute("entrantList", entrantList);
    model.addAttribute("mode", Mode.ADD);

```

```

        return "entity/student/StudentForm";
    }

    @PostMapping("/students/add")
    public String saveStudent(@Valid @ModelAttribute("student")
Student studentToSave,

                                BindingResult result,
                                RedirectAttributes
redirectAttributes,

                                Model model) {
        model.addAttribute("mode", Mode.ADD);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
IllegalArgumentException(result.getAllErrors().toString());
            }

            Student savedStudent =
studentService.saveStudent(studentToSave);

            redirectAttributes.addFlashAttribute("student",
savedStudent);
            redirectAttributes.addFlashAttribute("successMessage",
"The record successfully saved to the database.");
            redirectAttributes.addFlashAttribute("isSaved", true);

            String redirectUrl =
UriComponentsBuilder.fromPath("/students/{id}/edit")
                .buildAndExpand(savedStudent.getId())
                .toUriString();

            return "redirect:" + redirectUrl;

```

```

        } catch (Exception e) {
            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("entrantList", entrantList);
            model.addAttribute("student", studentToSave);
            model.addAttribute("action", "save");
            model.addAttribute("errorMessage", e.getMessage());

            return "entity/student/StudentForm";
        }
    }

    @GetMapping("/students/{id}/view")
    public String showViewStudentForm(@PathVariable Long id,
                                      Model model) {
        model.addAttribute("mode", Mode.VIEW_FORM);

        try {
            Student studentToUpdate =
studentService.findStudentById(id);
            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("student", studentToUpdate);
            model.addAttribute("entrantList", entrantList);

            return "entity/student/StudentForm";
        } catch (Exception e) {
            model.addAttribute("errorTitle", "Page Not Found
Error");
            model.addAttribute("errorCode", 404);
            model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
            model.addAttribute("errorException", e.getMessage());

```

```

        return "error/ErrorMessage";
    }
}

@GetMapping("/students/{id}/edit")
public String showUpdateStudentForm(@PathVariable Long id,
                                     Model model) {
    model.addAttribute("mode", Mode.EDIT);

    try {
        Student studentToUpdate =
studentService.findStudentById(id);
        List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

        model.addAttribute("student", studentToUpdate);
        model.addAttribute("entrantList", entrantList);

        return "entity/student/StudentForm";
    } catch (Exception e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");
        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorMessage";
    }
}

@PostMapping("/students/{id}/edit")
public String updateStudent(@Valid @ModelAttribute("student")
Student studentToUpdate,
                             BindingResult result,
                             Model model) {

```

```

        model.addAttribute("mode", Mode.EDIT);

        try {
            if (result.hasErrors()) {
                result.getAllErrors().forEach(e ->
logger.error(e.toString()));
                throw new
IllegalArgumentException(result.getAllErrors().toString());
            }

            Student updatedStudent =
studentService.updateStudent(studentToUpdate);
            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("student", updatedStudent);
            model.addAttribute("entrantList", entrantList);
            model.addAttribute("successMessage", "The record
successfully updated in the database.");

            return "entity/student/StudentForm";
        } catch (Exception e) {
            List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

            model.addAttribute("entrantList", entrantList);
            model.addAttribute("student", studentToUpdate);
            model.addAttribute("action", "update");
            model.addAttribute("errorMessage", e.getMessage());

            return "entity/student/StudentForm";
        }
    }

    @GetMapping("/students/{id}/delete")
    public String showDeleteStudentForm(@PathVariable Long id,

```



```

                                Model model) {

    model.addAttribute("mode", Mode.DELETE);

    try {
        Student studentToDelete =
studentService.findStudentById(id);
        List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

        model.addAttribute("entrantList", entrantList);
        model.addAttribute("student", studentToDelete);

        return "entity/student/StudentForm";
    } catch (Exception e) {
        model.addAttribute("errorTitle", "Page Not Found
Error");

        model.addAttribute("errorCode", 404);
        model.addAttribute("errorMessage", "Oops! Looks like
the page doesn't exist.");
        model.addAttribute("errorException", e.getMessage());

        return "error/ErrorPage";
    }
}

@PostMapping("/students/{id}/delete")
public String deleteStudent(@Valid @ModelAttribute("student")
Student studentToDelete,

                                BindingResult result,
                                RedirectAttributes
redirectAttributes,

                                Model model) {

    model.addAttribute("mode", Mode.DELETE);

    try {
        if (result.hasErrors()) {

```

```

        result.getAllErrors().forEach(e ->
logger.error(e.toString()));
        throw new
IllegalArgumentException(result.getAllErrors().toString());
    }

    studentService.deleteStudent(studentToDelete);

    redirectAttributes.addFlashAttribute("successMessage",
"The record successfully deleted from the database.");

    return "redirect:/students";
} catch (Exception e) {
    List<Entrant> entrantList =
entrantService.findEntrantsWithoutStudents();

    model.addAttribute("entrantList", entrantList);
    model.addAttribute("student", studentToDelete);
    model.addAttribute("action", "delete");
    model.addAttribute("errorMessage", e.getMessage());

    return "entity/student/StudentForm";
}
}
}

```

Г.4 Результати роботи вебзастосунку відповідно до реалізації коду до пунктів 3 та 4 лабораторної роботи

The screenshot displays a web application titled "Student Database Viewer" running on a browser at the URL "localhost:8080/students". The application features a sidebar with a search bar, filters for Funding Type (Budget, Contract), Scholarship Status (Enhanced, Ordinary, None), and an "Add record" button. The main area shows a table titled "Student Table" with columns: #, Full Name, Funding Type, Scholarship Status, Corporate Email, and Operations. The table contains 29 student records.

#	Full Name	Funding Type	Scholarship Status	Corporate Email	Operations
1	Барвний Лук'ян Ярославович	бюджет	звичайна	oleksii.babichiev@cs.khpi.edu.ua	[edit] [delete]
2	Гесенко Анна Олексіївна	бюджет	відсутня	anna.hiejenko@cs.khpi.edu.ua	[edit] [delete]
3	Голопоров Микита Сергійович	бюджет	звичайна	mykyta.holoporov@cs.khpi.edu.ua	[edit] [delete]
4	Грищенко Максим Георгійович	бюджет	підвищена	maksym.hrytsenko@cs.khpi.edu.ua	[edit] [delete]
5	Жарий Вадим Віталійович	контракт	відсутня	vadym.zharyi@cs.khpi.edu.ua	[edit] [delete]
6	Жупанов Андрій Володимирович	бюджет	звичайна	andrii.zhupanov@cs.khpi.edu.ua	[edit] [delete]
7	Кльоз Михайло Романович	контракт	відсутня	mykhailo.kloz@cs.khpi.edu.ua	[edit] [delete]
8	Коваль Микита Артемович	бюджет	відсутня	mykyta.koval@cs.khpi.edu.ua	[edit] [delete]
9	Ковтун Єгор Вячеславович	бюджет	підвищена	yehor.kovtun@cs.khpi.edu.ua	[edit] [delete]
10	Колесниченко Денис Євгенович	бюджет	звичайна	denys.kolesnychenko@cs.khpi.edu.ua	[edit] [delete]
11	Ласкевич Андрій Сергійович	бюджет	звичайна	andrii.laskevych@cs.khpi.edu.ua	[edit] [delete]
12	Лукаш Антон Олександрович	бюджет	звичайна	anton.lukash@cs.khpi.edu.ua	[edit] [delete]
13	Палій Нікіта Андрійович	бюджет	відсутня	nikita.palii@cs.khpi.edu.ua	[edit] [delete]
14	Прокопов Владислав Сергійович	бюджет	звичайна	vladyslav.prokopov@cs.khpi.edu.ua	[edit] [delete]
15	Репешко Інесса Віталіївна	бюджет	підвищена	inessa.repeshko@cs.khpi.edu.ua	[edit] [delete]
16	Скиба Андрій Григорович	контракт	відсутня	andrii.h.skyba@cit.khpi.edu.ua	[edit] [delete]
17	Цибань Ольга Юріївна	бюджет	відсутня	olha.tsyban@cs.khpi.edu.ua	[edit] [delete]
18	Шевченко Данило Валерійович	бюджет	звичайна	danylo.v.shevchenko@cs.khpi.edu.ua	[edit] [delete]
19	Антипенко Катерина Віталіївна	бюджет	звичайна	kateryna.antypenko@cs.khpi.edu.ua	[edit] [delete]
20	Багрянцева Марина Русланівна	бюджет	звичайна	marina.bahriantseva@cs.khpi.edu.ua	[edit] [delete]
21	Бондаренко Ярослав Юрійович	бюджет	підвищена	yaroslav.y.bondarenko@cs.khpi.edu.ua	[edit] [delete]
22	Булгаков Віктор Олександрович	бюджет	відсутня	viktor.bulhakov@cs.khpi.edu.ua	[edit] [delete]
23	Васильєв Михайло Олександрович	бюджет	відсутня	mykhailo.vasyliiev@cs.khpi.edu.ua	[edit] [delete]
24	Деркач Дарина Євгенівна	бюджет	звичайна	daryna.derkach@cs.khpi.edu.ua	[edit] [delete]
25	Ілюхін Тимур Ігорович	контракт	відсутня	tymur.iluikhin@cs.khpi.edu.ua	[edit] [delete]
26	Криженко Аліна Степанівна	бюджет	відсутня	alina.kryzhenko@cs.khpi.edu.ua	[edit] [delete]
27	Посмашний Ярослав Дмитрович	бюджет	звичайна	yaroslav.posmashnyi@cs.khpi.edu.ua	[edit] [delete]
28	Рожкова Вікторія Андріївна	бюджет	підвищена	viktoria.rozhkova@cs.khpi.edu.ua	[edit] [delete]
29	Сизоненко Єлизавета Олександрівна	контракт	відсутня	yelyzaveta.syzonenko@cs.khpi.edu.ua	[edit] [delete]

Powered by Inessa Repeshko CS-222a CSIT SEMIT NTU "KHPI"

Address National Technical University "Kharkiv Polytechnic Institute" 2 Kyrychova 61002 Kharkiv, Ukraine

Contact Us +38-(057)-787-66-34 omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Г.4.1 – Сторінка таблиці студентів на URL

“http://localhost:8080/students”

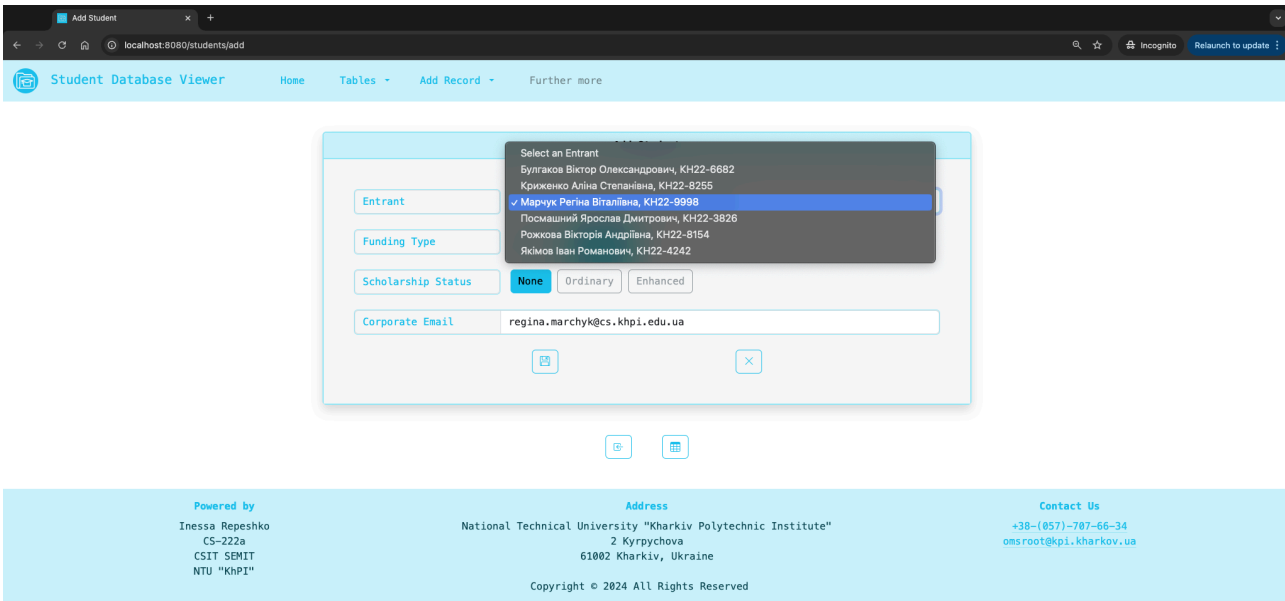


Рисунок Г.4.2 – Сторінка форми додавання студента зі списком доступних абітурієнтів на URL “http://localhost:8080/students/add”

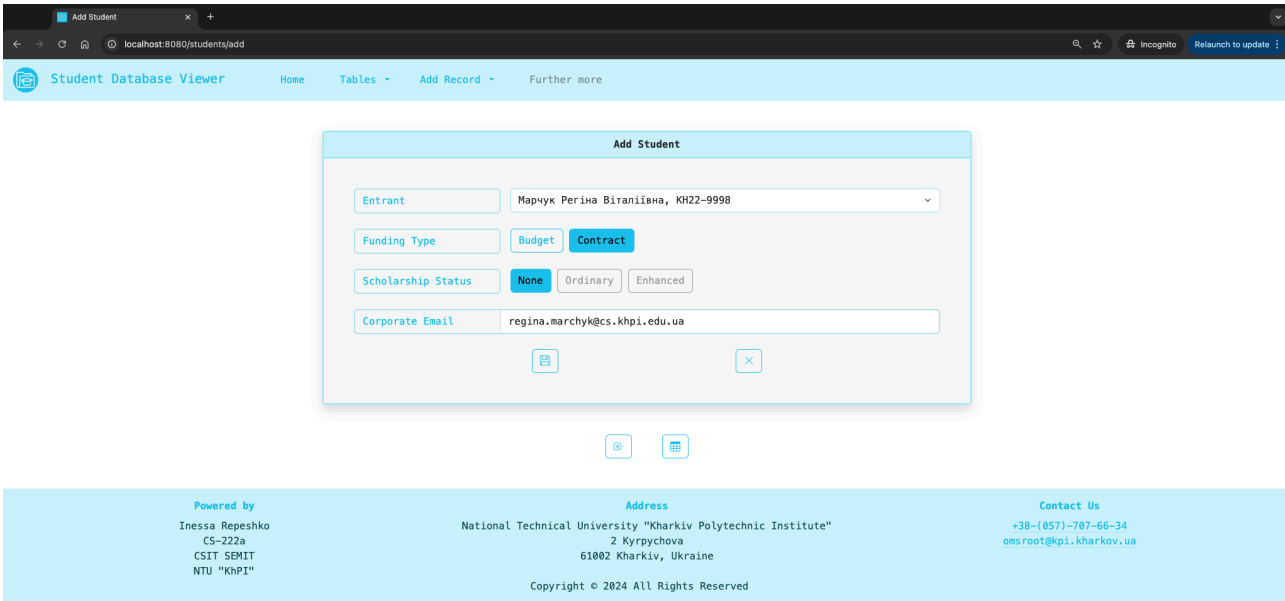


Рисунок Г.4.3 – Сторінка форми додавання студента на URL “http://localhost:8080/students/add”

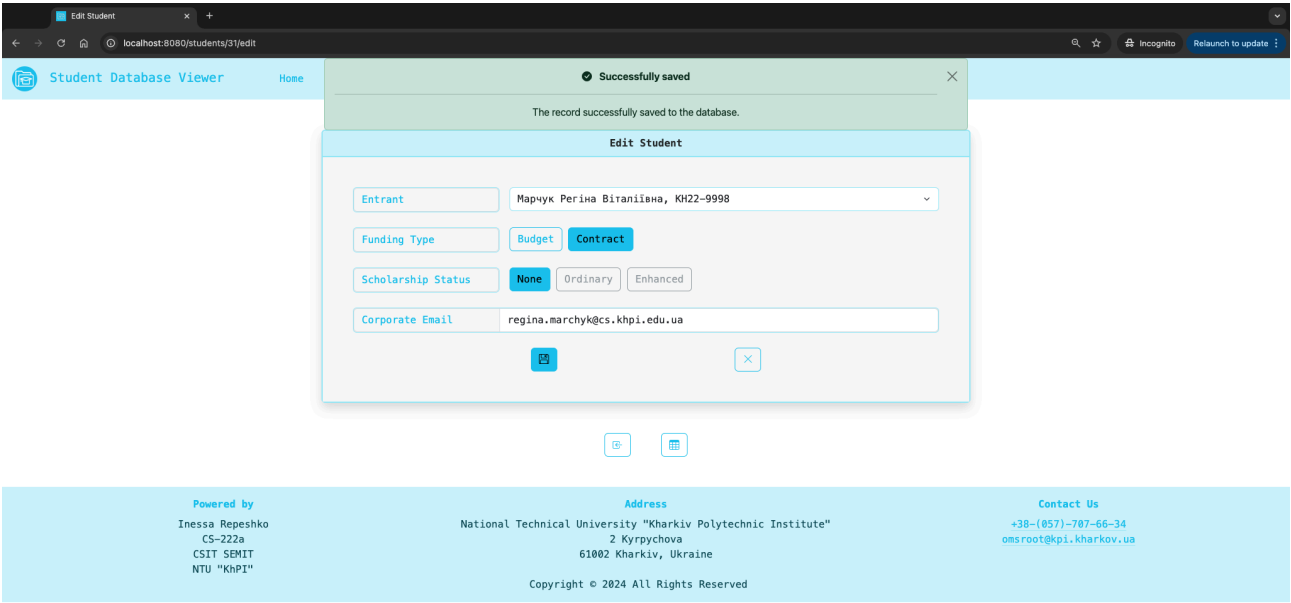


Рисунок Г.4.4 – Сторінка форми додавання студента після збереження даних на URL “`http://localhost:8080/students/31/edit`”

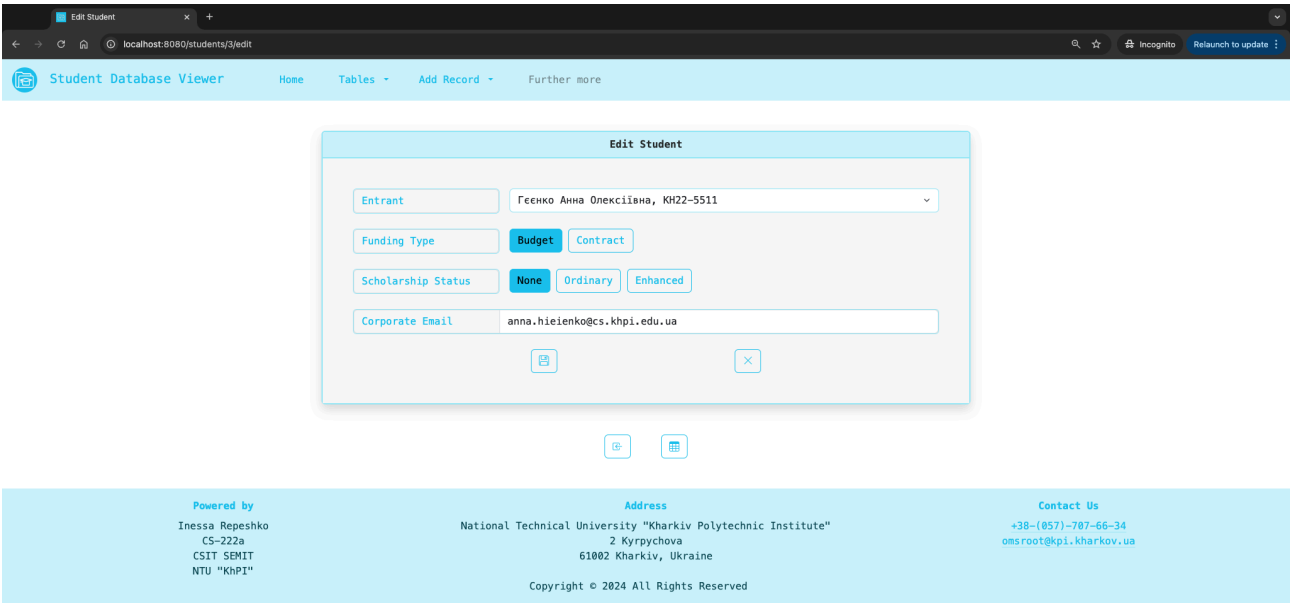


Рисунок Г.4.5 – Сторінка форми редагування студента на URL “`http://localhost:8080/students/3/edit`”

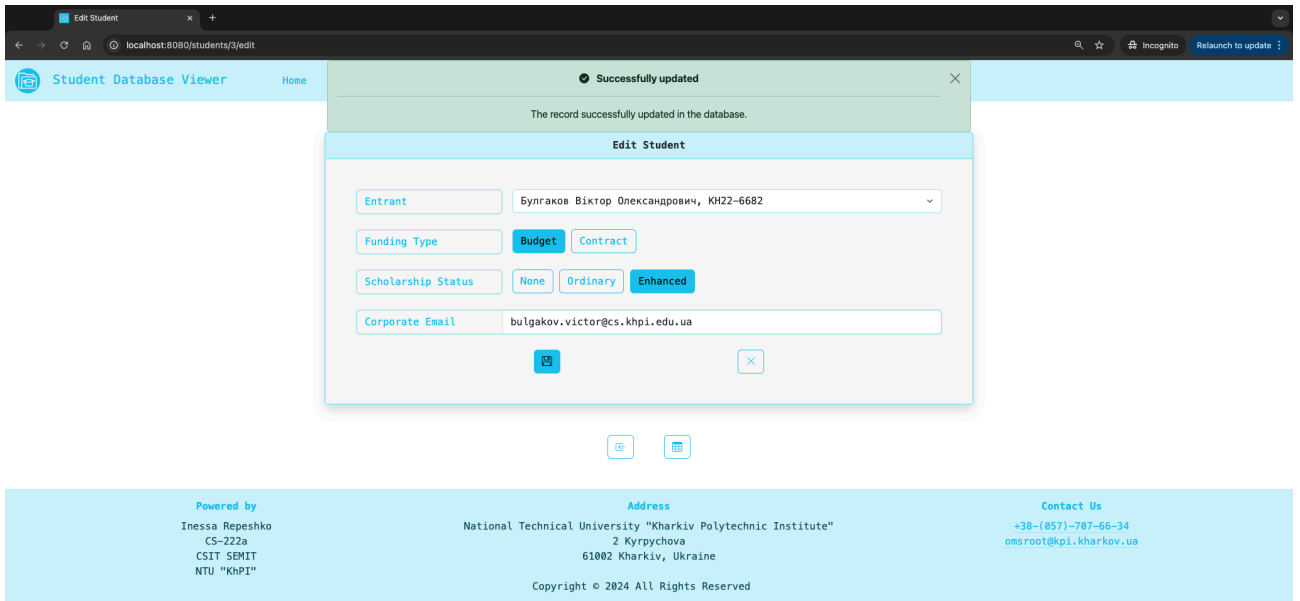


Рисунок Г.4.6 – Сторінка форми редагування студента після збереження даних на URL “`http://localhost:8080/students/3/edit`”

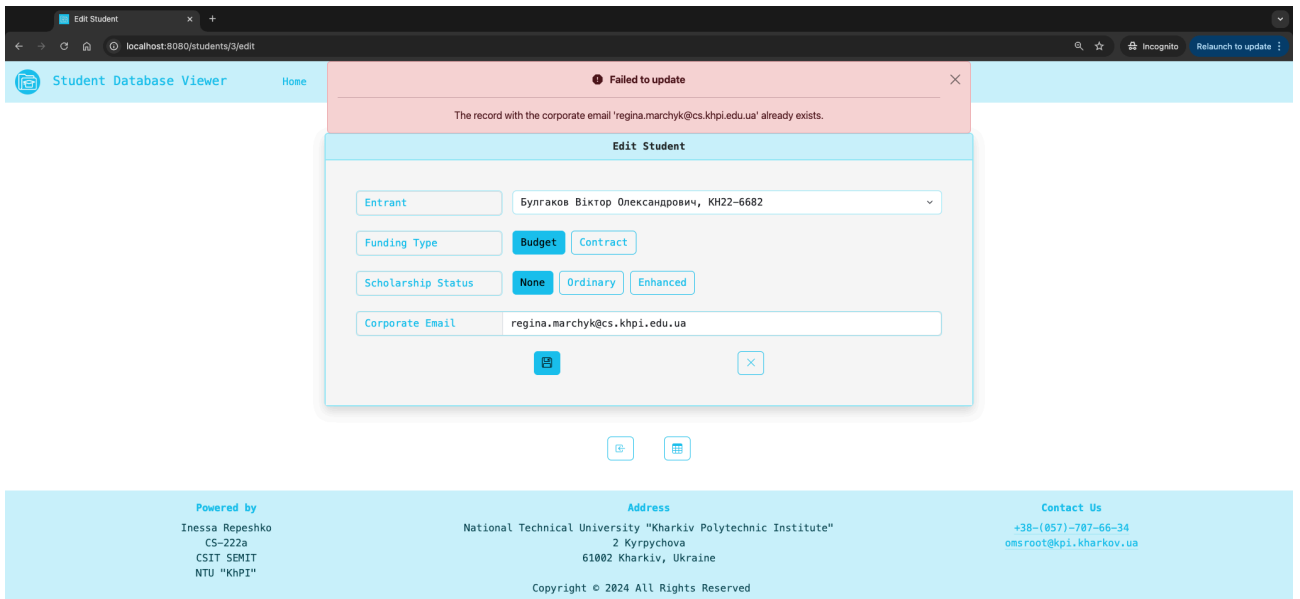


Рисунок Г.4.7 – Сторінка форми редагування студента на URL з помилкою дубліката “`http://localhost:8080/students/3/edit`”

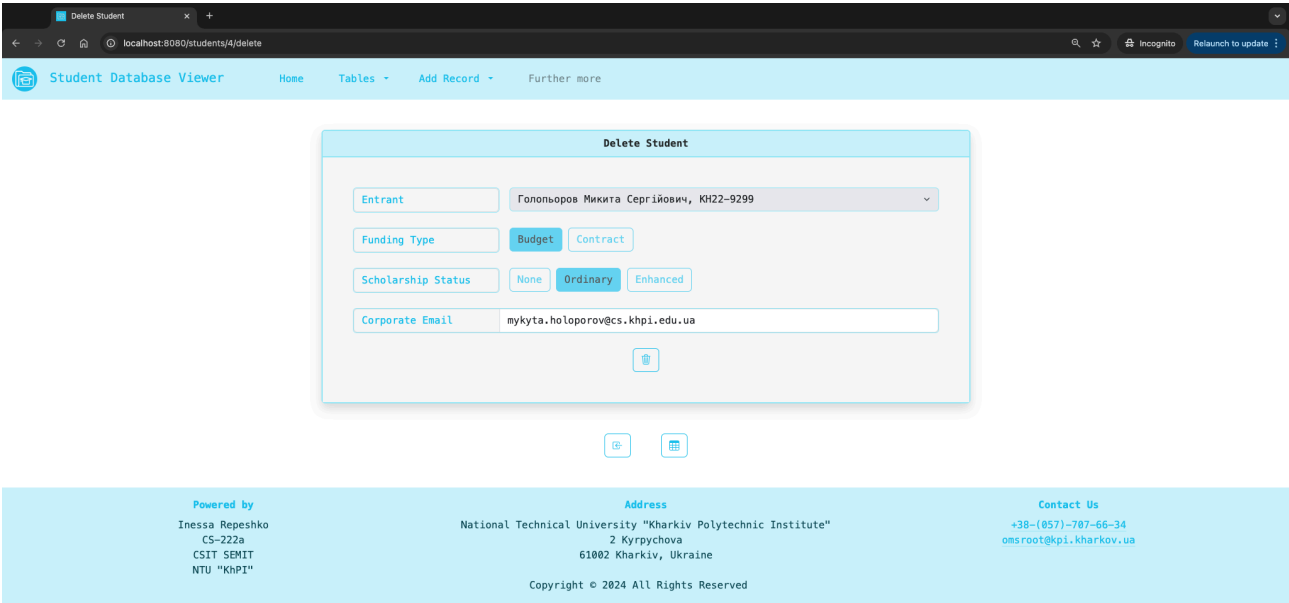


Рисунок Г.4.8 – Сторінка форми видалення студента на URL “http://localhost:8080/students/4/delete”

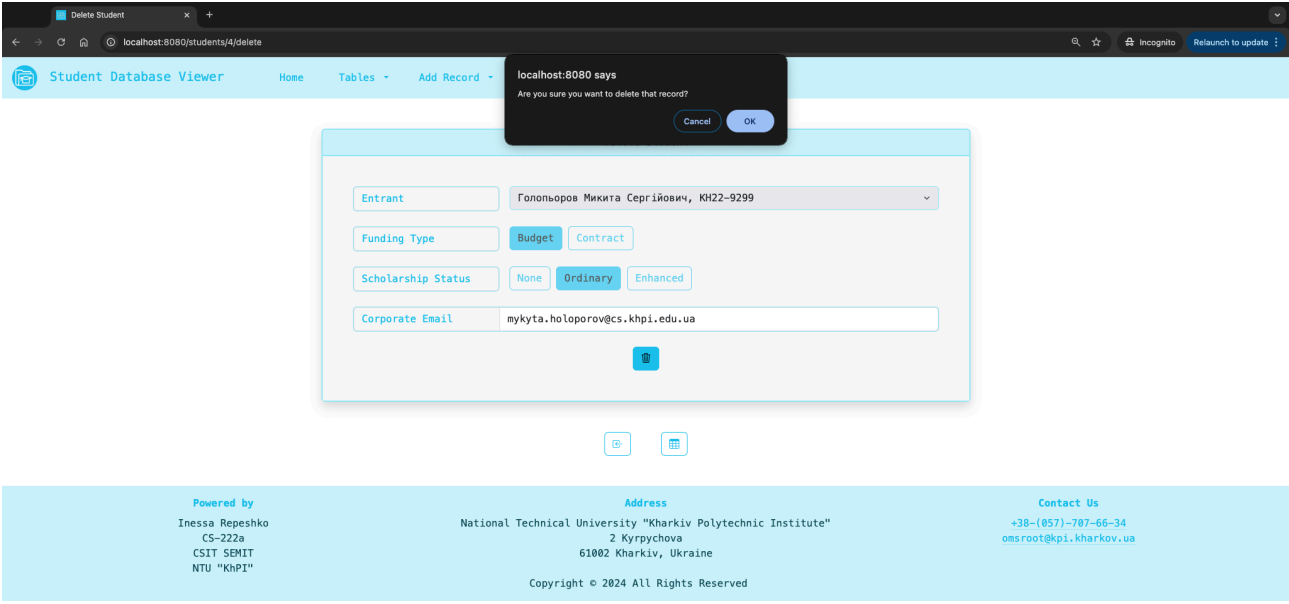


Рисунок Г.4.9 – Сторінка форми видалення студента з запитом підтвердження на URL “http://localhost:8080/students/4/delete”

Student Table

localhost:8080/students

Student Database Viewer

None

Successfully deleted

The record successfully deleted from the database.

Search

Search by full name (matching start, case-insensitive)

Type to search...

Q X

Filters

Funding Type

Budget

Contract

▽

X

Scholarship Status

Enhanced

Ordinary

None

▽

X

Add record

Add new record to the end of the table

+

#	Full Name	Funding Type	Scholarship Status	Corporate Email	Operations
1	Бараний Лук'ян Ярославович	бюджет	звичайна	oleksii.babichiev@cs.khpi.edu.ua	🔍 ✎ 🗑
2	Булгаков Віктор Олександрович	бюджет	підвищена	bulgakov.victor@cs.khpi.edu.ua	🔍 ✎ 🗑
3	Гриценко Максим Георгійович	бюджет	підвищена	maksym.hrytsenko@cs.khpi.edu.ua	🔍 ✎ 🗑
4	Жарий Вадим Віталійович	контракт	відсутня	vadym.zharyi@cs.khpi.edu.ua	🔍 ✎ 🗑
5	Жупанов Андрій Володимирович	бюджет	звичайна	andrii.zhupanov@cs.khpi.edu.ua	🔍 ✎ 🗑
6	Клюз Михайло Романович	контракт	відсутня	mykhailo.kloz@cs.khpi.edu.ua	🔍 ✎ 🗑
7	Коваль Микита Артемович	бюджет	відсутня	mykyta.koval@cs.khpi.edu.ua	🔍 ✎ 🗑
8	Ковтун Єгор Вячеславович	бюджет	підвищена	yehor.kovtun@cs.khpi.edu.ua	🔍 ✎ 🗑
9	Колесниченко Денис Євгенович	бюджет	звичайна	denys.kolesnychenko@cs.khpi.edu.ua	🔍 ✎ 🗑
10	Ласкевич Андрій Сергійович	бюджет	звичайна	andrii.laskevych@cs.khpi.edu.ua	🔍 ✎ 🗑
11	Лукаш Антон Олександрович	бюджет	звичайна	anton.lukash@cs.khpi.edu.ua	🔍 ✎ 🗑
12	Палій Нікіта Андрійович	бюджет	відсутня	nikita.palii@cs.khpi.edu.ua	🔍 ✎ 🗑
13	Прокопов Владислав Сергійович	бюджет	звичайна	vladyslav.prokopov@cs.khpi.edu.ua	🔍 ✎ 🗑
14	Репешко Інесса Віталіївна	бюджет	підвищена	inessa.repeshko@cs.khpi.edu.ua	🔍 ✎ 🗑
15	Скиба Андрій Григорович	контракт	відсутня	andrii.h.skyba@cit.khpi.edu.ua	🔍 ✎ 🗑
16	Цибань Ольга Юрііана	бюджет	відсутня	olha.tsyban@cs.khpi.edu.ua	🔍 ✎ 🗑
17	Шевченко Данило Валерійович	бюджет	звичайна	danylo.v.shevchenko@cs.khpi.edu.ua	🔍 ✎ 🗑
18	Антипенко Катерина Віталіївна	бюджет	звичайна	kateryna.antypenko@cs.khpi.edu.ua	🔍 ✎ 🗑
19	Багрянцева Марина Русланівна	бюджет	звичайна	marina.bahriantseva@cs.khpi.edu.ua	🔍 ✎ 🗑
20	Бондаренко Ярослав Юрійович	бюджет	підвищена	yaroslav.y.bondarenko@cs.khpi.edu.ua	🔍 ✎ 🗑
21	Булгаков Віктор Олександрович	бюджет	відсутня	viktor.bulhakov@cs.khpi.edu.ua	🔍 ✎ 🗑
22	Васильєв Михайло Олександрович	бюджет	відсутня	mykhailo.vasyliiev@cs.khpi.edu.ua	🔍 ✎ 🗑
23	Деркач Дарина Євгенівна	бюджет	звичайна	daryna.derkach@cs.khpi.edu.ua	🔍 ✎ 🗑
24	Ілюхін Тимур Ігорович	контракт	відсутня	tymur.iluikhin@cs.khpi.edu.ua	🔍 ✎ 🗑
25	Криженко Аліна Степанівна	бюджет	відсутня	alina.kryzhenko@cs.khpi.edu.ua	🔍 ✎ 🗑
26	Посмашний Ярослав Дмитрович	бюджет	звичайна	yaroslav.posmashnyi@cs.khpi.edu.ua	🔍 ✎ 🗑
27	Рожкова Вікторія Андріївна	бюджет	підвищена	viktoriia.rozhkova@cs.khpi.edu.ua	🔍 ✎ 🗑
28	Сизоненко Єлизавета Олександрівна	контракт	відсутня	yelyzaveta.syzonenko@cs.khpi.edu.ua	🔍 ✎ 🗑
29	Марчук Регіна Віталіївна	контракт	відсутня	regina.marchyk@cs.khpi.edu.ua	🔍 ✎ 🗑

🔍

Powered by

Inessa Repeshko

CS-222a

CSIT SEMIT

NTU "KhPI"

Address

National Technical University "Kharkiv Polytechnic Institute"

2 Kyrychova

61002 Kharkiv, Ukraine

Contact Us

+38-(057)-707-66-34

omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Г.4.10 – Сторінка таблиці студента з повідомленням про видалення запису на URL “http://localhost:8080/students”

The screenshot shows a web browser window with the address bar displaying `localhost:8080/entrants/34/view`. The application header is 'Student Database Viewer' with navigation links: Home, Tables, Add Record, and Further more. The main content area is titled 'View Entrant' and contains a form with the following fields:

Case Number	КН22-6682
Surname	Булгаков
Name	Віктор
Patronymic	Олександрович
Birthday	dd.mm.yyyy
Gender	☒ Male ☐ Female
Rating score	153,683

Below the form are two icons: a document and a calendar. The footer contains contact information for Inessa Repeshko, CS-222a, CSIT SEMIT, NTU "KhPI", and the National Technical University "Kharkiv Polytechnic Institute".

Powered by
Inessa Repeshko
CS-222a
CSIT SEMIT
NTU "KhPI"

Address
National Technical University "Kharkiv Polytechnic Institute"
2 Кирпичова
61002 Kharkiv, Ukraine

Contact Us
+38-(057)-707-66-34
omsroot@kpi.kharkov.ua

Copyright © 2024 All Rights Reserved

Рисунок Г.4.11 – Сторінка форми перегляду абітурієнта прив’язаного до обраного студента на URL “`http://localhost:8080/entrants/1/view`”